BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ

ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU CÔNG CỤ TỰ ĐỘNG PHÁT HIỆN LỖ HỔNG BẢO MẬT API CỦA ỨNG DỤNG ANDROID

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện: **Nguyễn Đức Tâm – Lớp: AT19A**
Người hướng dẫn: **TS. Phạm Duy Trung**
Khoa An toàn thông tin – Học viện Kỹ thuật mật mã

Hà Nội, 2026

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ

ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU CÔNG CỤ TỰ ĐỘNG PHÁT HIỆN LỖ HỔNG BẢO MẬT API CỦA ỨNG DỤNG ANDROID

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện: **Nguyễn Đức Tâm – Lớp: AT19A**
Người hướng dẫn: **TS. Phạm Duy Trung**
Khoa An toàn thông tin – Học viện Kỹ thuật mật mã

Hà Nội, 2026

MỤC LỤC

Chương 1. Tổng quan về bảo mật API của ứng dụng Android.....15
1.1. Tổng quan về ứng dụng Android và cơ chế giao tiếp API.....15
1.1.1. Kiến trúc giao tiếp và vòng đời một yêu cầu API.....15
1.1.2. Giao thức SSL/TLS và cơ chế Certificate Pinning.....17
1.1.3. Bề mặt tấn công API của ứng dụng Android.....18
1.2. Các lỗ hổng bảo mật API theo chuẩn OWASP.....20
1.2.1. OWASP Mobile Top 10 – 2024.....20
1.2.2. OWASP API Security Top 10 – 2023.....22
1.2.3. Mối quan hệ giữa hai bộ tiêu chuẩn và phạm vi lỗ hổng của đồ án.....23
1.3. Các phương pháp kiểm tra bảo mật API của ứng dụng Android.....24
1.3.1. Phân tích tĩnh.....24
1.3.2. Phân tích động dựa trên máy chủ uỷ nhiệm.....25
1.3.3. Phân tích động bằng can thiệp thời gian chạy.....26
1.3.4. Phân tích kết hợp và so sánh ba phương pháp.....27
1.4. Tổng quan các công cụ tự động phát hiện lỗ hổng bảo mật API.....28
1.4.1. MobSF – khung kiểm thử tự động kết hợp.....29
1.4.2. Burp Suite và OWASP ZAP – hai công cụ uỷ nhiệm.....30
1.4.3. Frida và objection – can thiệp thời gian chạy.....30
1.4.4. JADX và apktool – dịch ngược tệp APK.....31
1.4.5. Tình hình nghiên cứu liên quan và khoảng trống nghiên cứu.....31
1.5. Kết luận chương 1.....33
Chương 2. Công cụ, môi trường thực nghiệm và thiết kế đánh giá.....35
2.1. Phân tích chi tiết các công cụ được lựa chọn.....35
2.1.1. MobSF – khung phân tích tự động kết hợp.....36
2.1.2. Burp Suite Community Edition.....38
2.1.3. OWASP ZAP.....39
2.1.4. Frida và objection.....40
2.1.5. JADX và apktool.....42
2.1.6. So sánh tổng hợp và cơ sở phân công vai trò.....43

2.2. Ứng dụng đích InsecureBankv2.....44
2.2.1. Lý do lựa chọn ứng dụng đích.....44
2.2.2. Kiến trúc và luồng nghiệp vụ.....45
2.2.3. Các điểm cuối API.....47
2.2.4. Mô hình mối đe doạ.....48
2.2.5. Danh sách lỗ hổng đã biết.....49
2.3. Xây dựng môi trường thực nghiệm.....51
2.3.1. Cấu hình phần cứng và phần mềm.....52
2.3.2. Máy ảo Android và cấu hình mạng.....53
2.3.3. Triển khai từng công cụ.....54
2.3.4. Kiến trúc tổng thể môi trường lab.....55
2.3.5. Quy trình đưa môi trường về trạng thái sạch.....56
2.3.6. Các sự cố thường gặp và cách xử lý.....57
2.4. Thiết kế các kịch bản kiểm thử.....57
2.4.1. Minh hoạ chi tiết: kịch bản TC-08.....64
2.5. Khung đánh giá và quy trình benchmark.....65
2.5.1. Ma trận kiểm thử.....65
2.5.2. Quy ước gán nhãn kết quả.....66
2.5.3. Bảy tiêu chí đánh giá benchmark.....67
2.5.4. Cách tổng hợp kết quả và pipeline tự động hoá.....69
2.5.5. Quy trình thực hiện benchmark.....70
2.5.6. Các mối đe doạ tới tính hợp lệ của phép đo.....71
2.6. Kết luận chương 2.....72
CHƯƠNG 3. XÂY DỰNG VÀ ĐÁNH GIÁ CÔNG CỤ PHÁT HIỆN LỖ HỔNG TĨNH APISCAN.....74
3.1. Bối cảnh và câu hỏi nghiên cứu.....74
3.2. Thiết kế công cụ apiscan.....75
3.2.1. Kiến trúc tổng thể.....75
3.2.2. Bộ luật và ánh xạ OWASP.....76
3.2.3. Các kỹ thuật giảm báo nhầm theo ngữ cảnh.....76
3.2.4. Tự xác định phạm vi mã của ứng dụng.....78

3.2.5. Điểm tin cậy theo từng cảnh báo....79
3.2.6. Đầu ra của công cụ....80
3.3. Phương pháp đánh giá....81
3.3.1. Bộ đáp án InsecureBankv2 và hiệu chỉnh đáp án....81
3.3.2. Tập ứng dụng thật....82
3.3.3. Quy trình gán nhãn....83
3.3.4. Giao thức đánh giá holdout....84
3.3.5. Chỉ số đánh giá....84
3.4. Kết quả....84
3.4.1. Recall trên InsecureBankv2 (RQ1)....84
3.4.2. Precision trên tập phát triển....85
3.4.3. Precision trên holdout (RQ2)....85
3.4.4. Hiệu quả của điểm tin cậy (RQ3)....87
3.4.5. Phân tích báo nhầm theo luật và theo miền ứng dụng....88
3.4.6. Các phát hiện bảo mật trên ứng dụng thật....89
3.4.7. Tốc độ....90
3.5. Đối chứng với công cụ sẵn có....90
3.6. Recall trên lỗ hổng đã công bố....93
3.7. Quét động có dẫn đường trên InsecureBankv2....95
3.8. Thảo luận và hạn chế....97
3.9. Kết luận chương....98
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN....99
Kết quả đạt được....99
Hạn chế....102
Hướng phát triển....102
Tài liệu tham khảo....104
Phụ lục....107
Phụ lục A. Kịch bản Frida vô hiệu hoá ghim chứng chỉ....107
Phụ lục B. Các lệnh triển khai môi trường lab....107
Phụ lục C. Cấu trúc thư mục lưu kết quả....108
Phụ lục D. Mẫu phiếu ghi kết quả một ca kiểm thử....109

DANH MỤC KÍ HIỆU VÀ VIẾT TẮT

Kí hiệu	Nghĩa tiếng Anh
ADB	Android Debug Bridge
AES	Advanced Encryption Standard
API	Application Programming Interface
APK	Android Package
AVD	Android Virtual Device
BOLA	Broken Object Level Authorization
CA	Certificate Authority
CBC	Cipher Block Chaining
CI/CD	Continuous Integration / Continuous Delivery
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CWE	Common Weakness Enumeration
FN	False Negative
FP	False Positive
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IDOR	Insecure Direct Object Reference
IV	Initialization Vector
JSON	JavaScript Object Notation
JWT	JSON Web Token
KTC	Confidence Interval (khoảng tin cậy)
MASTG	Mobile Application Security Testing Guide
MASVS	Mobile Application Security Verification Standard
MitM	Man-in-the-Middle
MobSF	Mobile Security Framework
OWASP	Open Worldwide Application Security Project
REST	Representational State Transfer
RQ	Research Question (câu hỏi nghiên cứu)
SDK	Software Development Kit
SSL	Secure Sockets Layer
SSRF	Server Side Request Forgery
TLS	Transport Layer Security
TN	True Negative
TP	True Positive
URL	Uniform Resource Locator

Kí hiệu	Nghĩa tiếng Anh
XML	eXtensible Markup Language
XMPP	Extensible Messaging and Presence Protocol
ZAP	Zed Attack Proxy

DANH MỤC HÌNH VẼ

Hình 1.1. Kiến trúc giao tiếp API theo mô hình client – server của ứng dụng Android

Hình 1.2. Bề mặt tấn công API của ứng dụng Android phân theo ba tầng

Hình 1.3. Tấn công xen giữa và vai trò của cơ chế Certificate Pinning

Hình 1.4. Phân loại các công cụ tự động phát hiện lỗ hổng bảo mật API trên Android

Hình 2.1. Sơ đồ quyết định lựa chọn công cụ theo đặc trưng của lỗ hổng

Hình 2.2. Luồng nghiệp vụ chính của InsecureBankv2

Hình 2.3. Mô hình mối đe doạ đối với ứng dụng đích

Hình 2.4. Phân bố lỗ hổng theo hạng mục OWASP và theo phương pháp phát hiện

Hình 2.5. Kiến trúc tổng thể môi trường thực nghiệm

Hình 2.6. Trình tự thực hiện kịch bản kiểm thử TC-08 (phân quyền mức đối tượng)

Hình 2.7. Quy ước phân loại kết quả TP, FP, FN, TN

Hình 2.8. Kiến trúc pipeline tự động hoá thu thập và tổng hợp kết quả

Hình 2.9. Quy trình thực hiện benchmark theo bốn giai đoạn

Hình 2.10. Luồng kiểm thử tĩnh và động trên ứng dụng đích

Hình 3.1. Kiến trúc của công cụ apiscan

Hình 3.2. Một phần báo cáo HTML của apiscan trên InsecureBankv2

Hình 3.3. Precision trên tập phát triển (tại thời điểm đóng băng) và trên holdout qua các đợt

Hình 3.4. TP và FP theo tầng tin cậy: (a) 19 ứng dụng thật; (b) gộp holdout đợt 7–9

DANH MỤC BẢNG

Bảng 1.1. Danh sách OWASP Mobile Top 10 – 2024

Bảng 1.2. Danh sách OWASP API Security Top 10 – 2023

Bảng 1.3. So sánh khả năng phát hiện lỗ hổng của ba phương pháp kiểm tra

Bảng 1.4. Tổng quan bảy công cụ tự động phát hiện lỗ hổng bảo mật API trên Android

Bảng 1.5. Tổng hợp các công trình nghiên cứu liên quan tiêu biểu

Bảng 2.1. Đặc trưng kỹ thuật của bảy công cụ được lựa chọn

Bảng 2.2. Các điểm cuối REST API của MobSF dùng trong thực nghiệm

Bảng 2.3. Các thành phần của Burp Suite Community sử dụng trong kiểm thử API

Bảng 2.4. Các lời gọi REST API của OWASP ZAP dùng trong thực nghiệm

Bảng 2.5. Các nhóm lệnh của objection sử dụng trong thực nghiệm

Bảng 2.6. So sánh tổng hợp bảy công cụ theo năm chiều

Bảng 2.7. Các điểm cuối API chính của InsecureBankv2

Bảng 2.8. Tài sản cần bảo vệ và hệ quả khi bị xâm phạm

Bảng 2.9. Tác nhân đe doạ và kịch bản tấn công tương ứng

Bảng 2.10. Danh sách lỗ hổng đã biết trong InsecureBankv2

Bảng 2.11. Cấu hình phần cứng và phần mềm của môi trường thực nghiệm

Bảng 2.12. Cổng dịch vụ và cách truy cập trong môi trường lab

Bảng 2.13. Trình tự triển khai và cách xác nhận từng bước

Bảng 2.14. Các sự cố thường gặp khi thiết lập môi trường và cách xử lý

Bảng 2.15. Ma trận kiểm thử: công cụ × lỗ hổng

Bảng 2.16. Bảy tiêu chí đánh giá benchmark và cách đo

Bảng 2.17. Thang neo điểm cho tiêu chí mức độ dễ sử dụng

Bảng 2.18. Các mối đe doạ tới tính hợp lệ và biện pháp giảm thiểu

Bảng 3.1. Bộ luật của apiscan theo nhóm OWASP Mobile Top 10 – 2024

Bảng 3.2. Một số quy tắc chấm độ tin cậy theo tín hiệu của cảnh báo

Bảng 3.3. Hiệu chỉnh đáp án InsecureBankv2 sau khi đối chiếu mã nguồn

Bảng 3.4. Tập ứng dụng thật và vai trò trong đánh giá

Bảng 3.5. Precision trên NewPipe qua các cải tiến luật

Bảng 3.6. Precision trên holdout theo đợt

Bảng 3.7. Precision theo tầng tin cậy, phiên bản công cụ cuối trên 19 ứng dụng thật (331 cảnh báo)

Bảng 3.8. Precision theo luật, phiên bản công cụ cuối trên 19 ứng dụng thật

Bảng 3.9. Một số phát hiện bảo mật đáng chú ý

Bảng 3.10. Đối chứng apiscan với MobSF trên cùng tập ứng dụng và cùng định nghĩa nhãn

Bảng 3.11. Recall của apiscan trên 10 lỗ hổng đã công bố

Bảng 3.12. Kết quả ZAP trên backend InsecureBankv2 theo đầu vào dẫn đường

LỜI CẢM ƠN

Em xin bày tỏ lòng biết ơn sâu sắc tới Ban Giám đốc Học viện Kỹ thuật mật mã và các thầy cô trong Khoa An toàn thông tin đã tận tình giảng dạy, trang bị cho em nền tảng kiến thức chuyên ngành trong suốt những năm học vừa qua.

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới thầy TS. Phạm Duy Trung — giảng viên Khoa An toàn thông tin, Học viện Kỹ thuật mật mã. Thầy đã góp ý để em định lượng hoá khung đánh giá thay vì dừng ở nhận xét định tính, và đã đồng ý để em chuyển trọng tâm của đồ án sang xây dựng một công cụ có đầu ra dùng được khi kết quả thực nghiệm ban đầu cho thấy khoảng cách lớn giữa ứng dụng mẫu và ứng dụng thực tế.

Em cũng xin cảm ơn cộng đồng mã nguồn mở OWASP, các nhóm phát triển MobSF, OWASP ZAP, Frida, objection, JADX, apktool, tác giả Dinesh Shetty và những người đóng góp cho dự án InsecureBankv2, cùng các nhà phát triển của 19 ứng dụng mã nguồn mở được dùng để đánh giá — mã nguồn công khai của họ là điều kiện để đồ án có thể đánh giá công cụ trên ứng dụng thực tế. Cuối cùng, em xin cảm ơn gia đình và bạn bè đã luôn động viên em trong suốt quá trình thực hiện đồ án.

Do năng lực và thời gian còn hạn chế, đồ án chắc chắn không tránh khỏi thiếu sót. Em rất mong nhận được sự chỉ bảo, góp ý của các thầy cô và hội đồng.

SINH VIÊN THỰC HIỆN ĐỒ ÁN

Nguyễn Đức Tâm

LỜI NÓI ĐẦU

Ứng dụng Android giao tiếp với máy chủ qua API, và phần lớn rủi ro bảo mật của ứng dụng di động hiện nay nằm ở lớp giao tiếp đó: lưu và truyền thông tin xác thực không an toàn, cấu hình mạng cho phép kết nối không mã hoá, thành phần được mở cho ứng dụng khác gọi tới mà không có kiểm soát. Hai bộ tiêu chuẩn OWASP Mobile Top 10 (2024) và OWASP API Security Top 10 (2023) đã hệ thống hoá các nhóm lỗ hổng này, và nhiều công cụ tự động đã ra đời để phát hiện chúng. Tuy nhiên, các công cụ phân tích tĩnh thường được kiểm chứng trên ứng dụng có lỗ hổng có chủ đích; khi áp dụng lên ứng dụng thực tế, chúng sinh nhiều cảnh báo nhưng tỉ lệ cảnh báo đúng thấp, buộc người kiểm thử phải rà soát thủ công.

Đồ án ban đầu được định hướng như một nghiên cứu so sánh bảy công cụ tự động trên ứng dụng mẫu InsecureBankv2. Trong quá trình thực nghiệm, một quan sát đã làm thay đổi trọng tâm: cùng một bộ luật phân tích tĩnh cho mọi cảnh báo đúng trên InsecureBankv2 nhưng chỉ 1/19 cảnh báo đúng trên ứng dụng mã nguồn mở NewPipe. Với sự đồng ý của giáo viên hướng dẫn, đồ án chuyển sang xây dựng một công cụ phân tích tĩnh có đầu ra dùng được — công cụ apiscan — và một phương pháp đánh giá công cụ trung thực trên ứng dụng thực tế; phần khảo sát và benchmark công cụ được giữ lại làm cơ sở lựa chọn kỹ thuật và đối chứng.

Ngoài phần mở đầu và kết luận, đồ án gồm ba chương:

Chương 1 trình bày tổng quan về bảo mật API của ứng dụng Android: kiến trúc giao tiếp, cơ chế bảo vệ tầng vận chuyển, bề mặt tấn công, hai bộ tiêu chuẩn OWASP, ba phương pháp kiểm tra bảo mật và tổng quan các công cụ tự động, qua đó xác định khoảng trống nghiên cứu.

Chương 2 phân tích chi tiết ranh giới năng lực của từng công cụ, ứng dụng đích InsecureBankv2 cùng danh sách lỗ hổng đã biết, môi trường thực nghiệm trên máy Apple Silicon và khung đánh giá gồm ma trận kiểm thử, quy ước gán nhãn và tiêu chí đo.

Chương 3 trình bày thiết kế và cài đặt công cụ apiscan, các kỹ thuật giảm báo nhầm theo ngữ cảnh, điểm tin cậy theo từng cảnh báo; đánh giá công cụ trên

InsecureBankv2 và 19 ứng dụng mã nguồn mở theo giao thức holdout nhiều đợt, đối chứng với MobSF và thảo luận các hạn chế.

Mọi thực nghiệm trong đồ án chỉ dùng ứng dụng có lỗ hổng có chủ đích và ứng dụng mã nguồn mở, chỉ phân tích tĩnh tệp APK, không gửi yêu cầu tới máy chủ của bên thứ ba. Mọi con số đều từ lần chạy thật và có tệp minh chứng đi kèm mã nguồn của công cụ. Do thời gian thực hiện có hạn và lĩnh vực an toàn ứng dụng di động thay đổi rất nhanh, đồ án chắc chắn còn nhiều thiếu sót. Em rất mong nhận được sự góp ý của các thầy cô để đồ án được hoàn thiện hơn.

SINH VIÊN THỰC HIỆN ĐỒ ÁN

Nguyễn Đức Tâm

CHƯƠNG 1. TỔNG QUAN VỀ BẢO MẬT API CỦA ỨNG DỤNG ANDROID

Chương này trình bày cơ sở lý thuyết nền tảng của đề tài. Trước hết, chương làm rõ cách một ứng dụng Android giao tiếp với máy chủ phía sau thông qua API và những cơ chế bảo vệ được áp dụng trên kênh truyền đó. Trên nền hiểu biết này, chương phân tích bề mặt tấn công API của ứng dụng Android, hệ thống hoá các lỗ hổng bảo mật theo hai bộ tiêu chuẩn OWASP Mobile Top 10 – 2024 và OWASP API Security Top 10 – 2023, trình bày ba phương pháp kiểm tra bảo mật hiện đang được sử dụng và cuối cùng khảo sát tổng quan các công cụ tự động phát hiện lỗ hổng cùng tình hình nghiên cứu liên quan. Toàn bộ nội dung chương là cơ sở để lựa chọn công cụ, thiết kế môi trường thực nghiệm và xây dựng khung đánh giá sẽ được trình bày ở các chương sau.

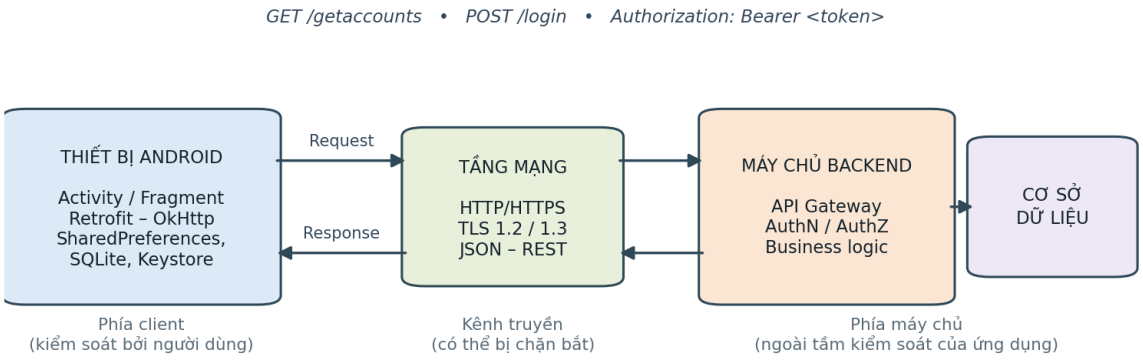
1.1. Tổng quan về ứng dụng Android và cơ chế giao tiếp API

Android là hệ điều hành di động mã nguồn mở do Google phát triển trên nền nhân Linux, hiện chiếm khoảng bảy mươi phần trăm thị phần thiết bị di động toàn cầu [5]. Kiến trúc bảo mật của Android được xây dựng trên nguyên tắc sandbox: mỗi ứng dụng được cấp một định danh người dùng Linux riêng và chỉ được truy cập vùng dữ liệu của chính mình, trừ khi chủ động chia sẻ dữ liệu ra bên ngoài thông qua các thành phần được khai báo trong tệp AndroidManifest.xml. Cơ chế này bảo vệ khá tốt ranh giới giữa các ứng dụng trên cùng một thiết bị, nhưng hoàn toàn không nói gì về mức độ an toàn của dữ liệu khi dữ liệu đó rời khỏi thiết bị để đi tới máy chủ. Chính khoảng trống này là nơi các lỗ hổng bảo mật API xuất hiện.

1.1.1. Kiến trúc giao tiếp và vòng đời một yêu cầu API

Phần lớn ứng dụng Android hiện đại được xây dựng theo mô hình client – server. Ứng dụng trên thiết bị đóng vai trò máy khách, chịu trách nhiệm hiển thị giao diện và thu thập thao tác của người dùng; toàn bộ dữ liệu nghiệp vụ và các quyết định phân quyền quan trọng nằm ở máy chủ phía sau. Hai bên trao đổi với nhau qua các API, phổ biến nhất hiện nay là API theo kiến trúc REST sử dụng định dạng dữ liệu JSON, bên cạnh một số ứng dụng dùng GraphQL hoặc gRPC. Ở phía Android, việc gọi API

thường được thực hiện qua các thư viện mạng như OkHttp và Retrofit, hoặc thư viện Volley đối với các ứng dụng cũ hơn. Hình 1.1 minh hoạ kiến trúc giao tiếp này.



Hình 1.1. Kiến trúc giao tiếp API theo mô hình client – server của ứng dụng Android

Vòng đời của một yêu cầu API điển hình gồm bốn bước. Thứ nhất, ứng dụng khởi tạo một yêu cầu HTTP gồm phương thức (GET, POST, PUT, DELETE), địa chỉ điểm cuối, các trường tiêu đề — trong đó quan trọng nhất là tiêu đề mang thông tin xác thực dạng Authorization: Bearer kèm mã thông báo — và phần thân dữ liệu. Thứ hai, yêu cầu được mã hoá bằng giao thức TLS và truyền qua mạng. Thứ ba, máy chủ tiếp nhận, xác thực danh tính của bên gọi, kiểm tra quyền truy cập đối với tài nguyên được yêu cầu, thực thi logic nghiệp vụ và truy vấn cơ sở dữ liệu. Thứ tư, máy chủ trả về phản hồi kèm mã trạng thái và dữ liệu, ứng dụng phân tích phản hồi và cập nhật giao diện.

Điểm cần nhấn mạnh về mặt an toàn thông tin là sự bất đối xứng trong khả năng kiểm soát giữa hai phía. Toàn bộ phần bên trái của Hình 1.1 — mã nguồn ứng dụng, dữ liệu lưu trên thiết bị, cấu hình mạng của thiết bị — nằm hoàn toàn trong tầm kiểm soát của người đang giữ thiết bị, chứ không phải của nhà phát triển. Người dùng, hoặc kẻ tấn công, có thể dịch ngược tệp APK để đọc mã nguồn, chỉnh sửa và đóng gói lại ứng dụng, cài đặt chứng chỉ số của riêng mình vào thiết bị, hoặc can thiệp vào tiến trình ứng dụng khi nó đang chạy. Do đó, mọi biện pháp kiểm tra được cài đặt duy nhất ở phía client — chẳng hạn ẩn nút chức năng dành cho quản trị viên, hoặc kiểm tra tính hợp lệ của dữ liệu chỉ bằng mã JavaScript hay mã Java trong ứng dụng — đều chỉ có giá trị về trải nghiệm người dùng, không có giá trị về an toàn. Nguyên tắc này là nền

tảng để hiểu vì sao nhóm lỗ hổng nguy hiểm nhất trong OWASP API Security Top 10 lại là các lỗ hổng về phân quyền ở phía máy chủ.

1.1.2. Giao thức SSL/TLS và cơ chế Certificate Pinning

Giao thức TLS là cơ chế bảo vệ cơ bản nhất cho kênh truyền giữa ứng dụng và máy chủ. TLS cung cấp đồng thời ba tính chất: tính bí mật thông qua mã hoá dữ liệu trên đường truyền, tính toàn vẹn thông qua mã xác thực thông điệp, và tính xác thực của máy chủ thông qua hệ thống chứng chỉ số. Phiên bản TLS 1.3, được chuẩn hoá trong RFC 8446 [8], đã loại bỏ nhiều bộ mã hoá lỗi thời và rút ngắn quá trình bắt tay so với TLS 1.2. Từ Android 9 (API 28), hệ điều hành mặc định chặn toàn bộ lưu lượng HTTP không mã hoá; ứng dụng muốn dùng HTTP phải khai báo tường minh thuộc tính android:usesCleartextTraffic hoặc cấu hình tệp Network Security Configuration [6]. Đây là một cải tiến quan trọng, nhưng trên thực tế nhiều ứng dụng vẫn khai báo ngoại lệ cho một số tên miền, hoặc bị lập trình viên vô hiệu hoá việc kiểm tra chứng chỉ trong quá trình phát triển và quên khôi phục trước khi phát hành.

Trong quá trình bắt tay TLS, ứng dụng nhận chứng chỉ từ máy chủ và xác minh chứng chỉ đó dựa trên kho chứng chỉ gốc của hệ thống. Điểm yếu của mô hình này là ứng dụng sẽ chấp nhận bất kỳ chứng chỉ nào được ký bởi một tổ chức chứng thực có trong kho tin cậy — bao gồm cả những chứng chỉ do người dùng tự cài đặt. Kẻ tấn công, hoặc người kiểm thử bảo mật, chỉ cần cài chứng chỉ của một máy chủ uỷ nhiệm như Burp Suite hay OWASP ZAP vào thiết bị là có thể thiết lập hai phiên TLS riêng biệt và đọc toàn bộ nội dung trao đổi giữa ứng dụng và máy chủ. Đây chính là tấn công xen giữa, minh hoạ ở phần (a) của Hình 1.3.

Certificate Pinning là kỹ thuật khắc phục điểm yếu nêu trên bằng cách "ghim" sẵn chứng chỉ hoặc khoá công khai của máy chủ vào trong ứng dụng. Khi thiết lập kết nối, ứng dụng không chỉ kiểm tra chuỗi chứng chỉ với hệ thống mà còn so khớp giá trị băm của khoá công khai nhận được với giá trị đã ghim; nếu không khớp, kết nối bị huỷ ngay lập tức, như minh hoạ ở phần (b) của Hình 1.3. Trên Android, cơ chế này có thể được khai báo bằng thẻ pin-set trong tệp Network Security Configuration, hoặc cài đặt bằng mã thông qua lớp CertificatePinner của thư viện OkHttp [6], [9].

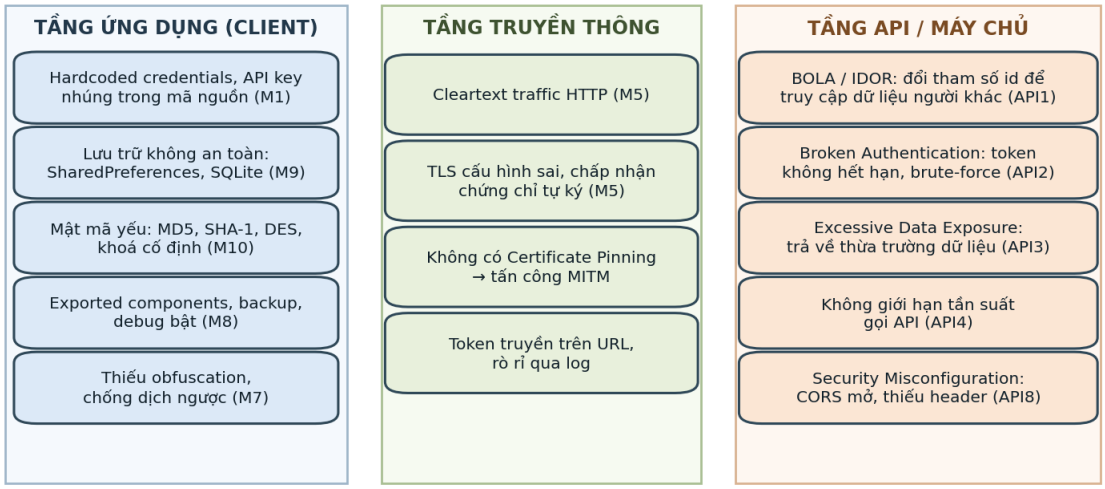


Hình 1.3. Tấn công xen giữa và vai trò của cơ chế Certificate Pinning

Certificate Pinning đặt ra một nghịch lý thú vị đối với công việc kiểm thử bảo mật. Từ góc độ phòng thủ, đây là biện pháp được khuyến nghị đối với các ứng dụng xử lý dữ liệu nhạy cảm. Nhưng từ góc độ kiểm thử, chính cơ chế này lại ngăn người kiểm thử hợp pháp quan sát lưu lượng API của ứng dụng, khiến toàn bộ nhóm công cụ dựa trên máy chủ uỷ nhiệm trở nên vô dụng. Đó là lý do các công cụ can thiệp thời gian chạy như Frida và objection trở thành thành phần bắt buộc trong bộ công cụ kiểm thử ứng dụng di động hiện đại: chúng cho phép móc vào các hàm kiểm tra chứng chỉ trong tiến trình ứng dụng và vô hiệu hoá tạm thời cơ chế ghim chứng chỉ, qua đó khôi phục khả năng quan sát lưu lượng [13], [14]. Quan hệ bổ trợ giữa hai nhóm công cụ này là một trong những nội dung sẽ được đo lường trong phần thực nghiệm của đồ án.

1.1.3. Bề mặt tấn công API của ứng dụng Android

Từ kiến trúc giao tiếp đã trình bày, có thể phân chia bề mặt tấn công API của một ứng dụng Android thành ba tầng tương ứng với ba vùng trách nhiệm kỹ thuật khác nhau, như tổng hợp trên Hình 1.2. Cách phân chia này cũng chính là cơ sở để lựa chọn phương pháp và công cụ kiểm tra phù hợp cho từng nhóm lỗ hổng.



Hình 1.2. Bề mặt tấn công API của ứng dụng Android phân theo ba tầng

Tầng thứ nhất là bản thân ứng dụng trên thiết bị. Vì tệp APK có thể được dịch ngược, mọi bí mật được nhúng trong mã nguồn hoặc trong tài nguyên ứng dụng đều có nguy cơ bị trích xuất: thông tin xác thực cố định, khoá API của dịch vụ bên thứ ba, khoá mã hoá đối xứng, địa chỉ các điểm cuối nội bộ chưa công bố. Bên cạnh đó, dữ liệu mà ứng dụng lưu lại sau khi gọi API — mã thông báo phiên, hồ sơ người dùng, lịch sử giao dịch — thường được ghi vào SharedPreferences hoặc cơ sở dữ liệu SQLite dưới dạng rõ, có thể bị đọc trên thiết bị đã root hoặc bị trích xuất qua cơ chế sao lưu nếu ứng dụng bật thuộc tính allowBackup [7]. Cấu hình sai trong AndroidManifest.xml, chẳng hạn để một thành phần ở trạng thái exported mà không đặt quyền bảo vệ, có thể cho phép một ứng dụng độc hại khác trên cùng thiết bị gọi trực tiếp vào chức năng nội bộ và qua đó gián tiếp gọi API với danh nghĩa của người dùng hợp pháp.

Tầng thứ hai là kênh truyền. Các rủi ro điển hình gồm: sử dụng HTTP thay vì HTTPS cho một phần luồng dữ liệu; cấu hình TLS chấp nhận chứng chỉ tự ký hoặc bỏ qua kiểm tra tên miền; không triển khai Certificate Pinning đối với ứng dụng xử lý dữ liệu nhạy cảm; và truyền mã thông báo hoặc thông tin định danh trên đường dẫn URL thay vì trong tiêu đề, khiến giá trị nhạy cảm bị ghi lại trong nhật ký máy chủ, nhật ký của máy chủ uỷ nhiệm trung gian hoặc lịch sử trình duyệt.

Tầng thứ ba, và cũng là tầng nguy hiểm nhất, là bản thân các điểm cuối API trên máy chủ. Các lỗ hổng ở tầng này không nằm trong mã nguồn ứng dụng và do đó hoàn toàn không thể phát hiện bằng cách phân tích tệp APK. Chúng chỉ bộc lộ khi người kiểm thử chủ động gửi các yêu cầu đã được sửa đổi tới máy chủ và quan sát phản hồi: thay đổi giá trị định danh trong tham số để truy cập dữ liệu của người dùng khác, tái sử dụng một mã thông báo đã hết hạn, gọi một điểm cuối dành cho quản trị viên bằng tài khoản người dùng thường, hoặc gửi hàng nghìn yêu cầu liên tiếp để kiểm tra cơ chế giới hạn tần suất. Nhận xét này giải thích vì sao một công cụ phân tích tĩnh, dù mạnh đến đâu, cũng không thể thay thế được kiểm thử động trong bài toán bảo mật API.

1.2. Các lỗ hổng bảo mật API theo chuẩn OWASP

OWASP là tổ chức phi lợi nhuận quốc tế chuyên về an toàn ứng dụng, nổi tiếng với các danh sách xếp hạng rủi ro được cập nhật định kỳ dựa trên dữ liệu thực tế từ các nền tảng săn lỗi nhận thưởng, cơ sở dữ liệu lỗ hổng và khảo sát cộng đồng chuyên gia. Trong phạm vi đề tài, hai danh sách liên quan trực tiếp là OWASP Mobile Top 10 phiên bản 2024 [1], mô tả rủi ro ở phía ứng dụng di động, và OWASP API Security Top 10 phiên bản 2023 [2], mô tả rủi ro ở tầng API. Ngoài ra, OWASP còn duy trì bộ tài liệu MASVS quy định các yêu cầu kiểm chứng an toàn cho ứng dụng di động và MASTG hướng dẫn quy trình kiểm thử tương ứng [3], [4]; hai tài liệu này được đồ án sử dụng làm cơ sở phương pháp luận khi thiết kế các trường hợp kiểm thử ở chương sau.

1.2.1. OWASP Mobile Top 10 – 2024

Phiên bản 2024 là bản cập nhật lớn đầu tiên của OWASP Mobile Top 10 kể từ năm 2016 và khác biệt căn bản so với phiên bản cũ cả về cách đặt tên lẫn thứ tự xếp hạng [1]. Bảng 1.1 trình bày danh sách đầy đủ mười rủi ro cùng mô tả tóm tắt và mức độ liên quan tới bảo mật API — tiêu chí phản ánh việc rủi ro đó có ảnh hưởng trực tiếp tới tính an toàn của dữ liệu trao đổi qua API hay không.

Bảng 1.1. Danh sách OWASP Mobile Top 10 – 2024

Mã	Tên rủi ro	Mô tả tóm tắt	Liên quan API
M1	Improper Credential Usage	Sử dụng sai thông tin xác thực: nhúng cố định mật khẩu, khoá API trong mã nguồn; truyền hoặc lưu trữ thông tin xác thực không an toàn.	Rất cao
M2	Inadequate Supply Chain Security	Rủi ro từ chuỗi cung ứng: thư viện, SDK bên thứ ba chứa mã độc hoặc lỗ hổng đã biết; quy trình xây dựng và ký ứng dụng không được bảo vệ.	Trung bình
M3	Insecure Authentication/Authorization	Cơ chế xác thực và phân quyền yếu: mã thông báo không hết hạn, kiểm tra quyền chỉ thực hiện ở phía client, cho phép thử mật khẩu không giới hạn.	Rất cao
M4	Insufficient Input/Output Validation	Không kiểm tra dữ liệu vào và ra, dẫn tới tấn công tiêm nhiễm hoặc xử lý sai dữ liệu do máy chủ trả về.	Cao
M5	Insecure Communication	Giao tiếp không an toàn: dùng HTTP, cấu hình TLS sai, không kiểm tra chứng chỉ, thiếu Certificate Pinning.	Rất cao
M6	Inadequate Privacy Controls	Thu thập, xử lý và chia sẻ dữ liệu cá nhân vượt quá mức cần thiết hoặc không có sự đồng ý rõ ràng của người dùng.	Trung bình
M7	Insufficient Binary Protections	Thiếu biện pháp bảo vệ tệp nhị phân: không làm rối mã, không chống dịch ngược, không phát hiện thiết bị đã root hoặc ứng dụng bị sửa đổi.	Thấp
M8	Security Misconfiguration	Cấu hình sai: bật chế độ gỡ lỗi, cho phép sao lưu dữ liệu, thành phần exported không được bảo vệ, ký ứng dụng bằng chứng chỉ gỡ lỗi.	Cao
M9	Insecure Data Storage	Lưu trữ dữ liệu nhạy cảm không an toàn trên thiết bị: SharedPreferences dạng rõ, cơ sở dữ liệu SQLite không mã hoá, ghi dữ liệu nhạy cảm vào nhật ký.	Cao
M10	Insufficient Cryptography	Sử dụng thuật toán mật mã lỗi thời hoặc sai cách: MD5, SHA-1, DES, chế độ ECB, khoá cố định trong mã nguồn.	Cao

So với phiên bản 2016, ba thay đổi đáng chú ý nhất như sau. Thứ nhất, mục M1 được đổi từ “Improper Platform Usage” thành “Improper Credential Usage”, phản ánh việc phần lớn sự cố thực tế hiện nay bắt nguồn từ việc để lộ thông tin xác thực chứ không phải từ việc dùng sai tính năng nền tảng. Thứ hai, mục M2 “Inadequate Supply Chain Security” là mục hoàn toàn mới, ghi nhận xu hướng tấn công thông qua thư viện và bộ công cụ phát triển của bên thứ ba. Thứ ba, các mục M9 “Insecure Data Storage” và M10 “Insufficient Cryptography” — vốn đứng ở vị trí cao trong phiên bản 2016 —

bị đẩy xuống cuối danh sách, không phải vì chúng ít nguy hiểm hơn mà vì các nền tảng hiện đại đã cung cấp sẵn nhiều cơ chế bảo vệ mặc định [1].

1.2.2. OWASP API Security Top 10 – 2023

Trong khi OWASP Mobile Top 10 tập trung vào phía ứng dụng, OWASP API Security Top 10 mô tả các rủi ro đặc thù của tầng API — phần lớn nằm ở phía máy chủ và trong logic nghiệp vụ. Bảng 1.2 trình bày mười rủi ro của phiên bản 2023 [2].

Bảng 1.2. Danh sách OWASP API Security Top 10 – 2023

Mã	Tên rủi ro	Mô tả tóm tắt	Mới
API1	Broken Object Level Authorization	Điểm cuối nhận định danh đối tượng từ tham số nhưng không kiểm tra người gọi có quyền với đối tượng đó; còn gọi là IDOR.	
API2	Broken Authentication	Cơ chế xác thực cài đặt sai: mã thông báo yếu hoặc không hết hạn, không giới hạn số lần thử, quy trình khôi phục mật khẩu thiếu an toàn.	
API3	Broken Object Property Level Authorization	Hợp nhất hai rủi ro cũ là lộ dữ liệu quá mức và gán giá trị hàng loạt: API trả về hoặc chấp nhận những trường dữ liệu mà người gọi không được phép đọc hoặc ghi.	
API4	Unrestricted Resource Consumption	Không giới hạn tần suất và khối lượng tài nguyên tiêu thụ, dẫn tới từ chối dịch vụ hoặc phát sinh chi phí ngoài dự kiến.	
API5	Broken Function Level Authorization	Phân quyền theo chức năng sai: người dùng thường gọi được các điểm cuối dành riêng cho quản trị viên.	
API6	Unrestricted Access to Sensitive Business Flows	Luồng nghiệp vụ nhạy cảm bị khai thác tự động bằng kịch bản hoặc bot, chẳng hạn mua hàng số lượng lớn, đăng ký tài khoản hàng loạt.	✓
API7	Server Side Request Forgery	API nhận địa chỉ URL từ người dùng và truy cập tài nguyên đó mà không kiểm tra, cho phép truy cập tới hạ tầng nội bộ.	
API8	Security Misconfiguration	Cấu hình sai: chính sách CORS quá rộng, thiếu tiêu đề bảo mật, bật chế độ gỡ lỗi, thông báo lỗi tiết lộ chi tiết hệ thống.	
API9	Improper Inventory Management	Quản lý danh mục API kém: phiên bản cũ vẫn hoạt động, điểm cuối thử nghiệm chưa gỡ, thiếu tài liệu về các API đang chạy.	
API10	Unsafe Consumption of APIs	Tin tưởng quá mức vào dữ liệu nhận từ API bên thứ ba, không kiểm tra và làm sạch trước khi sử dụng.	✓

Ba mục hoàn toàn mới trong phiên bản 2023 là API6, API7 và API10. API6 xuất hiện do sự gia tăng của các cuộc tấn công tự động nhắm vào luồng nghiệp vụ trong

thương mại điện tử, đặt vé và các nền tảng trực tuyến — đây là loại rủi ro không nằm ở lỗi kỹ thuật của từng yêu cầu riêng lẻ mà nằm ở chỗ hệ thống không phân biệt được người dùng thật với chương trình tự động. API7 (Server Side Request Forgery) được tách thành mục riêng vì các API hiện đại ngày càng thường xuyên tự truy cập địa chỉ URL do người dùng cung cấp — chẳng hạn webhook, xem trước liên kết hay tải ảnh từ URL — tạo điều kiện để kẻ tấn công điều khiển máy chủ gửi yêu cầu tới tài nguyên nội bộ. API10 phản ánh xu hướng ứng dụng hiện đại phụ thuộc ngày càng nhiều vào dịch vụ bên thứ ba, khiến một lỗ hổng ở nhà cung cấp có thể lan truyền sang hệ thống sử dụng nó [2].

Một điểm đáng lưu ý là API1 giữ nguyên vị trí đứng đầu danh sách trong cả hai phiên bản 2019 và 2023. Nguyên nhân nằm ở bản chất của lỗ hổng này: nó không phải là một lỗi cú pháp hay một mẫu mã nguy hiểm có thể nhận diện bằng biểu thức chính quy, mà là một sai sót về logic nghiệp vụ — máy chủ trả lời đúng theo yêu cầu nhưng lại trả lời cho một người không có quyền hỏi. Không một công cụ phân tích tĩnh nào có thể biết được rằng bản ghi có định danh 1002 thuộc về một người dùng khác với người đang gửi yêu cầu. Việc phát hiện API1 do đó bắt buộc phải dựa trên kiểm thử động, trong đó người kiểm thử chuẩn bị ít nhất hai tài khoản và thử truy cập chéo dữ liệu giữa chúng.

1.2.3. Mối quan hệ giữa hai bộ tiêu chuẩn và phạm vi lỗ hổng của đồ án

Hai bộ tiêu chuẩn nêu trên không thay thế mà bổ sung cho nhau: chúng mô tả cùng một hệ thống nhưng từ hai phía đối diện của kênh truyền API. Có thể hình dung Mobile Top 10 chịu trách nhiệm cho phần bên trái của Hình 1.1 còn API Security Top 10 chịu trách nhiệm cho phần bên phải, và một số rủi ro nằm ở vùng giao nhau giữa hai bên. Chẳng hạn, M5 “Insecure Communication” và M3 “Insecure Authentication/Authorization” của Mobile Top 10 có phần chồng lấn với API2 “Broken Authentication” của API Security Top 10; điểm khác biệt là Mobile Top 10 nhìn từ góc độ ứng dụng cài đặt cơ chế thế nào, còn API Security Top 10 nhìn từ góc độ máy chủ kiểm chứng cơ chế đó ra sao.

Sự phân đôi này có hệ quả trực tiếp tới việc lựa chọn công cụ. Các rủi ro thuần tuý nằm ở phía ứng dụng — M1, M7, M8, M9, M10 — để lại dấu vết trong mã nguồn

và tệp cấu hình, do đó có thể được phát hiện bằng phân tích tĩnh trên tệp APK. Ngược lại, các rủi ro nằm ở phía máy chủ — API1, API3, API4, API5 — không để lại bất kỳ dấu vết nào trong tệp APK và chỉ có thể được phát hiện bằng cách tương tác thật với máy chủ. Nhóm rủi ro ở kênh truyền — M5, API8 — nằm ở giữa: phân tích tĩnh có thể phát hiện dấu hiệu cấu hình sai, nhưng chỉ kiểm thử động mới khẳng định được lỗ hổng thực sự tồn tại và khai thác được. Đây chính là luận điểm nền tảng dẫn tới việc đồ án lựa chọn benchmark đồng thời cả công cụ tĩnh lẫn công cụ động thay vì chỉ tập trung vào một nhóm.

Trong phạm vi thực nghiệm, đồ án không đặt mục tiêu bao phủ toàn bộ hai mươi rủi ro của hai danh sách, mà tập trung vào tập lỗ hổng có thể tái hiện được một cách xác định trên ứng dụng đích và có ý nghĩa đối với bài toán bảo mật API, cụ thể gồm: M1, M3, M5, M8, M9, M10 ở phía ứng dụng và API1, API2, API3, API4, API8 ở phía máy chủ. Các rủi ro còn lại — đặc biệt là M2, M6 và API6 — mang tính quy trình hoặc tính nghiệp vụ, khó đánh giá khách quan trong môi trường phòng thí nghiệm và do đó chỉ được đề cập ở mức lý thuyết.

1.3. Các phương pháp kiểm tra bảo mật API của ứng dụng Android

Việc kiểm tra bảo mật một ứng dụng Android có thể được tiến hành theo ba hướng tiếp cận: phân tích tĩnh, phân tích động và phân tích kết hợp. Ba hướng này khác nhau ở chỗ ứng dụng có được thực thi hay không, và do đó khác nhau căn bản về loại thông tin mà người kiểm thử có thể quan sát được. Hiểu rõ ranh giới năng lực của từng phương pháp là điều kiện tiên quyết để giải thích kết quả benchmark ở chương sau: một công cụ không phát hiện được một lỗ hổng nào đó có thể vì bản thân công cụ yếu, nhưng cũng có thể đơn giản vì phương pháp mà nó dựa trên về nguyên tắc không thể nhìn thấy lỗ hổng đó.

1.3.1. Phân tích tĩnh

Phân tích tĩnh là phương pháp kiểm tra mà không thực thi ứng dụng. Đối với Android, đối tượng phân tích là tệp APK — thực chất là một tệp nén chứa mã bytecode Dalvik trong các tệp .dex, tệp kê khai AndroidManifest.xml đã được biên dịch sang dạng nhị phân, các tệp tài nguyên, thư viện dùng chung dạng .so và các tệp

cấu hình khác. Quy trình phân tích tĩnh điển hình gồm ba bước: giải nén và giải mã tệp kê khai để xác định các thành phần, quyền và cấu hình bảo mật của ứng dụng; dịch ngược bytecode sang dạng mã nguồn có thể đọc được; và quét toàn bộ mã cùng tài nguyên bằng một tập luật để tìm các mẫu nguy hiểm.

Điểm mạnh của phân tích tĩnh nằm ở độ phủ và chi phí. Phương pháp này quét được toàn bộ mã nguồn một cách có hệ thống, kể cả những nhánh mã hiếm khi được thực thi trong quá trình sử dụng bình thường; không cần thiết bị thật hay máy ảo; thời gian phân tích thường chỉ tính bằng phút; và có thể tích hợp thẳng vào quy trình tích hợp liên tục để chạy tự động sau mỗi lần xây dựng ứng dụng. Đối với các lỗ hổng có dấu hiệu rõ ràng trong mã — chuỗi mật khẩu cố định, lời gọi hàm băm MD5, thuộc tính android:debuggable được đặt bằng true — phân tích tĩnh cho kết quả nhanh và đáng tin cậy.

Hạn chế của phương pháp này cũng bắt nguồn từ chính bản chất của nó. Thứ nhất, phân tích tĩnh không quan sát được hành vi thực tế của ứng dụng, do đó không thể xác nhận một mẫu mã đáng ngờ có thực sự dẫn tới lỗ hổng khai thác được hay không; hệ quả là tỷ lệ báo động giả thường cao, và người kiểm thử phải bỏ nhiều công sức để sàng lọc kết quả. Thứ hai, các kỹ thuật làm rối mã, mã hoá chuỗi ký tự hoặc tải mã động khi chạy làm giảm đáng kể hiệu quả của việc quét theo mẫu. Thứ ba, và quan trọng nhất đối với đề tài này, phân tích tĩnh về nguyên tắc không thể phát hiện các lỗ hổng nằm ở phía máy chủ, bởi mã nguồn của máy chủ không có mặt trong tệp APK.

1.3.2. Phân tích động dựa trên máy chủ uỷ nhiệm

Phân tích động dựa trên máy chủ uỷ nhiệm đặt một thành phần trung gian giữa ứng dụng và máy chủ để ghi lại, phân tích và cho phép sửa đổi toàn bộ lưu lượng API. Người kiểm thử cấu hình thiết bị hoặc máy ảo Android trỏ lưu lượng qua máy chủ uỷ nhiệm, cài chứng chỉ của công cụ vào kho chứng chỉ tin cậy của hệ thống, sau đó sử dụng ứng dụng một cách bình thường. Mỗi thao tác của người dùng sinh ra một hoặc nhiều yêu cầu API được ghi lại đầy đủ cả tiêu đề lẫn nội dung.

Giá trị lớn nhất của phương pháp này không nằm ở việc quan sát thụ động, mà nằm ở khả năng chủ động sửa đổi và gửi lại yêu cầu. Đây chính là kỹ thuật duy nhất cho phép phát hiện nhóm lỗ hổng phân quyền: người kiểm thử đăng nhập bằng tài

khoản A, bắt lấy một yêu cầu truy vấn dữ liệu của A, thay đổi tham số định danh sang giá trị thuộc về tài khoản B rồi gửi lại; nếu máy chủ trả về dữ liệu của B, lỗ hổng API1 được xác nhận với bằng chứng không thể chối cãi. Cùng cách làm đó, việc gỡ bỏ tiêu đề xác thực để kiểm tra API2, gọi điểm cuối quản trị bằng mã thông báo của người dùng thường để kiểm tra API5, hay gửi hàng loạt yêu cầu liên tiếp để kiểm tra API4 đều trở nên khả thi. Ngoài ra, việc quan sát phản hồi còn giúp phát hiện lộ dữ liệu quá mức (API3) — trường hợp máy chủ trả về nhiều trường dữ liệu hơn mức giao diện ứng dụng hiển thị — và các sai sót cấu hình như thiếu tiêu đề bảo mật hoặc chính sách chia sẻ tài nguyên quá rộng (API8).

Hạn chế của phương pháp này gồm ba điểm. Thứ nhất, độ phủ phụ thuộc hoàn toàn vào việc người kiểm thử có thực hiện đủ các thao tác để kích hoạt hết các điểm cuối hay không; một chức năng không được sử dụng đồng nghĩa với một điểm cuối không được kiểm tra. Thứ hai, phương pháp này bị vô hiệu hoá hoàn toàn khi ứng dụng triển khai Certificate Pinning, trừ khi kết hợp với công cụ can thiệp thời gian chạy. Thứ ba, phần lớn công việc mang tính bán thủ công, khó tự động hoá trọn vẹn và đòi hỏi người kiểm thử phải hiểu logic nghiệp vụ của ứng dụng.

1.3.3. Phân tích động bằng can thiệp thời gian chạy

Hướng phân tích động thứ hai là can thiệp trực tiếp vào tiến trình của ứng dụng khi ứng dụng đang chạy. Công cụ tiêu biểu là Frida, hoạt động bằng cách chèn một máy ảo JavaScript vào không gian địa chỉ của tiến trình đích, từ đó cho phép người kiểm thử móc vào bất kỳ hàm nào của ứng dụng hoặc của thư viện hệ thống, đọc và thay đổi tham số đầu vào, thay đổi giá trị trả về, hoặc gọi trực tiếp các hàm nội bộ [13].

Trong bài toán bảo mật API, phương pháp này có ba ứng dụng chính. Ứng dụng thứ nhất và quan trọng nhất là vô hiệu hoá Certificate Pinning để khôi phục khả năng quan sát lưu lượng cho các công cụ dựa trên máy chủ uỷ nhiệm — nghĩa là phương pháp này đóng vai trò hỗ trợ, mở đường cho phương pháp trước đó hoạt động. Ứng dụng thứ hai là móc vào các hàm mật mã để đọc khoá và dữ liệu ở dạng rõ ngay trước thời điểm mã hoá, qua đó kiểm chứng chất lượng cài đặt mật mã của ứng dụng (M10). Ứng dụng thứ ba là quan sát quá trình ghi dữ liệu xuống thiết bị để xác định chính xác

dữ liệu nhạy cảm nào được lưu và lưu ở đâu (M9). Công cụ objection, được xây dựng trên nền Frida, đóng gói các kịch bản phổ biến nhất thành các lệnh dòng lệnh đơn giản, giúp người kiểm thử không cần tự viết mã móc hàm [14].

Điểm hạn chế của phương pháp này là yêu cầu môi trường khắt khe: thiết bị hoặc máy ảo phải có quyền quản trị cao nhất để chạy tiến trình phục vụ của Frida, kiến trúc bộ xử lý của tệp nhị phân phải khớp với thiết bị, và một số ứng dụng có cài đặt cơ chế phát hiện việc bị can thiệp sẽ tự thoát khi phát hiện dấu hiệu bất thường. Ngoài ra, phương pháp này không tự động phát hiện lỗ hổng theo nghĩa thông thường — nó cung cấp khả năng quan sát và can thiệp, còn việc kết luận có lỗ hổng hay không vẫn phụ thuộc vào phán đoán của người kiểm thử.

1.3.4. Phân tích kết hợp và so sánh ba phương pháp

Từ phân tích ở ba mục trên có thể thấy không một phương pháp đơn lẻ nào bao phủ được toàn bộ các loại lỗ hổng bảo mật API của ứng dụng Android. Phương pháp kết hợp sử dụng đồng thời cả ba hướng theo trình tự bổ trợ: phân tích tĩnh được thực hiện trước để lập bản đồ các điểm cuối API, phát hiện các bí mật nhúng trong mã và khoanh vùng những khu vực đáng ngờ; kết quả này định hướng cho giai đoạn kiểm thử động, giúp người kiểm thử biết cần tấn công vào điểm cuối nào; công cụ can thiệp thời gian chạy được dùng khi gặp rào cản kỹ thuật như Certificate Pinning. Bảng 1.3 tổng hợp khả năng phát hiện của ba phương pháp đối với các loại lỗ hổng nằm trong phạm vi nghiên cứu.

Bảng 1.3. So sánh khả năng phát hiện lỗ hổng của ba phương pháp kiểm tra

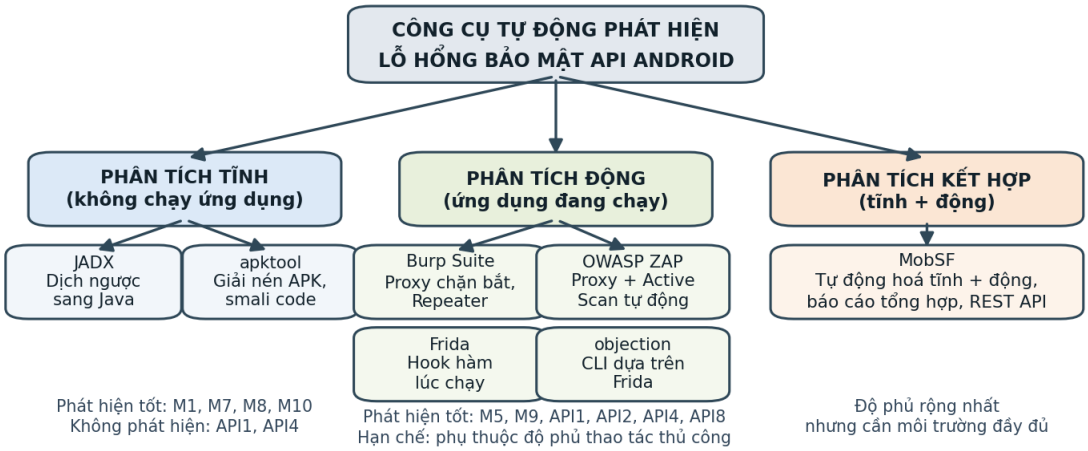
Loại lỗ hổng	Phân tích tĩnh	Động – uỷ nhiệm	Động – thời gian chạy
M1 – Thông tin xác thực nhúng trong mã	Tốt	Trung bình	Trung bình
M3 – Xác thực, phân quyền không an toàn	Một phần	Tốt	Trung bình
M5 – Giao tiếp không an toàn	Một phần	Tốt	Tốt
M8 – Cấu hình bảo mật sai	Tốt	Trung bình	Trung bình
M9 – Lưu trữ dữ liệu không an toàn	Một phần	Không	Tốt
M10 – Mật mã không đủ mạnh	Tốt	Không	Trung bình
API1 – Phân quyền mức đối tượng (BOLA)	Không	Tốt	Không
API2 – Xác thực bị phá vỡ	Một phần	Tốt	Trung bình

Loại lỗ hổng	Phân tích tĩnh	Động – uỷ nhiệm	Động – thời gian chạy
API3 – Phân quyền mức thuộc tính (BOPLA)	Không	Tốt	Không
API4 – Không giới hạn tài nguyên	Không	Tốt	Không
API8 – Cấu hình sai ở tầng API	Một phần	Tốt	Không
Vượt qua Certificate Pinning khi kiểm thử	Không	Không	Tốt

Bảng 1.3 cho thấy một quy luật rõ ràng: cột phân tích tĩnh mạnh ở nửa trên của bảng — các lỗ hổng phía ứng dụng, trong khi cột phân tích động dựa trên máy chủ uỷ nhiệm mạnh ở nửa dưới — các lỗ hổng phía máy chủ; còn cột can thiệp thời gian chạy đảm nhiệm những trường hợp mà hai cột kia không với tới. Không có cột nào phủ được toàn bộ các hàng. Đây là cơ sở lý luận trực tiếp cho đề xuất quy trình kiểm tra kết hợp sẽ được trình bày ở chương cuối của đồ án, và cũng là lý do việc benchmark các công cụ phải được thực hiện trên cùng một tập lỗ hổng để kết quả có thể so sánh được với nhau.

1.4. Tổng quan các công cụ tự động phát hiện lỗ hổng bảo mật API

Trên cơ sở ba phương pháp kiểm tra vừa trình bày, mục này khảo sát tổng quan bảy công cụ được lựa chọn đưa vào nghiên cứu của đồ án. Tiêu chí lựa chọn gồm bốn điểm: công cụ phải được sử dụng rộng rãi trong thực tế kiểm thử ứng dụng di động; có tài liệu chính thức đầy đủ và được duy trì cập nhật; miễn phí hoặc có phiên bản cộng đồng đủ dùng cho mục đích nghiên cứu; và tập hợp bảy công cụ phải bao phủ được cả ba phương pháp phân tích. Hình 1.4 thể hiện cách phân loại các công cụ này.



Hình 1.4. Phân loại các công cụ tự động phát hiện lỗ hổng bảo mật API trên Android

Bảng 1.4. Tổng quan bảy công cụ tự động phát hiện lỗ hổng bảo mật API trên Android

Công cụ	Phương pháp	Giấy phép	Điểm mạnh chính	Nhóm lỗ hổng hướng tới
MobSF	Tĩnh + Động	MIT	Tự động hoá cao, giao diện web, báo cáo JSON/PDF, có REST API	M1, M5, M7, M8, M9, M10
Burp Suite CE	Động – uỷ nhiệm	Community miễn phí	Chặn bắt, sửa đổi và gửi lại yêu cầu; công cụ Repeater, Intruder	API1, API2, API3, API5
OWASP ZAP	Động – uỷ nhiệm	Apache 2.0	Mã nguồn mở, có Active Scan tự động, REST API, chạy trong Docker	API2, API4, API8, M5
Frida	Động – thời gian chạy	wxWindows	Móc vào bất kỳ hàm nào lúc chạy, vô hiệu hoá Certificate Pinning	M5, M9, M10
objection	Động – thời gian chạy	MIT	Giao diện dòng lệnh đóng gói sẵn kịch bản Frida thông dụng	M5, M9
JADX	Tĩnh – dịch ngược	Apache 2.0	Dịch bytecode sang mã Java/Kotlin dễ đọc, có giao diện đồ hoạ	M1, M10
apktool	Tĩnh – dịch ngược	Apache 2.0	Giải nén và đóng gói lại APK, khôi phục tệp kê khai và mã smali	M7, M8

1.4.1. MobSF – khung kiểm thử tự động kết hợp

MobSF là khung kiểm thử bảo mật ứng dụng di động mã nguồn mở, hỗ trợ cả Android và iOS, và là công cụ duy nhất trong nhóm khảo sát tích hợp đồng thời cả phân tích tĩnh lẫn phân tích động trong một nền tảng thống nhất [10]. Ở chế độ phân tích tĩnh, MobSF tự động giải nén tệp APK, phân tích tệp kê khai, dịch ngược bytecode và quét mã bằng bộ luật tích hợp; kết quả được phân loại theo mức độ nguy hiểm, ánh xạ sang mã CWE và sang các mục của OWASP Mobile Top 10, đồng thời tổng hợp thành một điểm số an toàn chung. Ở chế độ phân tích động, MobSF yêu cầu một thiết bị hoặc máy ảo đang chạy, tự cài đặt và khởi chạy ứng dụng, ghi lại lưu lượng mạng, theo dõi các thao tác đọc ghi tệp và cơ sở dữ liệu, đồng thời chụp màn hình quá trình thực thi.

Đặc điểm khiến MobSF đặc biệt phù hợp với đề tài là việc công cụ cung cấp REST API cho phép tải tệp lên, kích hoạt quét và tải kết quả về dưới dạng JSON hoặc PDF một cách hoàn toàn tự động. Khả năng này cho phép tích hợp MobSF vào quy trình tích hợp liên tục, và cũng là cơ sở kỹ thuật cho phần đóng góp của đồ án ở giai

đoạn sau. Ảnh Docker chính thức của MobSF hỗ trợ nhiều kiến trúc bộ xử lý, trong đó có ARM64 — điều kiện cần thiết để công cụ chạy được trên môi trường lab của đồ án.

1.4.2. Burp Suite và OWASP ZAP – hai công cụ uỷ nhiệm

Burp Suite của PortSwigger là công cụ được sử dụng rộng rãi nhất trong kiểm thử xâm nhập ứng dụng web và API [11]. Khi kiểm tra ứng dụng Android, Burp Suite đóng vai trò máy chủ uỷ nhiệm trung gian: người kiểm thử cấu hình máy ảo Android trỏ lưu lượng qua Burp, cài chứng chỉ của Burp vào kho chứng chỉ hệ thống, sau đó toàn bộ yêu cầu và phản hồi HTTPS đều hiện ra ở dạng rõ. Ba thành phần quan trọng nhất đối với kiểm thử API gồm: Proxy Intercept cho phép xem và sửa từng yêu cầu trước khi gửi đi; Repeater cho phép gửi lại một yêu cầu nhiều lần với các tham số khác nhau — đây là công cụ chủ lực để kiểm tra lỗ hổng phân quyền; và Intruder cho phép tự động hoá việc thử nhiều giá trị trên một vị trí tham số. Phiên bản Community miễn phí không bao gồm bộ quét chủ động, do đó phần lớn công việc mang tính bán thủ công — một đặc điểm sẽ được ghi nhận khi đánh giá tiêu chí mức độ tự động hoá ở chương thực nghiệm.

OWASP ZAP là công cụ uỷ nhiệm mã nguồn mở do chính OWASP phát triển, có kiến trúc tương tự Burp Suite nhưng khác biệt ở hai điểm có ý nghĩa với đề tài [12]. Thứ nhất, ZAP tích hợp sẵn tính năng Active Scan cho phép tự động sinh và gửi các yêu cầu tấn công tới các điểm cuối đã phát hiện, cùng với Passive Scan chạy nền phân tích mọi lưu lượng đi qua — nghĩa là ZAP tự động hoá được phần công việc mà Burp Suite Community yêu cầu làm thủ công. Thứ hai, ZAP cung cấp REST API đầy đủ và có thể chạy ở chế độ nền trong Docker, cho phép điều khiển hoàn toàn bằng kịch bản. Hai đặc điểm này khiến ZAP trở thành công cụ động phù hợp nhất để ghép nối với MobSF trong một quy trình tự động hoá thống nhất.

1.4.3. Frida và objection – can thiệp thời gian chạy

Frida là bộ công cụ can thiệp thời gian chạy đa nền tảng, cho phép chèn mã JavaScript vào tiến trình đang chạy để móc vào các hàm, đọc và sửa tham số cũng như giá trị trả về [13]. Trên Android, việc sử dụng Frida đòi hỏi đẩy tệp nhị phân frida-server phù hợp với kiến trúc bộ xử lý của thiết bị vào thư mục tạm của hệ thống và

chạy nó với quyền quản trị cao nhất; máy ảo do Android Studio tạo ra đáp ứng được yêu cầu này thông qua lệnh adb root.

Ứng dụng quan trọng nhất của Frida trong đề tài là vô hiệu hoá Certificate Pinning. Kỹ thuật thực hiện là móc vào các lớp và phương thức chịu trách nhiệm kiểm tra chứng chỉ của nền tảng — chẳng hạn lớp quản lý tin cậy của Java hoặc lớp CertificatePinner của OkHttp — rồi thay thế phần thân hàm bằng một hàm luôn trả về kết quả hợp lệ. Tài liệu nghiên cứu đã hệ thống hoá bốn nhóm kỹ thuật vượt qua cơ chế kiểm tra chứng chỉ trên Android, trong đó nhóm dựa trên can thiệp thời gian chạy được đánh giá là linh hoạt nhất vì không đòi hỏi sửa đổi và ký lại tệp APK [21]. Công cụ objection đóng gói các kỹ thuật này thành một lệnh duy nhất, giúp rút ngắn đáng kể thời gian thiết lập và giảm yêu cầu về kỹ năng lập trình đối với người kiểm thử [14].

1.4.4. JADX và apktool – dịch ngược tệp APK

JADX là công cụ dịch ngược chuyển bytecode Dalvik sang mã nguồn Java hoặc Kotlin có thể đọc được, cung cấp cả giao diện đồ hoạ lẫn giao diện dòng lệnh [15]. Trong kiểm thử bảo mật API, JADX thường được dùng để lập bản đồ các điểm cuối API mà ứng dụng gọi tới, tìm các chuỗi ký tự đáng ngờ như khoá và mật khẩu cố định, và đọc trực tiếp logic xử lý xác thực để hiểu cách ứng dụng sinh và lưu trữ mã thông báo. Kết quả dịch ngược của JADX dễ đọc nhưng không phải lúc nào cũng chính xác tuyệt đối so với mã gốc, đặc biệt với mã đã bị làm rối.

apktool làm việc ở mức thấp hơn: nó giải nén tệp APK thành mã smali — dạng hợp ngữ của bytecode Dalvik — cùng với các tệp tài nguyên và tệp kê khai đã được khôi phục về dạng XML đọc được [16]. Mã smali khó đọc hơn nhiều so với đầu ra của JADX, nhưng có ưu điểm là phản ánh chính xác nội dung tệp nhị phân và cho phép sửa đổi rồi đóng gói lại ứng dụng. Trong thực tế, hai công cụ thường được dùng bổ trợ: JADX để hiểu logic, apktool để đọc chính xác tệp kê khai và để can thiệp khi cần.

1.4.5. Tình hình nghiên cứu liên quan và khoảng trống nghiên cứu

Hướng nghiên cứu đánh giá hiệu quả của công cụ phân tích bảo mật Android đã được đặt ra từ khá sớm. Công trình có tính nền tảng là nghiên cứu của Ranganath và Mitra công bố năm 2018 và sau đó đăng trên tạp chí Empirical Software Engineering,

trong đó nhóm tác giả đánh giá một loạt công cụ phân tích tĩnh miễn phí trên tập chuẩn Ghera gồm các lỗ hổng đã biết [19]. Kết luận đáng chú ý nhất của nghiên cứu là ngay cả khi gộp kết quả của cả mười bốn công cụ phát hiện lỗ hổng được khảo sát, các công cụ cũng chỉ phát hiện được 30 trong 42 lỗ hổng đã biết của tập chuẩn, cho thấy khoảng cách lớn giữa năng lực được quảng bá và năng lực thực tế của công cụ. Tập chuẩn Ghera do Mitra và Ranganath xây dựng [18] cũng chính là mô hình phương pháp luận mà đồ án tham khảo khi thiết kế ma trận kiểm thử: mỗi lỗ hổng phải được tái hiện một cách xác định, có thể lặp lại và có kết quả kỳ vọng rõ ràng thì phép đo mới có ý nghĩa.

Nhóm công trình thứ hai tập trung riêng vào MobSF. Nghiên cứu công bố trên IEEE năm 2024 phân tích lỗ hổng ứng dụng Android bằng MobSF và ghi nhận khả năng phát hiện của công cụ đối với các lỗ hổng như Janus, tiêm nhiễm câu lệnh truy vấn và các sai sót trong cài đặt mật mã, đồng thời chỉ ra hạn chế của phần phân tích động khi ứng dụng có cơ chế phát hiện thiết bị đã root [20]. Một nghiên cứu khác cùng năm báo cáo tỷ lệ phát hiện đúng đạt tới chín mươi bảy phần trăm khi kết hợp cả hai chế độ phân tích của MobSF — tuy nhiên con số này được đo trên tập ứng dụng và tập lỗ hổng do chính nhóm tác giả lựa chọn, nên khó so sánh trực tiếp với các nghiên cứu khác.

Nhóm công trình thứ ba đề xuất các khung kiểm thử kết hợp nhiều công cụ. Tiêu biểu là nghiên cứu năm 2024 đề xuất khung kết hợp một công cụ phân tích tĩnh với một công cụ tấn công động và bổ sung bước vượt qua cơ chế ghim chứng chỉ, có ánh xạ kết quả sang OWASP Mobile Top 10; và nghiên cứu năm 2025 kết hợp phân tích mã tĩnh với công cụ tấn công động để đánh giá lỗ hổng ở các thành phần exported của ứng dụng. Hai công trình này gần với hướng của đồ án nhất về mặt ý tưởng, nhưng đều tập trung vào lỗ hổng phía ứng dụng và không đặt trọng tâm vào tầng API. Bảng 1.5 tổng hợp các công trình tiêu biểu đã khảo sát.

Bảng 1.5. Tổng hợp các công trình nghiên cứu liên quan tiêu biểu

Công trình	Năm	Đối tượng khảo sát	Kết quả chính	Hạn chế so với đồ án
Ranganath & Mitra [19]	2018–2019	Các công cụ phân tích tĩnh miễn phí trên tập chuẩn Ghera	Gộp cả 14 công cụ cũng chỉ phát hiện được 30/42 lỗ hổng đã biết (khoảng 71%)	Chỉ khảo sát công cụ tĩnh; không đề cập tầng API

Công trình	Năm	Đối tượng khảo sát	Kết quả chính	Hạn chế so với đồ án
Mitra & Ranganath [18]	2017	Xây dựng tập chuẩn lỗ hổng Ghera	Cung cấp 25 lỗ hổng Android tái hiện được, dùng làm tập chuẩn đánh giá	Là tập chuẩn, không phải nghiên cứu so sánh công cụ
Nghiên cứu MobSF trên IEEE [20]	2024	MobSF trên nhiều ứng dụng Android	Phát hiện tốt lỗ hổng cấu hình và mật mã; phân tích động bị hạn chế bởi cơ chế phát hiện root	Chỉ khảo sát một công cụ duy nhất
Các khung kiểm thử kết hợp	2024–2025	Kết hợp công cụ tĩnh và công cụ tấn công động	Chứng minh hiệu quả của cách tiếp cận kết hợp, có ánh xạ OWASP Mobile Top 10	Không benchmark định lượng; không bao phủ API Security Top 10

Từ tổng quan trên, có thể xác định ba khoảng trống nghiên cứu mà đồ án hướng tới. Khoảng trống thứ nhất là chưa có nghiên cứu nào benchmark đồng thời cả bảy công cụ thuộc cả ba phương pháp phân tích trên cùng một ứng dụng đích, với cùng một tập lỗ hổng đã biết và cùng một khung tiêu chí đo lường — điều kiện cần để kết quả giữa các công cụ thực sự so sánh được với nhau. Khoảng trống thứ hai là phần lớn các nghiên cứu hiện có đánh giá công cụ theo OWASP Mobile Top 10, tức chỉ nhìn từ phía ứng dụng, trong khi nhóm lỗ hổng nguy hiểm nhất đối với dữ liệu người dùng lại nằm ở tầng API và được mô tả trong OWASP API Security Top 10. Khoảng trống thứ ba là các nghiên cứu thường dừng ở kết luận định tính về ưu nhược điểm của từng công cụ, chưa đề xuất một quy trình cụ thể để phối hợp các công cụ đó trong điều kiện triển khai thực tế.

1.5. Kết luận chương 1

Chương 1 đã trình bày toàn bộ nền tảng lý thuyết làm cơ sở cho việc nghiên cứu và đánh giá các công cụ tự động phát hiện lỗ hổng bảo mật API của ứng dụng Android. Cụ thể, chương đã phân tích kiến trúc giao tiếp API theo mô hình client – server của ứng dụng Android cùng vòng đời của một yêu cầu API, qua đó làm rõ sự bất đối xứng căn bản trong khả năng kiểm soát giữa phía client và phía máy chủ — cơ sở giải thích vì sao mọi biện pháp kiểm tra cài đặt duy nhất ở phía ứng dụng đều không có giá trị về an toàn; trình bày vai trò của giao thức TLS và cơ chế Certificate Pinning, đồng thời chỉ ra nghịch lý mà cơ chế này đặt ra đối với công việc kiểm thử

hợp pháp; và hệ thống hoá bề mặt tấn công API thành ba tầng tương ứng với ba vùng trách nhiệm kỹ thuật khác nhau.

Trên nền đó, chương đã hệ thống hoá hai bộ tiêu chuẩn OWASP Mobile Top 10 phiên bản 2024 và OWASP API Security Top 10 phiên bản 2023, phân tích những thay đổi quan trọng của từng phiên bản so với bản trước, làm rõ quan hệ bổ sung giữa hai bộ tiêu chuẩn và xác định tập lỗ hổng cụ thể nằm trong phạm vi thực nghiệm của đồ án. Chương cũng đã trình bày ba phương pháp kiểm tra bảo mật — phân tích tĩnh, phân tích động dựa trên máy chủ uỷ nhiệm và phân tích động bằng can thiệp thời gian chạy — cùng ranh giới năng lực của từng phương pháp, được tổng hợp trong Bảng 1.3. Kết luận quan trọng nhất rút ra từ bảng này là không một phương pháp đơn lẻ nào bao phủ được toàn bộ các loại lỗ hổng, và do đó việc kết hợp nhiều công cụ không phải là lựa chọn để tăng độ tin cậy mà là điều kiện bắt buộc để đạt được độ phủ chấp nhận được.

Cuối cùng, chương đã khảo sát tổng quan bảy công cụ được lựa chọn đưa vào nghiên cứu, phân loại chúng theo phương pháp phân tích và xác định nhóm lỗ hổng mà mỗi công cụ hướng tới, đồng thời tổng hợp tình hình nghiên cứu liên quan và chỉ ra ba khoảng trống nghiên cứu mà đồ án hướng tới lấp đầy: thiếu một benchmark đồng thời nhiều công cụ trên cùng một khung đo lường, thiếu góc nhìn từ tầng API bên cạnh góc nhìn từ phía ứng dụng, và thiếu một quy trình kết hợp công cụ có thể áp dụng trong thực tế.

Những nội dung này là cơ sở cần thiết để Chương 2 đi sâu phân tích kiến trúc, tính năng và cách triển khai chi tiết của từng công cụ, mô tả môi trường thực nghiệm được xây dựng trên nền máy ảo Android kiến trúc ARM64 cùng ứng dụng đích có lỗ hổng có chủ đích InsecureBankv2 [17], và thiết kế ma trận kiểm thử cùng khung bảy tiêu chí đánh giá sẽ được sử dụng trong phần thực nghiệm.

CHƯƠNG 2. CÔNG CỤ, MÔI TRƯỜNG THỰC NGHIỆM VÀ THIẾT KẾ ĐÁNH GIÁ

Chương 1 đã kết luận rằng không một phương pháp kiểm tra đơn lẻ nào bao phủ được toàn bộ các loại lỗ hổng bảo mật API của ứng dụng Android, và rằng khoảng trống nghiên cứu lớn nhất hiện nay là thiếu một phép đo thống nhất cho phép so sánh các công cụ với nhau. Chương 2 giải quyết bước tiếp theo của bài toán đó: chuẩn bị đầy đủ điều kiện để phép đo có thể diễn ra một cách khách quan.

Nội dung chương gồm năm phần nối tiếp nhau theo đúng trình tự mà một thực nghiệm cần được chuẩn bị. Mục 2.1 phân tích chi tiết cơ chế hoạt động, cách vận hành và ranh giới năng lực của từng công cụ trong bảy công cụ đã lựa chọn ở Chương 1, kết thúc bằng một sơ đồ quyết định giúp ánh xạ từ đặc trưng của lỗ hổng sang công cụ phù hợp. Mục 2.2 phân tích ứng dụng đích InsecureBankv2: luồng nghiệp vụ, mô hình mối đe doạ và tập lỗ hổng đã biết đóng vai trò đáp án chuẩn. Mục 2.3 mô tả môi trường thực nghiệm được xây dựng, cách triển khai từng công cụ và những sự cố kỹ thuật đã gặp phải. Mục 2.4 trình bày mười hai kịch bản kiểm thử chi tiết, mỗi kịch bản tương ứng với một lỗ hổng trong đáp án. Mục 2.5 trình bày ma trận kiểm thử, quy ước gán nhãn kết quả, bảy tiêu chí đánh giá định lượng và các mối đe doạ tới tính hợp lệ của phép đo.

Nguyên tắc xuyên suốt của chương là toàn bộ thiết kế được cố định trước khi chạy công cụ đầu tiên. Nếu ma trận kiểm thử hoặc tiêu chí chấm điểm được điều chỉnh sau khi đã nhìn thấy kết quả, kết luận rút ra sẽ mang tính hợp lý hoá chứ không còn là kết luận thực nghiệm. Vì vậy chương này được viết và hoàn tất như một bản thiết kế độc lập, có thể đem cho người khác thực hiện lại mà vẫn thu được kết quả so sánh được.

2.1. Phân tích chi tiết các công cụ được lựa chọn

Chương 1 đã giới thiệu tổng quan bảy công cụ và phân loại chúng theo ba phương pháp phân tích. Mục này đi sâu hơn một bậc. Với mỗi công cụ, chương làm rõ bốn nội dung: cơ chế kỹ thuật mà công cụ dựa vào để phát hiện lỗ hổng; cách vận hành cụ thể trong môi trường thực nghiệm của đồ án, bao gồm giao diện lập trình nếu có;

định dạng đầu ra và mức độ có thể xử lý tự động của đầu ra đó; và cuối cùng là ranh giới năng lực — tức những gì công cụ về nguyên tắc không thể phát hiện.

Nội dung cuối cùng là quan trọng nhất đối với chất lượng của phép đo. Khi một công cụ không báo một lỗ hổng, có hai khả năng hoàn toàn khác nhau về bản chất: hoặc công cụ có khả năng nhìn thấy lỗ hổng đó nhưng bộ luật của nó không đủ tốt, hoặc phương pháp mà công cụ dựa trên về nguyên tắc không tiếp cận được loại lỗ hổng đó. Trường hợp thứ nhất là điểm yếu của công cụ; trường hợp thứ hai không phải. Nếu không phân biệt được hai trường hợp này ngay từ khâu thiết kế, kết quả benchmark sẽ trở thành một bảng xếp hạng vô nghĩa, trong đó công cụ đa năng luôn thắng công cụ chuyên dụng chỉ vì nó được đo trên nhiều hạng mục hơn.

Bảng 2.1. Đặc trưng kỹ thuật của bảy công cụ được lựa chọn

Công cụ	Phương pháp	Ngôn ngữ	Giấy phép	Điều kiện triển khai bắt buộc
MobSF	Tĩnh + Động	Python	MIT	Docker ARM64; cần máy ảo đang chạy cho chế độ động
Burp Suite CE	Động – uỷ nhiệm	Java	Community miễn phí	JDK; cài chứng chỉ CA vào máy ảo; cấu hình proxy
OWASP ZAP	Động – uỷ nhiệm	Java	Apache 2.0	Docker; cài chứng chỉ CA; điều khiển qua REST API
Frida	Động – thời gian chạy	C, Python, JS	wxWindows	frida-server đúng kiến trúc ARM64; quyền root trên máy ảo
objection	Động – thời gian chạy	Python	MIT	Phụ thuộc Frida; cài qua pip
JADX	Tĩnh – dịch ngược	Java	Apache 2.0	JDK 17 trở lên; không cần máy ảo
apktool	Tĩnh – dịch ngược	Java	Apache 2.0	JDK 17 trở lên; không cần máy ảo

2.1.1. MobSF – khung phân tích tự động kết hợp

MobSF là công cụ duy nhất trong nhóm khảo sát tự động hoá được cả hai chế độ phân tích trong một nền tảng thống nhất [10]. Ở chế độ tĩnh, quy trình xử lý của MobSF gồm bốn bước nối tiếp. Bước thứ nhất, công cụ giải nén tệp APK và khôi phục tệp kê khai AndroidManifest.xml từ dạng nhị phân về dạng XML đọc được, từ đó liệt kê toàn bộ thành phần của ứng dụng, các quyền được khai báo, và các thuộc tính cấu hình nhạy cảm như allowBackup hay debuggable. Bước thứ hai, MobSF dịch ngược bytecode Dalvik sang mã Java để phục vụ việc quét mã. Bước thứ ba, công cụ chạy bộ luật tích hợp trên toàn bộ mã nguồn và tài nguyên nhằm tìm các mẫu nguy hiểm đã

biết: lời gọi tới các thuật toán mật mã lỗi thời, chuỗi ký tự trông giống khoá bí mật, cấu hình mạng cho phép lưu lượng không mã hoá. Bước thứ tư, kết quả được phân loại theo mức độ nguy hiểm, ánh xạ sang mã CWE và sang các mục của OWASP Mobile Top 10, đồng thời tổng hợp thành một điểm số an toàn chung.

Cần nói rõ về bản chất của bộ luật quét. Bộ luật của MobSF hoạt động chủ yếu theo cơ chế so khớp mẫu trên mã đã dịch ngược, chứ không phải phân tích luồng dữ liệu. Điều đó có hai hệ quả trái ngược nhau. Hệ quả tích cực là tốc độ rất cao và độ ổn định tốt: cùng một tệp APK luôn cho cùng một kết quả, không phụ thuộc vào thứ tự quét hay tài nguyên máy. Hệ quả tiêu cực là công cụ không phân biệt được một chuỗi ký tự nhìn giống khoá bí mật nhưng thực chất chỉ là dữ liệu mẫu, hoặc một lời gọi hàm mật mã yếu nằm trong nhánh mã không bao giờ được thực thi. Đây chính là nguồn gốc chủ yếu của báo động giả ở nhóm công cụ phân tích tĩnh, và là lý do tiêu chí tỷ lệ báo động giả được đưa vào khung đánh giá.

Ở chế độ động, MobSF yêu cầu một máy ảo đang chạy và có quyền quản trị cao nhất. Công cụ tự cài đặt ứng dụng, khởi chạy, ghi lại lưu lượng mạng qua một máy chủ uỷ nhiệm nội bộ, theo dõi các thao tác đọc ghi tệp và cơ sở dữ liệu, đồng thời chụp màn hình quá trình thực thi. Điểm cần lưu ý cho phần thực nghiệm là chế độ động của MobSF hoạt động theo hướng quan sát thụ động: nó ghi lại những gì ứng dụng làm chứ không chủ động gửi các yêu cầu đã sửa đổi tới máy chủ. Hệ quả trực tiếp là MobSF không có khả năng phát hiện nhóm lỗ hổng phân quyền ở phía máy chủ, dù chạy ở chế độ động — một ranh giới cần ghi nhận rõ để không quy kết nhầm cho chất lượng của công cụ.

Đặc điểm khiến MobSF phù hợp đặc biệt với đề tài là việc công cụ cung cấp REST API cho phép điều khiển toàn bộ vòng đời một lượt quét bằng kịch bản [10]. Bảng 2.2 liệt kê các điểm cuối được sử dụng trong thực nghiệm của đồ án.

Bảng 2.2. Các điểm cuối REST API của MobSF dùng trong thực nghiệm

Điểm cuối	Phương thức	Chức năng trong quy trình benchmark
/api/v1/upload	POST	Tải tệp APK lên máy chủ MobSF, trả về mã băm định danh của lượt phân tích
/api/v1/scan	POST	Kích hoạt quét tĩnh trên mã băm vừa nhận; đây là bước tiêu tốn thời gian chính, được đo để tính tiêu chí tốc độ

Điểm cuối	Phương thức	Chức năng trong quy trình benchmark
/api/v1/report_json	POST	Tải kết quả đầy đủ dưới dạng JSON để xử lý tự động
/api/v1/download_pdf	POST	Tải báo cáo PDF, dùng làm minh chứng đính kèm khi chấm tiêu chí chất lượng báo cáo
/api/v1/delete_scan	POST	Xoá kết quả lượt quét, phục vụ bước đưa môi trường về trạng thái sạch

Kết quả JSON của MobSF có cấu trúc phân cấp theo nhóm phát hiện, trong đó mỗi phát hiện gồm mô tả, mức độ nguy hiểm, vị trí trong mã và ánh xạ CWE. Cấu trúc này cho phép viết một bộ chuyển đổi tự động từ đầu ra của MobSF về lược đồ chung dùng cho cả bảy công cụ, trình bày ở mục 2.5. Ảnh Docker chính thức hỗ trợ kiến trúc ARM64, điều kiện cần thiết để công cụ chạy được nguyên bản trên máy Apple Silicon.

2.1.2. Burp Suite Community Edition

Burp Suite hoạt động như một máy chủ uỷ nhiệm đặt giữa ứng dụng và máy chủ [11]. Về mặt kỹ thuật, khi máy ảo Android được cấu hình trỏ lưu lượng qua Burp và chứng chỉ của Burp đã được cài vào kho chứng chỉ tin cậy của hệ thống, mỗi kết nối HTTPS từ ứng dụng sẽ bị tách thành hai phiên TLS độc lập; Burp giải mã, hiển thị nội dung ở dạng rõ, và mã hoá lại trước khi chuyển tiếp tới máy chủ thật. Toàn bộ tiêu đề và thân dữ liệu của yêu cầu cũng như phản hồi đều quan sát và sửa đổi được.

Bảng 2.3 liệt kê các thành phần của Burp Suite được sử dụng trong thực nghiệm cùng vai trò cụ thể của từng thành phần.

Bảng 2.3. Các thành phần của Burp Suite Community sử dụng trong kiểm thử API

Thành phần	Vai trò trong thực nghiệm	Lỗ hổng hướng tới
Proxy – Intercept	Tạm dừng từng yêu cầu trước khi rời thiết bị để quan sát trường dữ liệu được gửi đi và cách gửi	V02, V09
Proxy – HTTP history	Lưu toàn bộ lưu lượng của phiên làm việc, dùng để lập danh sách điểm cuối thực tế được gọi	V07
Repeater	Gửi lại một yêu cầu đã bắt được với tham số đã sửa đổi; công cụ chủ lực để kiểm tra phân quyền	V07, V08, V09
Intruder	Tự động thử nhiều giá trị trên một vị trí tham số, phục vụ kiểm tra giới hạn tần suất và chính sách mật khẩu	V12
Comparer	So sánh hai phản hồi để xác định chênh lệch trường dữ liệu, phục vụ phát hiện lộ dữ liệu quá mức	V08

Hạn chế quan trọng nhất của phiên bản Community là không có bộ quét chủ động và không có giao diện lập trình để điều khiển từ bên ngoài. Điều này có ba hệ quả đối với thiết kế thực nghiệm. Thứ nhất, phần lớn công việc mang tính bán thủ

công, do đó tiêu chí mức độ dễ sử dụng và tốc độ phân tích của Burp Suite sẽ phản ánh phần lớn là thời gian thao tác của người kiểm thử chứ không phải thời gian tính toán của máy. Thứ hai, để kết quả so sánh được với các công cụ tự động, quy trình thao tác thủ công phải được chuẩn hoá thành một kịch bản cố định, thực hiện giống nhau ở mọi lượt chạy — yêu cầu này được cụ thể hoá thành các kịch bản kiểm thử ở mục 2.4. Thứ ba, kết quả của Burp Suite phải được ghi lại thủ công theo cùng lược đồ dữ liệu với các công cụ có API, nếu không thì bước tổng hợp tự động sẽ không áp dụng được cho công cụ này.

2.1.3. OWASP ZAP

OWASP ZAP có kiến trúc uỷ nhiệm tương tự Burp Suite nhưng khác biệt ở ba điểm có ý nghĩa trực tiếp với đề tài [12]. Thứ nhất, ZAP tích hợp sẵn hai cơ chế quét: Passive Scan chạy nền, phân tích mọi lưu lượng đi qua để phát hiện các dấu hiệu như thiếu tiêu đề bảo mật, sử dụng giao thức không mã hoá, hoặc rò rỉ thông tin trong phản hồi; và Active Scan chủ động sinh ra các yêu cầu tấn công gửi tới những điểm cuối đã phát hiện. Nói cách khác, ZAP tự động hoá được chính phần công việc mà Burp Suite Community yêu cầu làm thủ công, và đó là lý do hai công cụ này được đưa vào cùng một nghiên cứu để đối chiếu mức độ tự động hoá.

Thứ hai, ZAP cung cấp REST API đầy đủ và có thể chạy ở chế độ nền trong Docker, cho phép điều khiển toàn bộ vòng đời một lượt quét bằng kịch bản. Bảng 2.4 liệt kê các lời gọi được sử dụng trong thực nghiệm. Thứ ba, ZAP là công cụ mã nguồn mở hoàn toàn theo giấy phép Apache 2.0, không có ràng buộc thương mại — một yếu tố được tính đến trong tiêu chí chi phí ở khung đánh giá.

Bảng 2.4. Các lời gọi REST API của OWASP ZAP dùng trong thực nghiệm

Lời gọi API	Chức năng trong quy trình benchmark
/JSON/core/action/newSession	Khởi tạo phiên làm việc mới, bảo đảm lượt chạy không mang theo dữ liệu của lượt trước
/JSON/core/action/accessUrl	Nạp thủ công từng điểm cuối đã biết vào cây địa chỉ của ZAP, thay cho bước dò tìm tự động vốn không áp dụng được với API di động
/JSON/ascan/action/scan	Kích hoạt quét chủ động trên các điểm cuối đã nạp
/JSON/ascan/view/status	Theo dõi tiến độ quét, dùng để xác định thời điểm kết thúc khi đo tốc độ phân tích
/JSON/core/view/alerts	Lấy toàn bộ cảnh báo đã sinh ra, kèm mức độ rủi ro và mức độ tin cậy do ZAP tự đánh giá

Lời gọi API	Chức năng trong quy trình benchmark
/OTHER/core/other/htmlreport	Xuất báo cáo HTML dùng làm minh chứng

Một chi tiết kỹ thuật cần lưu ý là cơ chế dò tìm tự động của ZAP được thiết kế cho ứng dụng web, tức dựa trên việc lần theo các liên kết trong trang HTML. Ứng dụng di động không có cấu trúc liên kết như vậy: các điểm cuối chỉ lộ ra khi ứng dụng gọi tới chúng trong quá trình sử dụng. Vì vậy trong thực nghiệm, danh sách điểm cuối phải được nạp vào ZAP một cách tường minh, lấy từ hai nguồn là lịch sử lưu lượng thu được khi thao tác với ứng dụng và kết quả dịch ngược tệp APK. Đây là một minh hoạ cụ thể cho quan hệ bổ trợ giữa nhánh tĩnh và nhánh động đã nêu ở Chương 1.

Ranh giới năng lực của ZAP cần được nhận thức rõ. Bộ quét chủ động hoạt động tốt với các lỗ hổng có đặc trưng kỹ thuật rõ ràng: tiêm nhiễm, cấu hình sai, thiếu giới hạn tần suất. Ngược lại, lỗ hổng phân quyền mức đối tượng không có đặc trưng kỹ thuật nào để máy nhận ra — muốn phát hiện, công cụ phải biết trước bản ghi nào thuộc về người dùng nào, tức phải có tri thức nghiệp vụ mà một bộ quét tự động không có. Do đó ZAP được kỳ vọng phát hiện một phần chứ không phải toàn bộ nhóm lỗ hổng phía máy chủ.

2.1.4. Frida và objection

Frida hoạt động bằng cách chèn một máy ảo JavaScript vào không gian địa chỉ của tiến trình đích, từ đó cho phép móc vào bất kỳ hàm nào của ứng dụng hoặc của thư viện hệ thống [13]. Trên Android, việc triển khai đòi hỏi đẩy tệp nhị phân frida-server có kiến trúc khớp với bộ xử lý của thiết bị vào thư mục tạm của hệ thống và chạy nó với quyền quản trị cao nhất. Trong môi trường của đồ án, máy ảo do Android Studio tạo ra dùng ảnh hệ thống arm64-v8a nên phải sử dụng bản frida-server biên dịch cho ARM64; quyền quản trị được cấp thông qua lệnh adb root, khả dụng vì máy ảo được tạo từ ảnh hệ thống không kèm dịch vụ Google Play.

Cơ chế móc hàm của Frida có thể minh hoạ bằng một đoạn mã ngắn. Đoạn mã dưới đây móc vào lớp quản lý tin cậy chứng chỉ của nền tảng và thay thế phần thân hàm kiểm tra bằng một hàm rỗng, khiến mọi chứng chỉ đều được chấp nhận:

```
Java.perform(function () {
    var TrustManagerImpl = Java.use(
        'com.android.org.conscrypt.TrustManagerImpl');
```

```
  TrustManagerImpl.verifyChain.implementation = function (
        untrustedChain, trustAnchorChain, host, clientAuth,
        ocspData, tlsSctData) {
    console.log('[+] Bo qua kiem tra chuoi chung chi cho: ' + host);
    return untrustedChain;   // tra ve chuoi goc, coi nhu hop le
  };
});
```

Đoạn mã trên cho thấy bản chất của phương pháp: công cụ không tìm lỗ hổng, mà thay đổi hành vi của ứng dụng để người kiểm thử quan sát được điều vốn bị che khuất. Cùng cơ chế đó, Frida còn được dùng để móc vào các hàm mật mã nhằm đọc khoá và dữ liệu ở dạng rõ ngay trước thời điểm mã hoá, và móc vào các hàm ghi tệp nhằm xác định chính xác dữ liệu nhạy cảm nào được lưu xuống thiết bị và lưu ở đâu. Trong tổng thể thiết kế, Frida giữ vai trò hỗ trợ: nó mở đường cho nhóm công cụ uỷ nhiệm hoạt động khi gặp cơ chế ghim chứng chỉ, và cung cấp bằng chứng trực tiếp cho nhóm lỗ hổng lưu trữ và mật mã.

objection được xây dựng trên nền Frida và đóng gói các kịch bản phổ biến nhất thành các lệnh dòng lệnh đơn giản [14]. Bảng 2.5 liệt kê các nhóm lệnh được sử dụng trong thực nghiệm.

Bảng 2.5. Các nhóm lệnh của objection sử dụng trong thực nghiệm

Nhóm lệnh	Chức năng	Lỗ hổng hướng tới
android sslpinning disable	Áp dụng đồng thời nhiều kỹ thuật vô hiệu hoá ghim chứng chỉ mà không cần viết mã móc hàm	V03
android hooking list classes	Liệt kê toàn bộ lớp đã nạp trong tiến trình, dùng để xác định lớp cần móc	—
android keystore list	Liệt kê nội dung kho khoá của ứng dụng	V01, V05
android shell_exec	Thực thi lệnh trong ngữ cảnh ứng dụng để đọc thư mục dữ liệu riêng	V04
android hooking watch class_method	Theo dõi lời gọi tới một phương thức cụ thể kèm tham số và giá trị trả về	V05, V10

Việc đưa cả Frida lẫn objection vào nghiên cứu cho phép so sánh trực tiếp giữa một công cụ linh hoạt nhưng đòi hỏi kỹ năng lập trình và một công cụ đóng gói sẵn nhưng kém linh hoạt hơn — đây chính là nội dung của tiêu chí mức độ dễ sử dụng. Điểm cần nhấn mạnh là cả hai đều không tự động phát hiện lỗ hổng theo nghĩa thông thường. Vì vậy, khi tính chỉ số phát hiện cho hai công cụ này, một lỗ hổng chỉ được

tính là phát hiện đúng khi kịch bản móc hàm đã chuẩn bị sẵn cho ra bằng chứng trực tiếp — chẳng hạn in ra được khoá mật mã ở dạng rõ — chứ không tính những trường hợp người kiểm thử tự suy luận từ ngữ cảnh.

2.1.5. JADX và apktool

JADX chuyển bytecode Dalvik trong các tệp .dex sang mã nguồn Java hoặc Kotlin có thể đọc được [15]. Trong thiết kế thực nghiệm, JADX đảm nhiệm hai vai trò. Vai trò thứ nhất là lập bản đồ các điểm cuối API mà ứng dụng gọi tới: bằng cách tìm kiếm các chuỗi ký tự chứa địa chỉ và các lớp cấu hình mạng trong mã đã dịch ngược, người kiểm thử thu được danh sách điểm cuối trước khi bắt đầu kiểm thử động — đây chính là đường nối giữa nhánh tĩnh và nhánh động trong quy trình. Vai trò thứ hai là kiểm chứng chéo: khi một công cụ tự động báo có thông tin xác thực nhúng trong mã hoặc có lời gọi thuật toán mật mã yếu, kết quả đó được xác nhận bằng cách đọc trực tiếp đoạn mã tương ứng trong JADX. Vai trò kiểm chứng này là cơ sở để phân biệt giữa phát hiện đúng và báo động giả.

apktool làm việc ở mức thấp hơn: nó giải nén tệp APK thành mã smali cùng các tệp tài nguyên và tệp kê khai đã được khôi phục về dạng XML [16]. Mã smali khó đọc hơn nhiều so với đầu ra của JADX, nhưng phản ánh chính xác nội dung tệp nhị phân và không chịu ảnh hưởng của những sai sót có thể phát sinh trong quá trình dịch ngược lên mã bậc cao. Trong thực nghiệm, apktool được dùng chủ yếu để đọc chính xác tệp kê khai — nguồn thông tin chuẩn để xác nhận các lỗ hổng cấu hình như cho phép sao lưu dữ liệu hay thành phần được xuất mà không đặt quyền bảo vệ.

Sự khác biệt giữa hai công cụ thể hiện rõ qua một ví dụ. Với thuộc tính allowBackup, JADX hiển thị tệp kê khai đã được diễn giải và trong một số trường hợp bỏ qua các thuộc tính mặc định; apktool khôi phục nguyên trạng và cho biết chính xác thuộc tính đó được khai báo tường minh hay chỉ nhận giá trị mặc định của nền tảng. Phân biệt này có ý nghĩa thực chất khi chấm điểm: một thuộc tính nhận giá trị mặc định không phải là lỗi do lập trình viên gây ra, còn một thuộc tính được khai báo tường minh sai thì có.

Ranh giới năng lực của cả hai công cụ là rõ ràng và cần được ghi nhận: chúng không phải công cụ quét lỗ hổng mà là công cụ đọc mã. Chúng không sinh ra cảnh

báo, do đó không có khái niệm báo động giả đối với chúng; chỉ số phát hiện của JADX và apktool trong thực nghiệm phản ánh số lỗ hổng mà người kiểm thử xác nhận được bằng cách đọc mã do công cụ tạo ra, chứ không phải số cảnh báo do công cụ tự đưa ra.

2.1.6. So sánh tổng hợp và cơ sở phân công vai trò

Từ phân tích ở năm tiểu mục trên, Bảng 2.6 tổng hợp bảy công cụ theo năm chiều có ảnh hưởng trực tiếp tới thiết kế thực nghiệm: mức độ tự động hoá, khả năng điều khiển bằng kịch bản, yêu cầu về môi trường, dạng đầu ra, và vai trò được phân công trong nghiên cứu này.

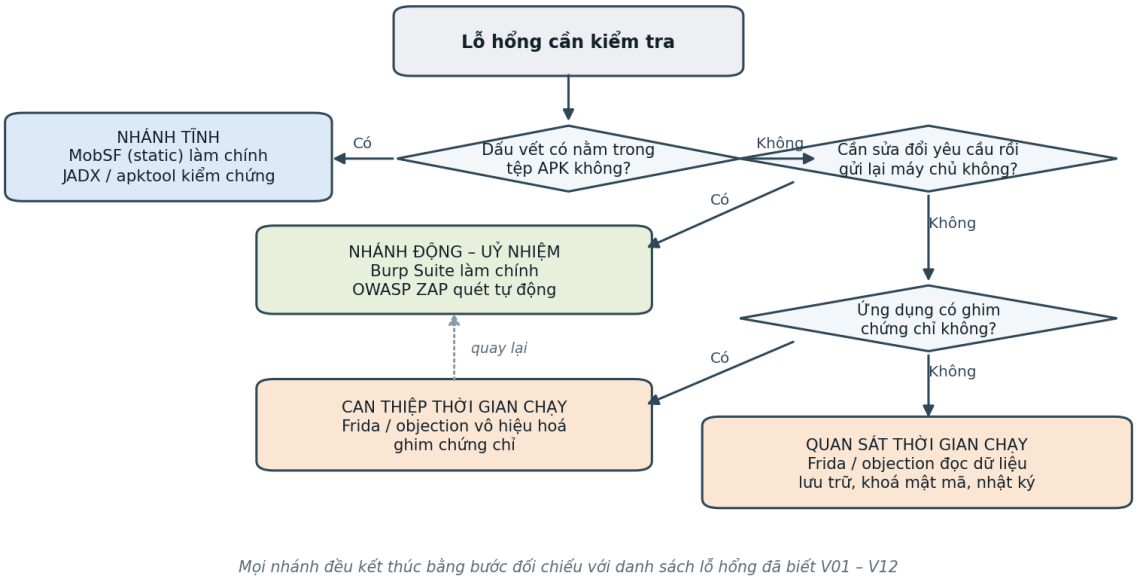
Bảng 2.6. So sánh tổng hợp bảy công cụ theo năm chiều

Công cụ	Mức tự động hoá	Điều khiển bằng kịch bản	Yêu cầu môi trường	Dạng đầu ra	Vai trò trong nghiên cứu
MobSF	Cao	REST API đầy đủ	Docker; máy ảo cho chế độ động	JSON, PDF	Phát hiện chính nhóm lỗ hổng phía ứng dụng
Burp Suite CE	Thấp	Không có	JDK; chứng chỉ CA; proxy	Thủ công	Phát hiện chính nhóm lỗ hổng phía máy chủ
OWASP ZAP	Trung bình – cao	REST API đầy đủ	Docker; chứng chỉ CA	JSON, HTML	Đối chứng tự động hoá với Burp Suite
Frida	Thấp	Kịch bản JavaScript	Quyền root; frida-server ARM64	Nhật ký văn bản	Hỗ trợ vượt ghim chứng chỉ; quan sát thời gian chạy
objection	Trung bình	Lệnh dòng lệnh	Phụ thuộc Frida	Nhật ký văn bản	Phiên bản đóng gói sẵn của Frida, dùng đối chứng
JADX	Không	Giao diện dòng lệnh	JDK	Mã Java	Kiểm chứng kết quả tĩnh; lập bản đồ điểm cuối
apktool	Không	Giao diện dòng lệnh	JDK	Mã smali, XML	Kiểm chứng tệp kê khai và cấu hình

Bảng 2.6 làm lộ ra một quy luật đáng chú ý: mức độ tự động hoá và chiều sâu phát hiện có xu hướng nghịch nhau. Hai công cụ tự động hoá cao nhất là MobSF và OWASP ZAP đều dựa trên bộ luật cố định, do đó chỉ phát hiện được những lỗ hổng có đặc trưng kỹ thuật nhận dạng được; còn hai công cụ đòi hỏi nhiều thao tác thủ công nhất là Burp Suite và Frida lại là phương tiện duy nhất tiếp cận được những lỗ hổng nằm ở logic nghiệp vụ. Quan sát này không phải là một nhận xét hiển nhiên mà là một giả thuyết sẽ được kiểm chứng bằng số liệu ở chương sau; nếu nó đúng, hệ quả thực tiễn là một quy trình kiểm tra bảo mật API không thể chỉ dựa vào tự động hoá.

Trên cơ sở phân tích ở toàn bộ mục 2.1, Hình 2.1 tổng hợp logic lựa chọn công cụ dưới dạng một sơ đồ quyết định. Sơ đồ này đi từ đặc trưng của lỗ hổng cần kiểm tra

chứ không đi từ tên công cụ, và chính vì vậy nó có giá trị sử dụng vượt ra ngoài phạm vi bảy công cụ của đồ án: khi xuất hiện một công cụ mới, chỉ cần xác định nó thuộc nhánh nào của sơ đồ là biết được nó có thể thay thế cho công cụ nào.



Hình 2.1. Sơ đồ quyết định lựa chọn công cụ theo đặc trưng của lỗ hổng

Sơ đồ có ba câu hỏi phân nhánh. Câu hỏi thứ nhất — dấu vết của lỗ hổng có nằm trong tệp APK hay không — tách nhóm lỗ hổng phía ứng dụng khỏi nhóm phía máy chủ, và là câu hỏi quyết định nhất. Câu hỏi thứ hai — có cần sửa đổi yêu cầu rồi gửi lại máy chủ hay không — tách nhóm lỗ hổng cần tương tác chủ động khỏi nhóm chỉ cần quan sát. Câu hỏi thứ ba — ứng dụng có ghim chứng chỉ hay không — xác định xem có phải chèn thêm bước can thiệp thời gian chạy trước khi dùng công cụ uỷ nhiệm hay không. Đường nét đứt trong sơ đồ thể hiện đúng quan hệ hỗ trợ này: sau khi ghim chứng chỉ bị vô hiệu hoá, luồng kiểm thử quay trở lại nhánh uỷ nhiệm chứ không kết thúc tại công cụ can thiệp.

2.2. Ứng dụng đích InsecureBankv2

2.2.1. Lý do lựa chọn ứng dụng đích

Chất lượng của một phép đo benchmark phụ thuộc trước hết vào chất lượng của đáp án chuẩn. Nếu tập lỗ hổng thực sự tồn tại trong ứng dụng đích không được biết trước một cách chắc chắn, thì mọi cảnh báo do công cụ đưa ra đều không thể phân loại

thành phát hiện đúng hay báo động giả, và chỉ số phát hiện trở nên vô nghĩa. Vì lý do đó, đồ án không sử dụng một ứng dụng thương mại thật mà chọn một ứng dụng có lỗ hổng có chủ đích — loại ứng dụng được thiết kế riêng cho mục đích học tập và kiểm thử, trong đó từng lỗ hổng được cài đặt một cách cố ý và được ghi chép công khai.

Lựa chọn này cũng loại bỏ một rủi ro pháp lý và đạo đức. Kiểm thử xâm nhập trên một ứng dụng thương mại mà không có sự cho phép của chủ sở hữu là hành vi vi phạm pháp luật ở phần lớn các quốc gia, kể cả khi mục đích là nghiên cứu. Ứng dụng có lỗ hổng có chủ đích được phát hành kèm giấy phép cho phép mọi hình thức kiểm thử, nên không đặt ra vấn đề này.

InsecureBankv2 do Dinesh Shetty phát triển được lựa chọn dựa trên bốn tiêu chí [17]. Thứ nhất, ứng dụng mô phỏng một ngân hàng di động, tức thuộc đúng nhóm ứng dụng mà đề tài quan tâm: xử lý dữ liệu nhạy cảm và giao tiếp liên tục với máy chủ qua API. Thứ hai, ứng dụng bao gồm đầy đủ hai thành phần — ứng dụng Android và máy chủ phía sau viết bằng Python Flask — nên cho phép kiểm thử cả lỗ hổng phía client lẫn lỗ hổng phía máy chủ, điều mà phần lớn ứng dụng mẫu chỉ có tệp APK không đáp ứng được. Thứ ba, tập lỗ hổng của ứng dụng đủ đa dạng, bao phủ cả nhóm phát hiện được bằng phân tích tĩnh lẫn nhóm chỉ phát hiện được bằng kiểm thử động — điều kiện cần để so sánh công bằng giữa các nhóm công cụ. Thứ tư, InsecureBankv2 được OWASP liệt kê chính thức là một trong các ứng dụng tham chiếu của bộ tài liệu kiểm thử an toàn ứng dụng di động [3], nên việc lựa chọn có căn cứ chuẩn mực chứ không dựa trên ý muốn chủ quan.

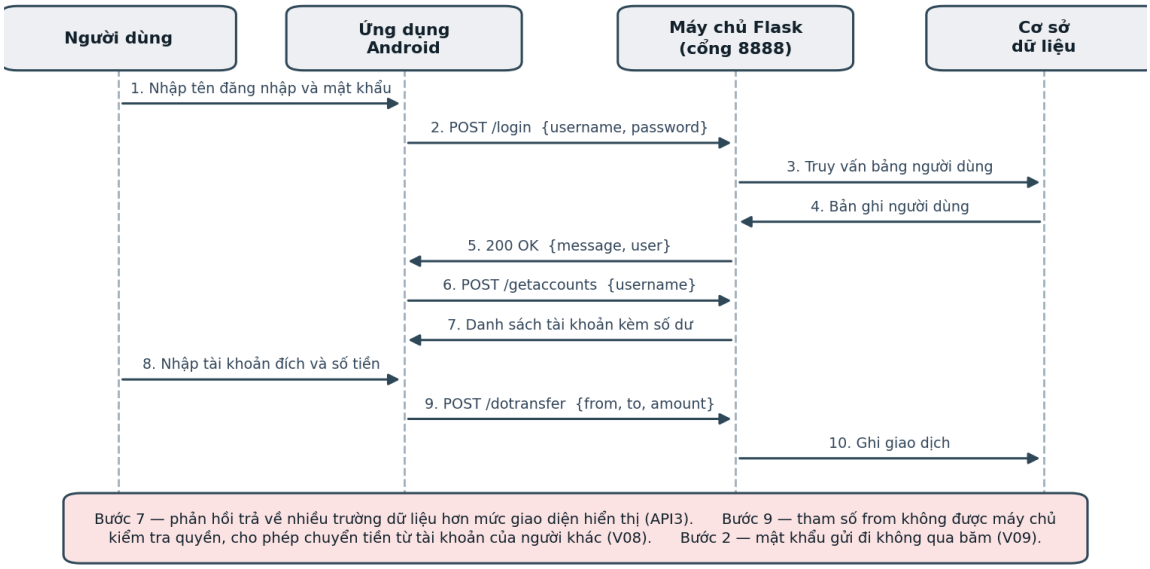
Hạn chế của lựa chọn này cũng cần được nêu thẳng. Một ứng dụng có lỗ hổng có chủ đích được viết để lỗ hổng dễ tìm, trong khi ứng dụng thương mại thật thường có mã nguồn lớn hơn nhiều lần, có làm rối mã và có nhiều nhánh xử lý gây nhiễu. Do đó các con số thu được trong thực nghiệm này nên được hiểu là cận trên của năng lực công cụ, chứ không phải mức hiệu quả kỳ vọng trong môi trường sản xuất. Nhận định này được đưa trở lại trong phần bàn về tính hợp lệ của phép đo ở mục 2.5.

2.2.2. Kiến trúc và luồng nghiệp vụ

InsecureBankv2 gồm hai thành phần chạy độc lập. Thành phần thứ nhất là ứng dụng Android đóng gói trong tệp InsecureBankv2.apk, cài đặt trên máy ảo. Thành

phần thứ hai là máy chủ phía sau viết bằng Python Flask, chạy trên máy chủ host tại cổng 8888 [24]. Hai thành phần trao đổi với nhau qua các điểm cuối HTTP theo kiến trúc REST với dữ liệu định dạng JSON. Vì máy ảo Android truy cập dịch vụ trên máy host thông qua địa chỉ đặc biệt 10.0.2.2, ứng dụng được cấu hình trỏ tới địa chỉ này thay vì localhost.

Hình 2.2 mô tả luồng nghiệp vụ chính của ứng dụng, gồm hai kịch bản người dùng nối tiếp nhau: đăng nhập và thực hiện một giao dịch chuyển khoản. Đây là luồng sẽ được dùng làm đường cơ sở cho toàn bộ các kịch bản kiểm thử động ở mục 2.4.



Hình 2.2. Luồng nghiệp vụ chính của InsecureBankv2

Đọc kỹ luồng trên Hình 2.2 có thể thấy ba đặc điểm thiết kế dẫn thẳng tới lỗ hổng. Đặc điểm thứ nhất nằm ở bước 2: mật khẩu được gửi lên máy chủ ở dạng có thể khôi phục chứ không phải giá trị băm, nên bất kỳ ai quan sát được kênh truyền đều đọc được mật khẩu gốc. Đặc điểm thứ hai nằm ở bước 6 và 7: ứng dụng không dùng mã thông báo phiên mà gửi lại tên người dùng và mật khẩu ở mọi yêu cầu, kể cả yêu cầu lấy danh sách tài khoản, nên thông tin đăng nhập đi qua kênh truyền nhiều lần; riêng chức năng đổi mật khẩu chỉ mang theo tên người dùng, nghĩa là máy chủ định danh người gọi bằng chính tham số do người gọi cung cấp. Đặc điểm thứ ba nằm ở bước 9: tham số chỉ định tài khoản nguồn của giao dịch cũng do phía client cung cấp và không được đối chiếu với danh tính của người đang đăng nhập.

Ba đặc điểm này cùng chia sẻ một nguyên nhân gốc duy nhất: máy chủ tin tưởng vào dữ liệu do phía client gửi lên để xác định người gọi là ai và được phép làm gì. Đây chính là biểu hiện cụ thể của nguyên tắc đã nêu ở mục 1.1 của Chương 1 — mọi kiểm tra cài đặt duy nhất ở phía client đều không có giá trị về an toàn. Việc ứng dụng đích thể hiện nguyên nhân gốc này một cách rõ ràng là một lý do nữa khiến nó phù hợp làm đối tượng nghiên cứu.

2.2.3. Các điểm cuối API

Bảng 2.7 liệt kê các điểm cuối API chính của ứng dụng cùng rủi ro liên quan tới từng điểm cuối. Danh sách này được xây dựng từ hai nguồn độc lập và đối chiếu chéo với nhau: tài liệu chính thức của dự án, và kết quả dịch ngược ứng dụng bằng JADX để tìm các chuỗi địa chỉ trong mã. Cách làm hai nguồn nhằm tránh bỏ sót những điểm cuối có trong mã nhưng không được tài liệu nhắc tới — mà đó thường lại là những điểm cuối nguy hiểm nhất, chẳng hạn cửa hậu dành cho nhà phát triển.

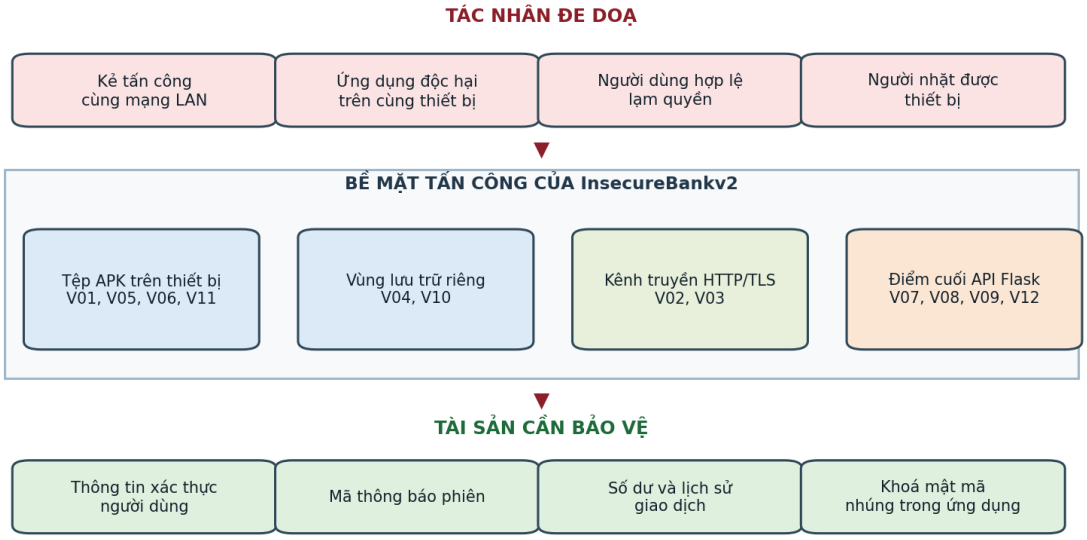
Bảng 2.7. Các điểm cuối API chính của InsecureBankv2

Điểm cuối	Phương thức	Chức năng	Rủi ro liên quan
/login	POST	Đăng nhập bằng tên người dùng và mật khẩu	API2 – không giới hạn số lần thử; phản hồi khác nhau cho phép dò tên người dùng
/devlogin	POST	Đăng nhập dành cho nhà phát triển, bỏ qua kiểm tra thông thường	API9 – điểm cuối thử nghiệm chưa gỡ bỏ
/getaccounts	POST	Lấy danh sách tài khoản của người dùng	API2 – tên người dùng và mật khẩu gửi lại ở mọi yêu cầu, không có phiên
/dotransfer	POST	Thực hiện chuyển khoản giữa hai tài khoản	API1 – không kiểm tra quyền trên tài khoản nguồn
/changepassword	POST	Đổi mật khẩu người dùng	API2 – không xác thực: chỉ cần tên người dùng là đổi được mật khẩu của bất kỳ ai

Bảng 2.7 có vai trò quan trọng hơn một bảng liệt kê thông thường: nó chính là danh sách mục tiêu cho nhánh kiểm thử động. Với mỗi điểm cuối, kịch bản kiểm thử được chuẩn bị trước gồm một yêu cầu hợp lệ làm đường cơ sở và một hoặc nhiều yêu cầu đã sửa đổi nhằm kiểm tra giả thuyết về lỗ hổng tương ứng. Toàn bộ các kịch bản này được trình bày chi tiết ở mục 2.4.

2.2.4. Mô hình mối đe doạ

Trước khi liệt kê lỗ hổng, cần trả lời một câu hỏi nền tảng: bảo vệ cái gì, khỏi ai. Nếu bỏ qua bước này, danh sách lỗ hổng sẽ chỉ là một tập hợp các khiếm khuyết kỹ thuật không có thứ tự ưu tiên, và việc đánh giá công cụ sẽ đối xử như nhau với một lỗ hổng làm lộ toàn bộ tài khoản người dùng và một lỗ hổng chỉ có ý nghĩa lý thuyết. Mục này vì vậy xây dựng một mô hình mối đe doạ rút gọn cho ứng dụng đích, gồm ba thành phần: tài sản cần bảo vệ, tác nhân đe doạ, và ánh xạ giữa chúng qua bề mặt tấn công.



Hình 2.3. Mô hình mối đe doạ đối với ứng dụng đích

Bảng 2.8 liệt kê bốn nhóm tài sản cần bảo vệ cùng hệ quả khi mỗi nhóm bị xâm phạm. Việc xác định hệ quả là cần thiết vì nó cho phép xếp hạng mức độ nghiêm trọng một cách có căn cứ, thay vì dựa vào cảm nhận.

Bảng 2.8. Tài sản cần bảo vệ và hệ quả khi bị xâm phạm

Tài sản	Vị trí lưu trữ hoặc truyền tải	Hệ quả khi bị xâm phạm	Mức
Thông tin xác thực người dùng	SharedPreferences trên thiết bị; thân yêu cầu /login trên kênh truyền	Kẻ tấn công đăng nhập với danh nghĩa nạn nhân trên bất kỳ thiết bị nào	Nghiêm trọng
Nhật ký hệ thống (logcat)	Bộ đệm nhật ký của thiết bị (ghi khi đăng nhập)	Lộ tên đăng nhập và mật khẩu cho bất kỳ tiến trình nào đọc được nhật ký	Cao
Số dư và lịch sử giao dịch	Phản hồi của /getaccounts và /dotransfer	Lộ thông tin tài chính; thực hiện giao dịch trái phép	Nghiêm trọng

Tài sản	Vị trí lưu trữ hoặc truyền tải	Hệ quả khi bị xâm phạm	Mức
Khoá mật mã nhúng trong ứng dụng	Hằng số trong mã nguồn của tệp APK	Giải mã được mọi dữ liệu mà ứng dụng đã mã hoá, trên mọi thiết bị	Cao

Bảng 2.9 mô tả bốn nhóm tác nhân đe doạ. Cách phân nhóm dựa trên vị trí và năng lực của kẻ tấn công chứ không dựa trên động cơ, vì chỉ có vị trí và năng lực mới quyết định kẻ tấn công tiếp cận được bề mặt tấn công nào.

Bảng 2.9. Tác nhân đe doạ và kịch bản tấn công tương ứng

Tác nhân	Năng lực giả định	Kịch bản tấn công điển hình	Lỗ hổng khai thác
Kẻ tấn công cùng mạng	Đặt được thiết bị trung gian trên đường truyền, không có quyền trên thiết bị nạn nhân	Chặn bắt lưu lượng tại điểm phát Wi-Fi công cộng, đọc tên đăng nhập và mật khẩu	V02, V03
Ứng dụng độc hại cùng thiết bị	Chạy trên cùng thiết bị, không có quyền quản trị	Gọi trực tiếp thành phần được xuất của ứng dụng; đọc nhật ký hệ thống	V06, V10
Người dùng hợp lệ lạm quyền	Có tài khoản hợp lệ và thông tin đăng nhập của chính mình	Sửa tham số định danh trong yêu cầu để truy cập dữ liệu của người dùng khác	V07, V08, V09
Người chiếm được thiết bị	Có thiết bị trong tay, có thể bật chế độ gỡ lỗi hoặc nâng quyền	Trích xuất vùng dữ liệu riêng của ứng dụng; sao lưu dữ liệu qua công cụ hệ thống	V01, V04, V05, V11

Đối chiếu Bảng 2.8 với Bảng 2.9 cho thấy một kết luận có ý nghĩa đối với việc thiết kế thực nghiệm: nhóm tác nhân nguy hiểm nhất không phải là kẻ tấn công từ bên ngoài mà là người dùng hợp lệ lạm quyền. Lý do là tác nhân này không cần vượt qua bất kỳ lớp bảo vệ kỹ thuật nào — họ đã có tài khoản hợp lệ và mã thông báo hợp lệ; họ chỉ cần sửa một tham số trong yêu cầu. Đồng thời, đây cũng chính là nhóm lỗ hổng mà không một công cụ phân tích tĩnh nào phát hiện được. Nhận xét này củng cố quyết định đưa cả công cụ tĩnh lẫn công cụ động vào cùng một khung đo lường, thay vì chỉ benchmark nhóm công cụ tự động hoá cao như phần lớn nghiên cứu đã khảo sát ở Chương 1.

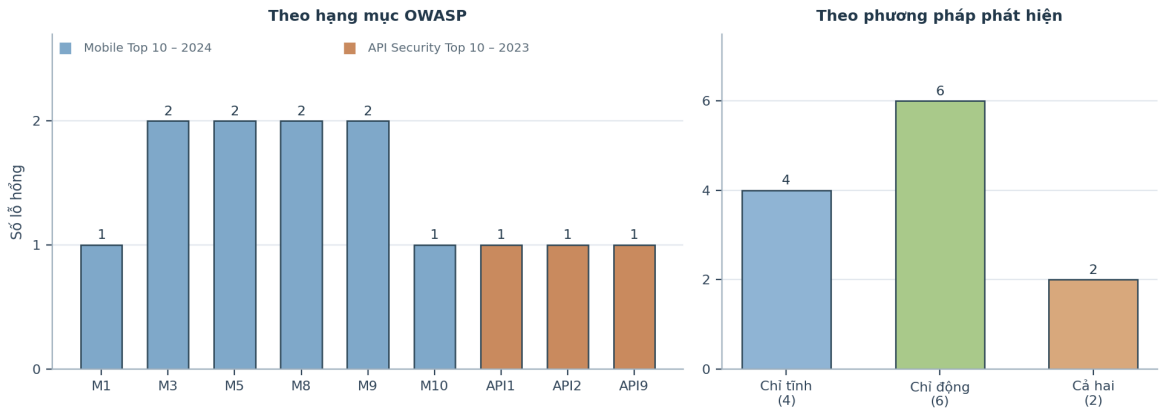
2.2.5. Danh sách lỗ hổng đã biết

Bảng 2.10 trình bày tập lỗ hổng đã biết của InsecureBankv2, được đánh mã từ V01 đến V12 và ánh xạ sang các mục tương ứng của hai bộ tiêu chuẩn OWASP đã trình bày ở Chương 1. Đây là đáp án chuẩn của toàn bộ thực nghiệm: mọi cảnh báo do công cụ đưa ra sẽ được đối chiếu với bảng này để gán nhãn phát hiện đúng hay báo động giả, và mọi lỗ hổng trong bảng mà công cụ không báo sẽ được tính là bỏ sót.

Bảng 2.10. Danh sách lỗ hổng đã biết trong InsecureBankv2

Mã	Tên lỗ hổng	OWASP	Mô tả	Nhóm
V01	Thông tin xác thực nhúng trong mã	M1	Khoá bí mật dùng để mã hoá được viết cố định trong mã nguồn	Tĩnh
V02	Lưu lượng HTTP không mã hoá	M5	Một số điểm cuối dùng HTTP thay vì HTTPS	Cả hai
V03	Không có Certificate Pinning	M5	Ứng dụng chấp nhận mọi chứng chỉ trong kho tin cậy của hệ thống	Động
V04	Lưu trữ không an toàn	M9	Tên đăng nhập (Base64) và mật khẩu (AES khoá cứng) lưu trong SharedPreferences, giải mã ngược được	Động
V05	Mật mã yếu	M10	Dùng AES/CBC với khoá viết cứng và IV tĩnh toàn số 0 để mã hoá mật khẩu	Tĩnh
V06	Thành phần exported không bảo vệ	M8	Nhiều Activity đặt exported bằng true mà không yêu cầu quyền	Tĩnh
V07	Cửa hậu dành cho nhà phát triển	API9	Điểm cuối /devlogin bỏ qua quy trình xác thực thông thường	Cả hai
V08	Phân quyền mức đối tượng bị phá vỡ	API1	Đổi tham số tài khoản nguồn (from_acc) cho phép chuyển tiền từ tài khoản của người dùng khác	Động
V09	Xác thực bị phá vỡ	M3, API2	Không có cơ chế phiên; đổi mật khẩu không cần xác thực; phản hồi đăng nhập cho phép dò tên người dùng	Động
V10	Rò rỉ dữ liệu qua nhật ký	M9	Thông tin nhạy cảm được ghi vào nhật ký hệ thống	Động
V11	Cho phép sao lưu dữ liệu	M8	Thuộc tính allowBackup đặt bằng true trong tệp kê khai	Tĩnh
V12	Chính sách mật khẩu yếu	M3	Độ phức tạp chỉ kiểm tra phía client, không giới hạn số lần đăng nhập	Động

Hình 2.4 trực quan hoá phân bố của tập lỗ hổng này theo hai chiều: theo hạng mục OWASP và theo phương pháp phát hiện.



Hình 2.4. Phân bố lỗ hổng theo hạng mục OWASP và theo phương pháp phát hiện

Biểu đồ bên trái cho thấy tập lỗ hổng trải trên chín hạng mục OWASP khác nhau, trong đó sáu hạng mục thuộc Mobile Top 10 và ba hạng mục thuộc API Security Top

10. Độ trải này là một ưu điểm của ứng dụng đích: nếu toàn bộ lỗ hổng tập trung vào một vài hạng mục thì kết quả benchmark sẽ chỉ phản ánh năng lực của công cụ trên đúng những hạng mục đó chứ không phản ánh độ phủ chung.

Biểu đồ bên phải mới là biểu đồ quan trọng đối với thiết kế thực nghiệm. Phân bố thu được là bốn lỗ hổng chỉ phát hiện được bằng phân tích tĩnh, sáu lỗ hổng chỉ phát hiện được bằng kiểm thử động, và hai lỗ hổng phát hiện được bằng cả hai phương pháp. Từ phân bố này rút ra được một dự đoán định lượng có thể kiểm chứng: một công cụ thuần tĩnh, dù hoàn hảo đến đâu, cũng chỉ có thể đạt tối đa sáu trên mười hai lỗ hổng, tương ứng năm mươi phần trăm; một công cụ thuần động tối đa tám trên mười hai, tương ứng gần sáu mươi bảy phần trăm. Không có công cụ đơn lẻ nào trong nhóm khảo sát có thể đạt mức phát hiện tuyệt đối. Nếu thực nghiệm ở chương sau cho ra một con số vượt các cận này, điều đó có nghĩa là hoặc việc gán nhãn đã sai, hoặc phân loại trong Bảng 2.10 cần xem lại — nói cách khác, dự đoán này đồng thời đóng vai trò một phép kiểm tra tính nhất quán cho chính quy trình đo.

Một điểm cần nói rõ về phạm vi: tài liệu của dự án InsecureBankv2 liệt kê hơn hai mươi lỗ hổng. Đồ án chỉ đưa vào tập đánh giá mười hai lỗ hổng thoả mãn đồng thời ba điều kiện: tái hiện được một cách xác định trên môi trường lab đã xây dựng, có liên quan trực tiếp tới bảo mật API theo phạm vi đã xác định ở Chương 1, và nằm trong khả năng phát hiện về nguyên tắc của ít nhất một trong bảy công cụ. Những lỗ hổng bị loại gồm các lỗ hổng phụ thuộc thiết bị vật lý, các lỗ hổng chỉ biểu hiện trên phiên bản Android cũ hơn phiên bản dùng trong lab, và các lỗ hổng mang tính giao diện không liên quan tới tầng API. Việc loại bỏ này được ghi lại tường minh để người đọc biết mẫu số mười hai đến từ đâu, thay vì coi đó là toàn bộ lỗ hổng của ứng dụng.

2.3. Xây dựng môi trường thực nghiệm

Yêu cầu đặt ra cho môi trường thực nghiệm gồm ba điểm. Thứ nhất, môi trường phải tái lập được: bất kỳ ai có cùng cấu hình đều phải thu được kết quả tương đương, nếu không thì số liệu benchmark không có giá trị đối chứng. Thứ hai, môi trường phải đưa về được trạng thái sạch trước mỗi lượt chạy, để kết quả của công cụ chạy sau không bị ảnh hưởng bởi dấu vết mà công cụ chạy trước để lại. Thứ ba, môi trường phải

cô lập hoàn toàn với mạng ngoài — vừa để tránh mọi rủi ro pháp lý khi thực hiện các thao tác tấn công, vừa để loại bỏ nhiễu do độ trễ mạng gây ra khi đo tốc độ phân tích.

2.3.1. Cấu hình phần cứng và phần mềm

Bảng 2.11. Cấu hình phần cứng và phần mềm của môi trường thực nghiệm

Thành phần	Cấu hình sử dụng	Ghi chú về ràng buộc
Bộ xử lý	Apple Silicon M-series, kiến trúc ARM64	Quyết định toàn bộ lựa chọn công cụ phía dưới
Hệ điều hành	macOS Ventura trở lên	Máy chủ host duy nhất
Bộ nhớ	16 GB trở lên	Cần cho việc chạy đồng thời máy ảo và các vùng chứa Docker
Ổ cứng	SSD 256 GB trở lên	Lưu ảnh máy ảo và ảnh Docker
Android Studio	Hedgehog 2023.1.1 trở lên	Tạo và quản lý máy ảo
Docker Desktop	Bản dành cho Apple Silicon (ARM64)	Chạy MobSF và OWASP ZAP
Java	JDK 17 trở lên	Yêu cầu của Burp Suite, JADX và apktool
Python	Phiên bản 3.10 trở lên	Chạy máy chủ phía sau, Frida và objection

Ràng buộc quan trọng nhất của cấu hình này là kiến trúc ARM64 của bộ xử lý Apple Silicon, và nó ảnh hưởng tới gần như mọi lựa chọn kỹ thuật phía dưới. Cụ thể, các trình giả lập Android phổ biến dựa trên nền ảo hoá x86 không chạy được nguyên bản trên nền tảng này; chạy qua lớp chuyển dịch kiến trúc thì tốc độ giảm mạnh và làm sai lệch phép đo thời gian. Vì vậy đồ án sử dụng máy ảo do Android Studio tạo ra với ảnh hệ thống arm64-v8a, chạy nguyên bản trên bộ xử lý [22]. Tương tự, ảnh Docker của MobSF và OWASP ZAP phải là bản hỗ trợ đa kiến trúc [23], và tệp nhị phân frida-server phải là bản biên dịch cho ARM64 — dùng nhầm bản x86 sẽ khiến tiến trình không khởi động được và toàn bộ nhánh can thiệp thời gian chạy bị vô hiệu.

Cần nói thêm về lựa chọn chạy toàn bộ trên một máy duy nhất thay vì phân tán trên nhiều máy. Cách làm này có một nhược điểm rõ ràng: các công cụ chia sẻ tài nguyên bộ xử lý và bộ nhớ với nhau, nên phép đo thời gian chịu ảnh hưởng của tải hệ thống tại thời điểm đo. Bù lại, nó loại bỏ hoàn toàn độ trễ mạng giữa các thành phần — vốn là một biến nhiễu khó kiểm soát hơn nhiều và có thể lên tới hàng chục mili giây cho mỗi yêu cầu. Biện pháp giảm thiểu cho nhược điểm còn lại là chạy mỗi lượt đo ba lần và lấy giá trị trung vị, đồng thời không chạy hai công cụ đồng thời trong cùng một lượt đo.

2.3.2. Máy ảo Android và cấu hình mạng

Máy ảo được tạo với cấu hình sau: hồ sơ thiết bị Pixel 6; ảnh hệ thống Android 13 tương ứng mức API 33, kiến trúc arm64-v8a; bộ nhớ 2048 MB; dung lượng lưu trữ trong 2 GB. Một lựa chọn cần giải thích là việc sử dụng ảnh hệ thống có Google APIs nhưng không kèm cửa hàng ứng dụng Google Play. Lý do là ảnh hệ thống kèm Google Play không cho phép nâng quyền bằng lệnh adb root, trong khi quyền này là điều kiện bắt buộc để chạy frida-server. Nói cách khác, chính yêu cầu kỹ thuật của nhánh can thiệp thời gian chạy quyết định lựa chọn ảnh hệ thống, chứ không phải ngược lại.

Việc chọn mức API 33 cũng có lý do. Từ Android 7 trở đi, hệ điều hành không còn tin cậy mặc định các chứng chỉ do người dùng cài đặt đối với lưu lượng của ứng dụng, trừ khi ứng dụng khai báo tường minh trong tệp cấu hình an toàn mạng [6]. InsecureBankv2 có khai báo cho phép, nên công cụ uỷ nhiệm hoạt động được mà không cần sửa đổi tệp APK. Nếu chọn một ứng dụng đích không có khai báo này, toàn bộ nhánh kiểm thử động sẽ phải bổ sung thêm bước đóng gói lại ứng dụng, làm thay đổi bản chất của phép đo. Đây là một ràng buộc kỹ thuật cần được nêu rõ vì nó giới hạn phạm vi kết luận của đồ án.

Về mạng, máy ảo Android không chia sẻ không gian địa chỉ mạng với máy host: địa chỉ localhost bên trong máy ảo trỏ về chính máy ảo chứ không phải máy host. Máy ảo truy cập các dịch vụ đang chạy trên host thông qua địa chỉ quy ước 10.0.2.2. Do đó, ứng dụng đích được cấu hình gọi tới 10.0.2.2 cổng 8888 để tới máy chủ phía sau, và proxy của máy ảo được đặt thủ công về 10.0.2.2 cổng 8080 để lưu lượng đi qua Burp Suite. Để công cụ uỷ nhiệm giải mã được lưu lượng HTTPS, chứng chỉ của công cụ được cài vào kho chứng chỉ tin cậy của hệ thống trên máy ảo. Bảng 2.12 tổng hợp toàn bộ cổng dịch vụ trong môi trường.

Bảng 2.12. Cổng dịch vụ và cách truy cập trong môi trường lab

Dịch vụ	Cổng	Truy cập từ máy host	Truy cập từ máy ảo Android
MobSF	8000	http://localhost:8000	Không cần
Burp Suite proxy	8080	127.0.0.1:8080	10.0.2.2:8080
OWASP ZAP proxy và API	8090	http://localhost:8090	10.0.2.2:8090
Máy chủ InsecureBankv2	8888	http://localhost:8888	10.0.2.2:8888

Dịch vụ	Cổng	Truy cập từ máy host	Truy cập từ máy ảo Android
frida-server	27042	Qua chuyển tiếp cổng của adb	Chạy trên máy ảo
Cầu gỡ lỗi adb	5555	adb connect	Dịch vụ hệ thống

2.3.3. Triển khai từng công cụ

Trình tự triển khai không tuỳ ý mà tuân theo quan hệ phụ thuộc giữa các thành phần. Máy chủ phía sau phải chạy trước để có mục tiêu cho các yêu cầu; máy ảo phải khởi động trước khi cài ứng dụng; frida-server phải được đẩy lên và chạy trước khi objection kết nối. Bảng 2.13 trình bày các bước theo đúng thứ tự thực hiện, kèm cách kiểm tra xác nhận từng bước đã thành công — bước kiểm tra này quan trọng vì nhiều sự cố chỉ biểu hiện ở giai đoạn sau, khi đã khó truy ngược nguyên nhân.

Bảng 2.13. Trình tự triển khai và cách xác nhận từng bước

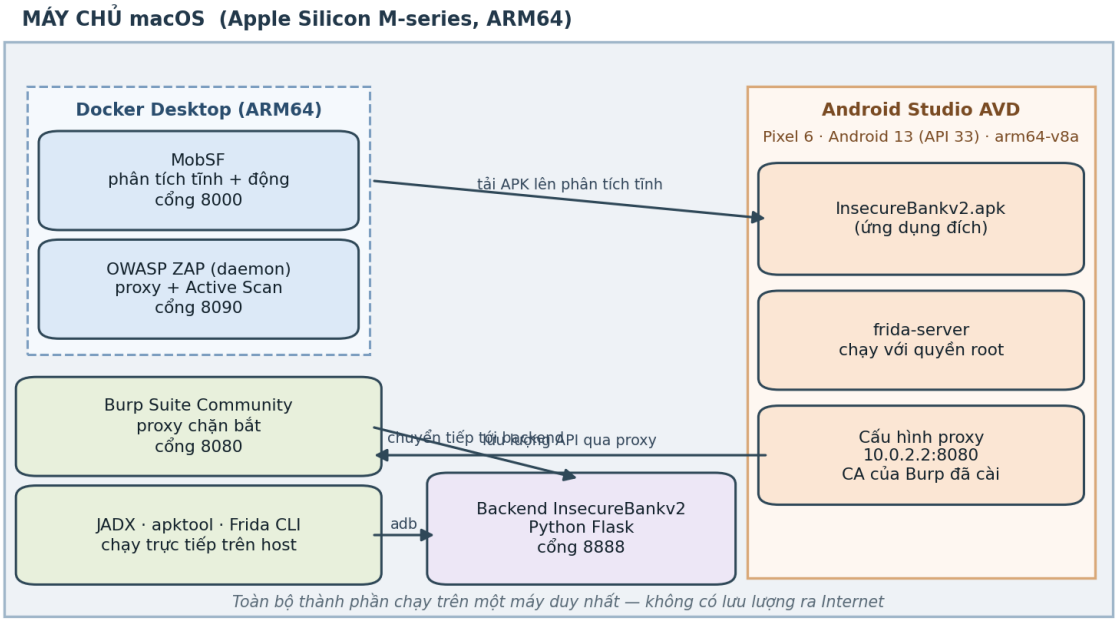
Bước	Thành phần	Thao tác	Cách xác nhận thành công
1	Máy chủ phía sau	Cài các gói phụ thuộc Python, khởi chạy ứng dụng Flask tại cổng 8888	Gọi thử /login bằng curl với tài khoản mẫu, nhận mã 200
2	Máy ảo Android	Tạo máy ảo Pixel 6 với ảnh arm64-v8a, khởi động, nâng quyền bằng adb root	Lệnh adb shell id trả về định danh người dùng 0
3	Ứng dụng đích	Cài tệp InsecureBankv2.apk bằng adb install	Ứng dụng khởi chạy và đăng nhập được với tài khoản mẫu
4	MobSF	Chạy vùng chứa Docker, ánh xạ cổng 8000, đặt khoá API	Truy cập giao diện web và gọi thử điểm cuối tải tệp lên
5	OWASP ZAP	Chạy vùng chứa ở chế độ nền, ánh xạ cổng 8090, bật giao diện API	Gọi /JSON/core/view/version nhận về số phiên bản
6	Burp Suite	Khởi động, đặt cổng lắng nghe 8080, xuất chứng chỉ CA	Giao diện Proxy hiển thị trạng thái đang lắng nghe
7	Chứng chỉ CA	Đẩy chứng chỉ lên máy ảo và cài vào kho chứng chỉ hệ thống	Mở một trang HTTPS trên máy ảo và thấy lưu lượng trong Burp
8	frida-server	Đẩy tệp nhị phân ARM64 vào thư mục tạm, cấp quyền thực thi, chạy nền	Lệnh frida-ps -U liệt kê được tiến trình trên máy ảo
9	objection	Cài qua pip trong môi trường ảo Python	Lệnh objection --gadget <gói> explore mở được phiên làm việc
10	JADX và apktool	Giải nén vào thư mục công cụ, thêm vào biến môi trường đường dẫn	Chạy thử trên tệp APK và thu được thư mục mã nguồn

Trong mười bước trên, bước 7 là bước hay thất bại nhất và cũng là bước có ảnh hưởng lan rộng nhất nếu thất bại. Từ Android 7, chứng chỉ do người dùng cài đặt được lưu vào một kho riêng và không được ứng dụng tin cậy theo mặc định; muốn chứng chỉ có hiệu lực ở mức hệ thống thì phải sao chép nó vào thư mục chứng chỉ hệ thống với tên tệp là giá trị băm của chủ thể chứng chỉ, và thao tác này cần quyền quản trị

cùng việc gắn lại phân vùng hệ thống ở chế độ ghi. Nếu bước này chỉ thành công một nửa — chứng chỉ được cài nhưng ở kho người dùng — thì triệu chứng quan sát được là ứng dụng vẫn chạy bình thường nhưng Burp Suite không thấy bất kỳ lưu lượng HTTPS nào, rất dễ bị chẩn đoán nhầm thành lỗi cấu hình proxy.

2.3.4. Kiến trúc tổng thể môi trường lab

Hình 2.5 thể hiện cách toàn bộ các thành phần nêu trên được bố trí trên một máy duy nhất và cách chúng kết nối với nhau.



Hình 2.5. Kiến trúc tổng thể môi trường thực nghiệm

Trên Hình 2.5, các thành phần được chia thành ba nhóm theo cách triển khai. Nhóm chạy trong Docker gồm MobSF ở cổng 8000 và OWASP ZAP ở chế độ nền tại cổng 8090; việc đóng gói bằng Docker giúp hai công cụ này có môi trường phụ thuộc cố định và có thể đưa về trạng thái ban đầu chỉ bằng việc tạo lại vùng chứa. Nhóm chạy trực tiếp trên máy host gồm Burp Suite tại cổng 8080, các công cụ dịch ngược JADX và apktool, giao diện dòng lệnh của Frida, và máy chủ phía sau của ứng dụng đích tại cổng 8888. Nhóm thứ ba là máy ảo Android, nơi cài ứng dụng đích, chạy frida-server và đặt cấu hình proxy.

Có ba luồng dữ liệu chính trong sơ đồ. Luồng thứ nhất là luồng phân tích tĩnh: tệp APK được nạp trực tiếp vào MobSF, JADX hoặc apktool mà không cần máy ảo

hoạt động. Luồng thứ hai là luồng kiểm thử động qua uỷ nhiệm: yêu cầu API từ ứng dụng trên máy ảo đi qua Burp Suite hoặc OWASP ZAP rồi mới tới máy chủ phía sau. Luồng thứ ba là luồng can thiệp thời gian chạy: Frida và objection kết nối tới tiến trình của ứng dụng trên máy ảo thông qua cầu gỡ lỗi Android. Điểm chung của cả ba luồng là không có luồng nào đi ra khỏi máy chủ host, thoả mãn yêu cầu cô lập đã đặt ra ở đầu mục này.

2.3.5. Quy trình đưa môi trường về trạng thái sạch

Để bảo đảm các lượt chạy độc lập với nhau, trước mỗi lượt môi trường được đưa về trạng thái sạch theo một trình tự cố định gồm năm bước. Bước một, gỡ cài đặt ứng dụng đích khỏi máy ảo rồi cài lại từ tệp APK gốc, nhằm xoá toàn bộ dữ liệu ứng dụng đã lưu ở lượt trước. Bước hai, khởi động lại máy chủ phía sau và nạp lại cơ sở dữ liệu mẫu ban đầu, để dữ liệu tài khoản luôn ở cùng một trạng thái xuất phát. Bước ba, xoá lịch sử phiên của công cụ uỷ nhiệm đang dùng. Bước bốn, xoá nhật ký hệ thống của máy ảo. Bước năm, xác nhận môi trường hoạt động bình thường bằng một thao tác đăng nhập chuẩn với tài khoản mẫu; nếu thao tác này thất bại thì lượt chạy bị huỷ và môi trường được thiết lập lại từ đầu.

Bước hai xứng đáng được nhấn mạnh. Nhiều kịch bản kiểm thử ở mục 2.4 làm thay đổi dữ liệu của máy chủ — chẳng hạn ca kiểm thử phân quyền thực hiện một giao dịch chuyển tiền thật trong cơ sở dữ liệu mẫu. Nếu không nạp lại dữ liệu ban đầu, số dư tài khoản ở lượt chạy sau sẽ khác lượt trước, và trong trường hợp xấu nhất giao dịch có thể thất bại vì không đủ số dư, dẫn tới kết luận sai rằng lỗ hổng không tồn tại. Đây là một ví dụ cho thấy tại sao việc tái lập trạng thái không phải là thủ tục hình thức mà là điều kiện đúng đắn của phép đo.

Bên cạnh đó, phiên bản cụ thể của từng công cụ được ghi nhận lại tại thời điểm chạy và được giữ cố định trong suốt quá trình thực nghiệm. Đây là một chi tiết dễ bị bỏ qua nhưng có ảnh hưởng thực chất: bộ luật quét của MobSF cũng như tập quy tắc của OWASP ZAP được cập nhật thường xuyên, nên hai lượt chạy cách nhau vài tuần với phiên bản khác nhau có thể cho kết quả khác nhau mà nguyên nhân không nằm ở bản thân phép đo.

2.3.6. Các sự cố thường gặp và cách xử lý

Quá trình thiết lập môi trường đã gặp một số sự cố kỹ thuật lặp lại. Việc ghi lại chúng có hai mục đích: giúp người đọc tái lập môi trường nhanh hơn, và quan trọng hơn, làm rõ những trường hợp mà triệu chứng bên ngoài dễ bị hiểu nhầm thành kết quả của phép đo. Nếu một công cụ không phát hiện được lỗ hổng vì môi trường cấu hình sai chứ không phải vì năng lực của nó, mà sự cố đó không được nhận ra, thì số liệu benchmark sẽ sai một cách âm thầm.

Bảng 2.14. Các sự cố thường gặp khi thiết lập môi trường và cách xử lý

Sự cố	Triệu chứng quan sát được	Nguyên nhân	Cách xử lý
frida-server không chạy	Lệnh frida-ps -U báo không tìm thấy thiết bị	Tệp nhị phân sai kiến trúc hoặc chưa có quyền quản trị	Tải lại bản ARM64 đúng phiên bản; chạy adb root trước khi khởi động
Không thấy lưu lượng HTTPS	Ứng dụng hoạt động bình thường nhưng công cụ uỷ nhiệm không ghi được yêu cầu nào	Chứng chỉ CA nằm ở kho người dùng chứ không phải kho hệ thống	Sao chép chứng chỉ vào thư mục chứng chỉ hệ thống với tên là giá trị băm chủ thể
Ứng dụng không kết nối được máy chủ	Màn hình đăng nhập báo lỗi mạng	Ứng dụng trỏ tới localhost thay vì 10.0.2.2	Sửa cấu hình địa chỉ máy chủ trong ứng dụng về 10.0.2.2
Vùng chứa Docker thoát ngay	Vùng chứa khởi động rồi dừng sau vài giây	Ảnh Docker không hỗ trợ ARM64	Dùng ảnh đa kiến trúc chính thức; tránh ảnh chỉ có bản x86
Quét động của MobSF treo	Tiến trình quét dừng ở bước cài đặt ứng dụng	MobSF không tìm thấy máy ảo hoặc máy ảo chưa nâng quyền	Khai báo địa chỉ máy ảo trong cấu hình MobSF; xác nhận adb root thành công
Kết quả khác nhau giữa các lượt	Cùng công cụ, cùng ứng dụng nhưng số cảnh báo chênh lệch	Môi trường chưa được đưa về trạng thái sạch giữa hai lượt	Thực hiện đủ năm bước ở mục 2.3.5 trước mỗi lượt chạy

Trong sáu sự cố trên, hai sự cố cuối là nguy hiểm nhất đối với chất lượng số liệu, bởi chúng không làm hỏng quy trình một cách rõ ràng mà chỉ làm sai lệch kết quả. Một lượt quét động bị treo có thể bị ghi nhận thành công với kết quả rỗng nếu người thực hiện không kiểm tra; một môi trường chưa sạch cho ra số liệu khác nhau giữa các lượt mà không có dấu hiệu bất thường nào. Đây chính là lý do quy trình benchmark ở mục 2.5 đặt bước xác nhận môi trường thành một giai đoạn riêng có tiêu chí thành công tường minh, thay vì coi đó là thao tác chuẩn bị thông thường.

2.4. Thiết kế các kịch bản kiểm thử

Ma trận kiểm thử ở mục 2.5 xác định công cụ nào được dùng cho lỗ hổng nào, nhưng bản thân ma trận chưa đủ để một người khác thực hiện lại thực nghiệm. Điều

còn thiếu là mô tả cụ thể từng thao tác: gửi yêu cầu gì, sửa tham số nào, quan sát dấu hiệu nào để kết luận. Mục này bù đắp khoảng trống đó bằng mười hai kịch bản kiểm thử, mỗi kịch bản tương ứng với một lỗ hổng trong đáp án và được đánh mã từ TC-01 đến TC-12 theo đúng thứ tự các lỗ hổng V01 đến V12.

Mỗi kịch bản được mô tả theo một khuôn cố định gồm sáu phần: công cụ sử dụng, mục tiêu, tiền điều kiện, các bước thực hiện, kết quả kỳ vọng, và quy tắc gán nhãn. Phần quy tắc gán nhãn là phần thường bị bỏ qua trong các tài liệu kiểm thử nhưng lại quan trọng nhất đối với một nghiên cứu benchmark: nó xác định trước, ngay từ khâu thiết kế, kết quả nào sẽ được tính là phát hiện đúng và kết quả nào bị tính là báo động giả hoặc bỏ sót, nhờ đó việc chấm điểm ở chương sau không còn chỗ cho suy diễn chủ quan.

Trước khi đi vào từng kịch bản, cần nói về đường cơ sở. Với mọi kịch bản kiểm thử động, bước đầu tiên luôn là thực hiện một thao tác hợp lệ và ghi lại yêu cầu cùng phản hồi tương ứng — đây là đường cơ sở. Việc phát hiện lỗ hổng sau đó luôn là việc so sánh hành vi của máy chủ khi nhận một yêu cầu đã sửa đổi với hành vi ở đường cơ sở. Không có đường cơ sở thì không có cách nào khẳng định một phản hồi bất thường là do lỗ hổng chứ không phải do lỗi ngẫu nhiên.

TC-01. Phát hiện thông tin xác thực nhúng trong mã

Lỗ hổng: V01 (M1)

Công cụ: MobSF (static), kiểm chứng bằng JADX

Mục tiêu: Xác định xem khoá bí mật dùng để mã hoá có được viết cố định trong mã nguồn hay không.

Tiền điều kiện: Có tệp InsecureBankv2.apk; MobSF đang chạy.

Các bước thực hiện:

1. Tải tệp APK lên MobSF qua điểm cuối upload.
2. Kích hoạt quét tĩnh và tải kết quả JSON.
3. Lọc các phát hiện thuộc nhóm hardcoded secret.
4. Mở lớp tương ứng trong JADX để xác nhận giá trị khoá là hằng số thật, không phải dữ liệu mẫu.

Kết quả kỳ vọng: MobSF liệt kê ít nhất một khoá bí mật nhúng trong mã; JADX xác nhận đó là hằng số dùng cho mã hoá.

Quy tắc gán nhãn: TP nếu khoá được xác nhận là thật; FP nếu chuỗi bị báo thực chất là dữ liệu mẫu.

TC-02. Phát hiện lưu lượng HTTP không mã hoá

Lỗ hổng: V02 (M5)

Công cụ: MobSF (static) và Burp Suite / OWASP ZAP

Mục tiêu: Xác định xem có điểm cuối nào được gọi qua HTTP thay vì HTTPS.

Tiền điều kiện: Ứng dụng đã cài trên máy ảo; công cụ uỷ nhiệm đang chạy; đã đăng nhập được.

Các bước thực hiện:

1. Ở nhánh tĩnh: kiểm tra cấu hình an toàn mạng và các chuỗi địa chỉ trong kết quả MobSF.
2. Ở nhánh động: thao tác đầy đủ chức năng ứng dụng và quan sát lược đồ của từng yêu cầu trong lịch sử lưu lượng.
3. Đối chiếu danh sách điểm cuối dùng HTTP thu được từ hai nhánh.

Kết quả kỳ vọng: Ít nhất một yêu cầu được ghi nhận đi qua HTTP; hai nhánh cho kết quả nhất quán.

Quy tắc gán nhãn: TP nếu quan sát được yêu cầu HTTP thật; FN nếu công cụ bỏ sót một điểm cuối HTTP đã biết.

TC-03. Kiểm tra thiếu Certificate Pinning

Lỗ hổng: V03 (M5)

Công cụ: Burp Suite làm chính, Frida/objection hỗ trợ

Mục tiêu: Xác định xem ứng dụng có ghim chứng chỉ hay không, tức có chống được tấn công xen giữa hay không.

Tiền điều kiện: Công cụ uỷ nhiệm đang chạy; chứng chỉ CA đã cài ở kho hệ thống.

Các bước thực hiện:

1. Cấu hình proxy và quan sát xem lưu lượng HTTPS có hiển thị ở dạng rõ trong công cụ uỷ nhiệm hay không.
2. Nếu lưu lượng hiển thị ngay: kết luận ứng dụng không ghim chứng chỉ.
3. Nếu lưu lượng bị chặn: chạy objection với lệnh vô hiệu hoá ghim chứng chỉ rồi thử lại.

Kết quả kỳ vọng: Lưu lượng HTTPS hiển thị được ở dạng rõ (trực tiếp hoặc sau khi vô hiệu hoá pinning).

Quy tắc gán nhãn: TP nếu quan sát được nội dung phiên; kết quả này khẳng định ứng dụng thiếu pinning.

TC-04. Kiểm tra lưu trữ dữ liệu không an toàn

Lỗ hổng: V04 (M9)

Công cụ: objection làm chính, MobSF (dynamic) đối chứng

Mục tiêu: Xác định xem tên đăng nhập và mật khẩu có bị lưu trên thiết bị ở dạng đọc được hoặc giải mã ngược được hay không.

Tiền điều kiện: Đã đăng nhập thành công một lần; frida-server đang chạy.

Các bước thực hiện:

1. Dùng objection liệt kê nội dung thư mục dữ liệu riêng của ứng dụng.
2. Mở tệp SharedPreferences và kiểm tra các cặp khoá–giá trị.
3. Đối chiếu với kết quả theo dõi thao tác lưu trữ của MobSF ở chế độ động.

Kết quả kỳ vọng: Tìm thấy tên đăng nhập (Base64) và mật khẩu (mã hoá AES bằng khoá viết cứng) trong SharedPreferences, khôi phục được dạng rõ.

Quy tắc gán nhãn: TP nếu đọc được hoặc giải mã ngược được thông tin xác thực; FN nếu công cụ không truy cập được vùng lưu trữ.

TC-05. Phát hiện thuật toán mật mã yếu

Lỗ hổng: V05 (M10)

Công cụ: MobSF (static) và Frida

Mục tiêu: Xác định xem ứng dụng có dùng mật mã sai cách (thuật toán lỗi thời, khoá viết cứng hoặc IV tĩnh) để bảo vệ mật khẩu hay không.

Tiền điều kiện: Có tệp APK; frida-server đang chạy.

Các bước thực hiện:

1. Ở nhánh tĩnh: MobSF liệt kê các lời gọi thuật toán mật mã và cảnh báo với thuật toán yếu.
2. Ở nhánh động: dùng Frida móc vào hàm mật mã, thao tác đăng nhập và quan sát tham số khởi tạo thuật toán được truyền vào.
3. Xác nhận thuật toán, chế độ mã hoá, nguồn gốc khoá và giá trị IV.

Kết quả kỳ vọng: Xác định được ứng dụng dùng AES/CBC với khoá viết cứng và IV tĩnh toàn số 0 để mã hoá mật khẩu.

Quy tắc gán nhãn: TP nếu chỉ ra đúng điểm yếu cài đặt mật mã (khoá viết cứng hoặc IV tĩnh); FP nếu cảnh báo không chỉ ra được điểm yếu cụ thể.

TC-06. Kiểm tra thành phần exported không bảo vệ

Lỗ hổng: V06 (M8)

Công cụ: MobSF (static), kiểm chứng bằng apktool

Mục tiêu: Xác định xem có thành phần nào được xuất ra ngoài mà không đặt quyền bảo vệ hay không.

Tiền điều kiện: Có tệp APK.

Các bước thực hiện:

1. MobSF liệt kê các thành phần có thuộc tính exported bằng true.
2. Dùng apktool khôi phục tệp kê khai để xác nhận thuộc tính được khai báo tường minh, không phải giá trị mặc định.
3. Xác định thành phần nào không kèm khai báo quyền bảo vệ.

Kết quả kỳ vọng: Tìm thấy ít nhất một Activity exported không được bảo vệ bằng quyền.

Quy tắc gán nhãn: TP nếu apktool xác nhận khai báo tường minh; FP nếu thuộc tính chỉ nhận giá trị mặc định.

TC-07. Phát hiện cửa hậu dành cho nhà phát triển

Lỗ hổng: V07 (API9)

Công cụ: Burp Suite / OWASP ZAP làm chính, JADX hỗ trợ

Mục tiêu: Xác định xem có điểm cuối ẩn nào cho phép bỏ qua quy trình xác thực thông thường hay không.

Tiền điều kiện: Đã lập bản đồ điểm cuối bằng JADX; công cụ uỷ nhiệm đang chạy.

Các bước thực hiện:

1. Dùng JADX tìm các chuỗi địa chỉ trong mã, phát hiện điểm cuối /devlogin không xuất hiện trên giao diện.
2. Dùng Repeater gửi yêu cầu tới /devlogin với tham số tối thiểu.
3. Quan sát phản hồi xem có cấp quyền truy cập mà không cần mật khẩu hợp lệ hay không.

Kết quả kỳ vọng: Điểm cuối /devlogin trả về trạng thái đăng nhập thành công mà không cần xác thực đầy đủ.

Quy tắc gán nhãn: TP nếu truy cập được qua cửa hậu; FN nếu công cụ không phát hiện điểm cuối ẩn này.

TC-08. Kiểm tra phân quyền mức đối tượng

Lỗ hổng: V08 (API1)

Công cụ: Burp Suite (Repeater)

Mục tiêu: Xác định xem một người dùng có thể truy cập hoặc thao tác trên dữ liệu của người dùng khác bằng cách sửa tham số định danh hay không.

Tiền điều kiện: Có sẵn hai tài khoản A và B; đã đăng nhập tài khoản A và bắt được yêu cầu chuyển khoản.

Các bước thực hiện:

1. Đăng nhập tài khoản A, thực hiện một giao dịch hợp lệ làm đường cơ sở.
2. Gửi yêu cầu sang Repeater, sửa tham số tài khoản nguồn sang tài khoản B, giữ nguyên thông tin đăng nhập của A.
3. Gửi lại và quan sát phản hồi của máy chủ.

Kết quả kỳ vọng: Máy chủ chấp nhận và thực hiện giao dịch trên tài khoản của B, xác nhận lỗ hổng phân quyền.

Quy tắc gán nhãn: TP nếu giao dịch trái phép thành công; TN nếu máy chủ trả về 401 hoặc 403.

TC-09. Kiểm tra cơ chế xác thực phía máy chủ

Lỗ hổng: V09 (M3, API2)

Công cụ: Burp Suite / OWASP ZAP, Frida hỗ trợ

Mục tiêu: Xác định các điểm yếu trong cách máy chủ xác thực người gọi ở từng điểm cuối.

Tiền điều kiện: Có tài khoản A hợp lệ và biết tên người dùng của tài khoản B; Burp Suite đã bắt được lưu lượng của ứng dụng.

Các bước thực hiện:

1. Quan sát các yêu cầu đã bắt được để xác định cơ chế xác thực; gửi /login với tên không tồn tại và với mật khẩu sai, so sánh hai phản hồi.
2. Gửi yêu cầu /changepassword chỉ với tên người dùng B và mật khẩu mới, không kèm mật khẩu hiện tại, để kiểm tra máy chủ có xác thực người gọi hay không.
3. Dùng Frida quan sát cách ứng dụng lưu và gửi thông tin đăng nhập ở mỗi yêu cầu.

Kết quả kỳ vọng: Phát hiện ít nhất một điểm yếu: không có cơ chế phiên (thông tin đăng nhập gửi lại ở mọi yêu cầu), đổi được mật khẩu của người khác mà không cần xác thực, hoặc phản hồi đăng nhập cho phép dò tên người dùng.

Quy tắc gán nhãn: TP cho mỗi điểm yếu xác nhận được; các điểm yếu trùng nguyên nhân gốc được gộp làm một.

TC-10. Kiểm tra rò rỉ dữ liệu qua nhật ký

Lỗ hổng: V10 (M9)

Công cụ: objection / Frida làm chính, MobSF (dynamic) đối chứng

Mục tiêu: Xác định xem thông tin nhạy cảm có bị ghi vào nhật ký hệ thống hay không.

Tiền điều kiện: Ứng dụng đang chạy; có quyền đọc nhật ký hệ thống.

Các bước thực hiện:

1. Bật thu nhật ký hệ thống của máy ảo.
2. Thao tác đăng nhập và chuyển khoản đầy đủ.
3. Lọc nhật ký tìm các chuỗi chứa tên đăng nhập, mật khẩu hoặc số tài khoản.

Kết quả kỳ vọng: Tìm thấy ít nhất một mục nhật ký chứa thông tin nhạy cảm ở dạng rõ.

Quy tắc gán nhãn: TP nếu quan sát được mục nhật ký nhạy cảm; FN nếu không phát hiện dù lỗ hổng tồn tại.

TC-11. Kiểm tra cho phép sao lưu dữ liệu

Lỗ hổng: V11 (M8)

Công cụ: MobSF (static), kiểm chứng bằng apktool

Mục tiêu: Xác định xem ứng dụng có cho phép sao lưu dữ liệu qua công cụ hệ thống hay không.

Tiền điều kiện: Có tệp APK.

Các bước thực hiện:

1. MobSF kiểm tra thuộc tính allowBackup trong tệp kê khai.
2. Dùng apktool khôi phục tệp kê khai để xác nhận thuộc tính được đặt tường minh bằng true.
3. Ghi nhận hệ quả: dữ liệu ứng dụng có thể trích xuất qua cơ chế sao lưu.

Kết quả kỳ vọng: Xác nhận thuộc tính allowBackup được đặt bằng true.

Quy tắc gán nhãn: TP nếu apktool xác nhận khai báo tường minh; FP nếu thuộc tính chỉ nhận giá trị mặc định.

TC-12. Kiểm tra chính sách mật khẩu và giới hạn tần suất

Lỗ hổng: V12 (M3)

Công cụ: Burp Suite (Intruder) / OWASP ZAP

Mục tiêu: Xác định xem máy chủ có kiểm tra độ phức tạp mật khẩu và giới hạn số lần đăng nhập hay không.

Tiền điều kiện: Có tài khoản hợp lệ; công cụ uỷ nhiệm đang chạy.

Các bước thực hiện:

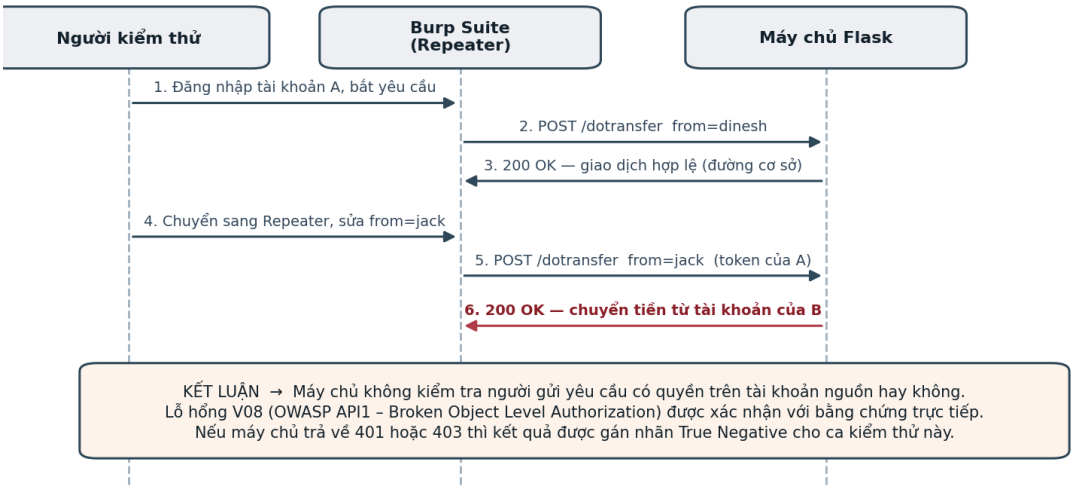
1. Dùng Intruder gửi nhiều yêu cầu đăng nhập liên tiếp với mật khẩu sai.
2. Quan sát máy chủ có khoá tài khoản hoặc làm chậm phản hồi sau một số lần thất bại hay không.
3. Thử đổi mật khẩu sang một chuỗi quá đơn giản để kiểm tra chính sách độ phức tạp.

Kết quả kỳ vọng: Máy chủ chấp nhận số lần thử không giới hạn và chấp nhận mật khẩu đơn giản.

Quy tắc gán nhãn: TP cho mỗi điểm yếu; đo thêm thời gian phản hồi để phục vụ tiêu chí tốc độ.

2.4.1. Minh hoạ chi tiết: kịch bản TC-08

Trong mười hai kịch bản, TC-08 kiểm tra lỗ hổng phân quyền mức đối tượng là kịch bản đáng phân tích kỹ nhất, vì hai lý do. Thứ nhất, đây là loại lỗ hổng đứng đầu OWASP API Security Top 10 và cũng là loại mà không công cụ tự động nào phát hiện được, nên cách kiểm thử nó thể hiện rõ giá trị của thao tác thủ công có phương pháp. Thứ hai, nó minh hoạ trọn vẹn vai trò của đường cơ sở trong việc phân biệt phát hiện đúng với âm tính thật. Hình 2.6 mô tả chi tiết trình tự thực hiện kịch bản này.



Hình 2.6. Trình tự thực hiện kịch bản kiểm thử TC-08 (phân quyền mức đối tượng)

Trình tự trên Hình 2.6 chia làm hai nửa. Nửa đầu — các bước 1 tới 3 — thiết lập đường cơ sở: người kiểm thử đăng nhập bằng tài khoản A, thực hiện một giao dịch

hợp lệ từ chính tài khoản của mình, và xác nhận máy chủ trả về thành công. Nửa sau — các bước 4 tới 6 — là phép thử thật: cùng thông tin đăng nhập của A, nhưng tham số tài khoản nguồn được sửa thành tài khoản B. Điểm mấu chốt nằm ở bước 6: nếu máy chủ trả về thành công, có nghĩa nó đã cho phép A thao tác trên tài khoản của B chỉ dựa vào tham số do A cung cấp, và lỗ hổng được xác nhận với bằng chứng không thể chối cãi.

Cách thiết kế này cũng định nghĩa rõ trường hợp âm tính thật. Nếu ở bước 6 máy chủ trả về mã lỗi từ chối truy cập, điều đó không phải là công cụ thất bại mà là ứng dụng an toàn ở điểm này; kết quả được gán nhãn âm tính thật. Sự phân biệt giữa âm tính thật — ứng dụng an toàn — và bỏ sót — ứng dụng có lỗ hổng nhưng công cụ không thấy — là điều mà chỉ một kịch bản có đường cơ sở rõ ràng mới bảo đảm được, và đó là lý do mọi kịch bản động trong mục này đều bắt đầu bằng việc dựng đường cơ sở.

Mười hai kịch bản trên phủ đúng mười hai lỗ hổng của đáp án chuẩn. Chúng không cố gắng phủ mọi công cụ cho mọi lỗ hổng — việc phân công công cụ nào chạy kịch bản nào được quyết định bởi ma trận kiểm thử ở mục tiếp theo. Quan hệ giữa hai mục là: mục 2.4 định nghĩa cách kiểm tra từng lỗ hổng, còn mục 2.5 định nghĩa công cụ nào được đánh giá trên kịch bản nào và kết quả được tổng hợp thành điểm số ra sao.

2.5. Khung đánh giá và quy trình benchmark

Mục này trình bày bốn thành phần của khung đánh giá: ma trận kiểm thử xác định công cụ nào được dùng để kiểm tra lỗ hổng nào, quy ước gán nhãn kết quả, bảy tiêu chí đo lường, và cách tổng hợp kết quả thành điểm số cùng quy trình tự động hoá. Nguyên tắc xuyên suốt là toàn bộ khung được cố định trước khi chạy công cụ đầu tiên, nhằm tránh việc điều chỉnh tiêu chí cho phù hợp với kết quả đã quan sát được.

2.5.1. Ma trận kiểm thử

Ma trận kiểm thử được xây dựng bằng cách giao hai chiều thông tin: danh sách mười hai lỗ hổng đã biết ở Bảng 2.10 và ranh giới năng lực của bảy công cụ đã phân tích ở mục 2.1. Mỗi ô của ma trận nhận một trong bốn giá trị. Giá trị “Chính” nghĩa là công cụ được kỳ vọng là phương tiện phát hiện chính đối với lỗ hổng đó. Giá trị

“Kiểm chứng” nghĩa là công cụ không dùng để phát hiện mà dùng để xác nhận lại kết quả của công cụ khác. Giá trị “Hỗ trợ” nghĩa là công cụ không phát hiện lỗ hổng mà tạo điều kiện cho công cụ khác phát hiện được — trường hợp điển hình là việc vô hiệu hoá ghim chứng chỉ. Ô để trống nghĩa là công cụ về nguyên tắc không thể phát hiện lỗ hổng đó.

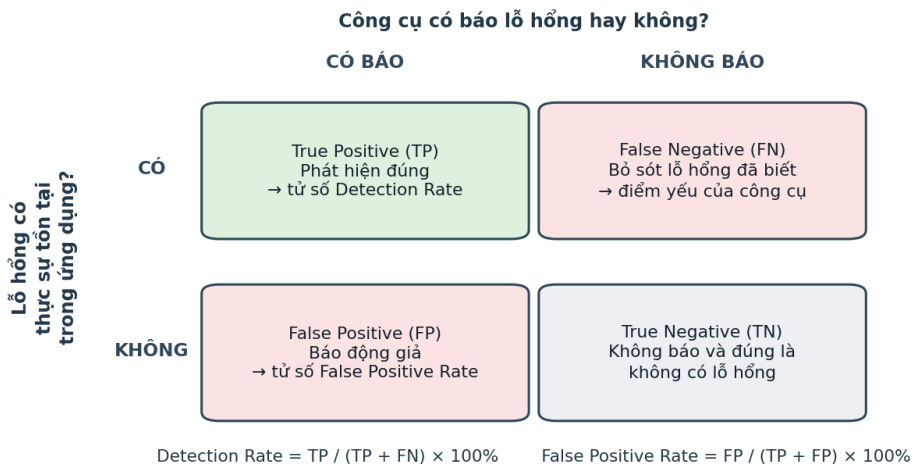
Bảng 2.15. Ma trận kiểm thử: công cụ × lỗ hổng

Lỗ hổng	MobSF	Burp	ZAP	Frida	object.	JADX	apktool
V01 – Thông tin xác thực nhúng	Chính					K.chứng	
V02 – HTTP không mã hoá	Chính	Chính	Chính				
V03 – Không có pinning		Chính	K.chứng	Hỗ trợ	Hỗ trợ		
V04 – Lưu trữ không an toàn	Chính			K.chứng	Chính		
V05 – Mật mã yếu	Chính			K.chứng		K.chứng	
V06 – Thành phần exported	Chính						K.chứng
V07 – Cửa hậu nhà phát triển	Một phần	Chính	Chính			K.chứng	
V08 – Phân quyền mức đối tượng		Chính	Một phần				
V09 – Xác thực bị phá vỡ	Một phần	Chính	Chính	K.chứng			
V10 – Rò rỉ qua nhật ký	Chính				K.chứng		
V11 – Cho phép sao lưu	Chính						K.chứng
V12 – Chính sách mật khẩu yếu	Một phần	Chính	Chính				

Ma trận này có hai công dụng. Công dụng thứ nhất là xác định phạm vi công việc: mỗi ô khác rỗng tương ứng với một trường hợp kiểm thử cụ thể cần được chuẩn bị kịch bản và thực hiện, tổng cộng có ba mươi tư trường hợp. Công dụng thứ hai, quan trọng hơn, là xác định mẫu số khi tính chỉ số phát hiện. Nếu lấy mẫu số là toàn bộ mười hai lỗ hổng cho mọi công cụ thì kết quả sẽ bị bóp méo một cách có hệ thống: JADX chẳng hạn sẽ có chỉ số rất thấp, nhưng phần lớn nguyên nhân là vì nó không phải công cụ quét chứ không phải vì nó kém. Vì vậy đồ án tính chỉ số phát hiện trên mẫu số là số lỗ hổng mà công cụ được kỳ vọng phát hiện theo ma trận, đồng thời báo cáo song song cả tỷ lệ trên tổng mười hai lỗ hổng để người đọc có cả hai góc nhìn.

2.5.2. Quy ước gán nhãn kết quả

Mọi cảnh báo do công cụ đưa ra và mọi lỗ hổng trong danh sách đáp án được gán nhãn theo bốn khả năng, tổng hợp trên Hình 2.7.



Hình 2.7. Quy ước phân loại kết quả TP, FP, FN, TN

Trong thực tế áp dụng, quy ước trên cần ba điều kiện bổ sung để tránh tranh cãi khi chấm. Điều kiện thứ nhất là về mức độ khớp: một cảnh báo chỉ được tính là phát hiện đúng khi nó chỉ đúng vị trí và đúng bản chất của lỗ hổng trong danh sách đáp án. Một cảnh báo chung chung kiểu "ứng dụng có vấn đề về mật mã" mà không chỉ ra được thuật toán nào ở đâu sẽ không được tính, bởi nếu chấp nhận thì tiêu chí trở nên co giãn theo cảm tính của người chấm. Điều kiện thứ hai là về trùng lặp: nhiều công cụ đưa ra hàng chục cảnh báo cho cùng một nguyên nhân gốc; các cảnh báo trùng nhau được gộp thành một trước khi đếm, nếu không thì công cụ càng ồn ào lại càng có lợi thế. Điều kiện thứ ba là về những cảnh báo nằm ngoài danh sách đáp án: một cảnh báo về vấn đề có thật nhưng không nằm trong mười hai lỗ hổng của đáp án sẽ không tính là báo động giả mà được ghi nhận riêng, vì đó là phát hiện có giá trị chứ không phải sai sót của công cụ.

2.5.3. Bảy tiêu chí đánh giá benchmark

Bảy tiêu chí đánh giá được xây dựng dựa trên khung đánh giá công cụ phân tích bảo mật Android của Ranganath và Mitra [19], điều chỉnh cho phù hợp với bối cảnh kiểm thử API và bổ sung hai tiêu chí phản ánh điều kiện triển khai thực tế là mức độ dễ sử dụng và chi phí. Bảng 2.16 trình bày định nghĩa, cách đo và thang đo của từng tiêu chí.

Bảng 2.16. Bảy tiêu chí đánh giá benchmark và cách đo

Tiêu chí	Định nghĩa	Cách đo	Thang đo
Tỷ lệ phát hiện	Khả năng tìm ra các lỗ hổng đã biết	TP chia cho tổng số lỗ hổng công cụ được kỳ vọng phát hiện theo ma trận	0 – 100%
Tỷ lệ báo động giả	Mức độ nhiễu trong kết quả đầu ra	FP chia cho tổng số cảnh báo sau khi đã gộp trùng lặp	0 – 100%, thấp hơn là tốt
Mức độ dễ sử dụng	Công sức cài đặt, cấu hình và vận hành	Chấm theo thang Likert dựa trên số bước cấu hình và yêu cầu kỹ năng	1 – 5
Tốc độ phân tích	Thời gian hoàn thành một lượt kiểm tra	Đo từ lúc khởi động lượt chạy tới khi có kết quả, lấy trung vị của ba lần	Tính bằng phút
Độ phủ	Số loại lỗ hổng OWASP công cụ chạm tới được	Đếm số lỗ hổng khác nhau trong đáp án mà công cụ phát hiện đúng ít nhất một lần	0 – 12
Chất lượng báo cáo	Mức độ rõ ràng và khả năng hành động của kết quả	Chấm theo thang Likert dựa trên bốn yếu tố: vị trí, mức nguy hiểm, giải thích, khuyến nghị khắc phục	1 – 5
Chi phí	Điều kiện tài chính để sử dụng công cụ	Phân loại theo giấy phép và giới hạn của bản miễn phí	Miễn phí / hạn chế / trả phí

Hai tiêu chí chấm theo thang Likert là mức độ dễ sử dụng và chất lượng báo cáo mang tính chủ quan, và đây là điểm yếu cần thừa nhận thẳng thắn của khung đánh giá. Để giảm tính tuỳ tiện, mỗi mức điểm được mô tả trước bằng tiêu chí quan sát được thay vì để người chấm tự cảm nhận. Bảng 2.17 trình bày thang neo cho tiêu chí mức độ dễ sử dụng làm ví dụ; tiêu chí chất lượng báo cáo được neo theo cách tương tự.

Bảng 2.17. Thang neo điểm cho tiêu chí mức độ dễ sử dụng

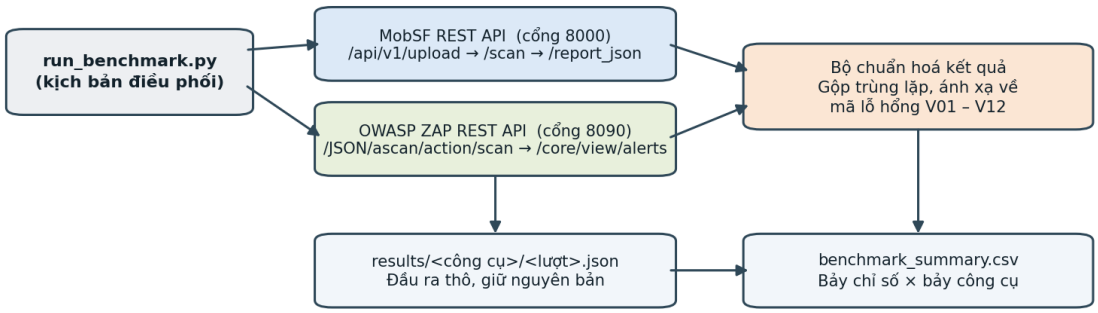
Điểm	Mô tả quan sát được	Ví dụ công cụ dự kiến
5	Cài đặt bằng một lệnh hoặc một vùng chứa, chạy được ngay không cần cấu hình thêm	MobSF, OWASP ZAP qua Docker
4	Cần cài đặt đơn giản kèm một vài bước cấu hình cơ bản	JADX, apktool
3	Cần cấu hình chứng chỉ hoặc mạng trước khi dùng được	Burp Suite
2	Cần chuẩn bị môi trường đặc biệt như nâng quyền hoặc đẩy tệp phục vụ	objection
1	Đòi hỏi viết mã hoặc kịch bản để dùng được cho tác vụ cơ bản	Frida

Cách neo thang điểm vào hiện tượng quan sát được như vậy không loại bỏ hoàn toàn tính chủ quan nhưng làm cho điểm số có thể tranh luận được trên cơ sở bằng chứng: nếu ai đó không đồng ý với mức điểm, họ phải chỉ ra công cụ thực tế cần bao nhiêu bước cấu hình, chứ không thể chỉ nói rằng cảm nhận của họ khác.

2.5.4. Cách tổng hợp kết quả và pipeline tự động hoá

Kết quả của từng lượt chạy được ghi vào một phiếu ghi kết quả theo mẫu thống nhất, trình bày ở Phụ lục D. Từ các phiếu này, hai chỉ số định lượng chính được tính theo công thức đã nêu trong Hình 2.7: tỷ lệ phát hiện bằng số phát hiện đúng chia cho số lỗ hổng kỳ vọng, và tỷ lệ báo động giả bằng số báo động giả chia cho tổng số cảnh báo. Năm tiêu chí còn lại được ghi trực tiếp: tốc độ là trung vị thời gian ba lần chạy, độ phủ là số lỗ hổng phát hiện đúng, còn ba tiêu chí về mức độ dễ sử dụng, chất lượng báo cáo và chi phí được chấm theo thang neo.

Đối với ba công cụ có giao diện lập trình là MobSF và OWASP ZAP, cùng với công cụ điều phối chung, toàn bộ quá trình tải tệp, kích hoạt quét, thu kết quả và chuẩn hoá được thiết kế để tự động hoá bằng một kịch bản Python; trong thực nghiệm ở Chương 3, phần đã được cài đặt và sử dụng là bộ chạy phân tích tĩnh của MobSF (mobsf_batch.py). Hình 2.8 mô tả kiến trúc của pipeline này.



Hình 2.8. Kiến trúc pipeline tự động hoá thu thập và tổng hợp kết quả

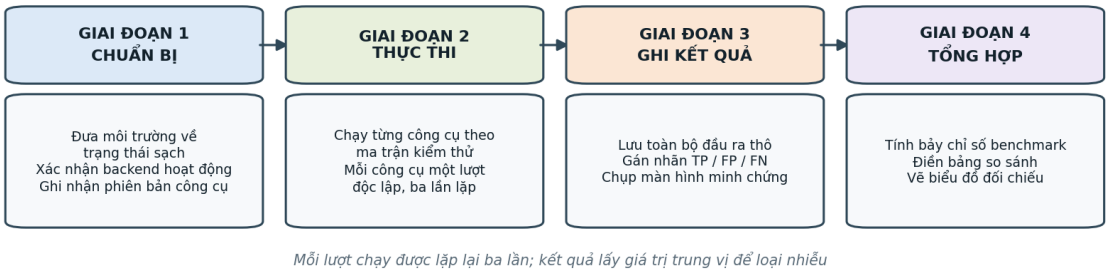
Trung tâm của pipeline là bộ chuẩn hoá kết quả. Vấn đề cần giải quyết là mỗi công cụ trả về kết quả theo một định dạng riêng: MobSF trả JSON phân cấp theo nhóm phát hiện, OWASP ZAP trả danh sách cảnh báo phẳng, còn các công cụ thủ công được ghi tay. Bộ chuẩn hoá ánh xạ tất cả về một lược đồ chung, trong đó mỗi bản ghi gồm mã công cụ, mã lỗ hổng V01–V12 nếu khớp được, nhãn phát hiện đúng hay báo động giả, và tham chiếu tới minh chứng. Nhờ có lược đồ chung này, bước tính chỉ

số và điền bảng tổng hợp áp dụng được cho cả bảy công cụ mà không cần xử lý riêng từng công cụ.

Cần nói rõ giới hạn của việc tự động hoá. Bốn công cụ không có giao diện lập trình — Burp Suite Community, Frida, JADX và apktool — không thể đưa vào pipeline một cách hoàn toàn tự động; kết quả của chúng được người kiểm thử ghi lại thủ công theo đúng lược đồ JSON chung. Điều này có nghĩa pipeline giúp giảm công sức và loại bỏ sai sót sao chép cho phần tự động hoá được, chứ không loại bỏ hoàn toàn thao tác thủ công. Việc mở rộng pipeline để bao trùm cả các công cụ này, chẳng hạn qua tiện ích mở rộng của Burp Suite, là một hướng phát triển được nêu ở phần kết luận của đồ án.

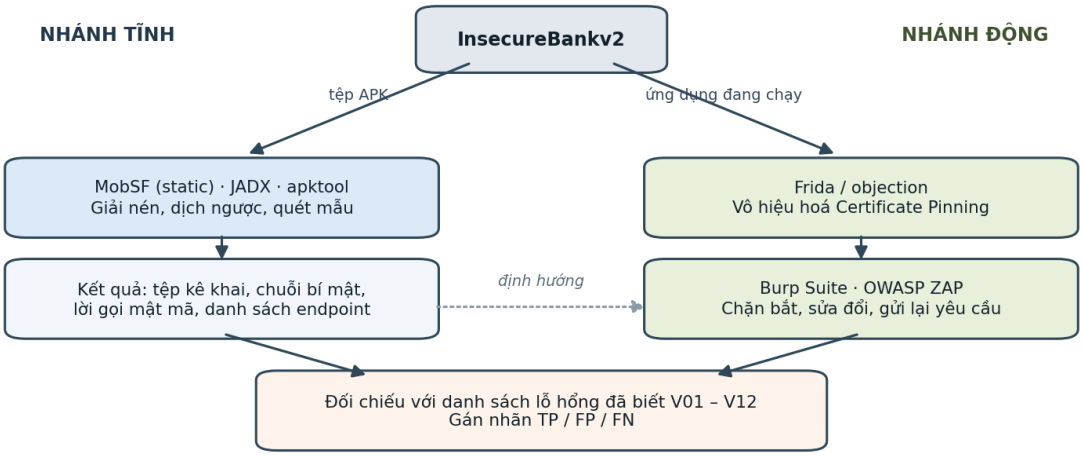
2.5.5. Quy trình thực hiện benchmark

Toàn bộ thực nghiệm được thực hiện theo quy trình bốn giai đoạn cố định, minh hoạ trên Hình 2.9. Việc chuẩn hoá quy trình nhằm bảo đảm điều kiện chạy của các công cụ là như nhau, để chênh lệch trong kết quả phản ánh chênh lệch về năng lực của công cụ chứ không phải chênh lệch về điều kiện thực nghiệm.



Hình 2.9. Quy trình thực hiện benchmark theo bốn giai đoạn

Hình 2.10 bổ sung góc nhìn theo luồng dữ liệu cho cùng quy trình đó, làm rõ quan hệ giữa hai nhánh kiểm thử: kết quả của nhánh tĩnh — đặc biệt là danh sách điểm cuối thu được từ việc dịch ngược — được dùng làm đầu vào định hướng cho nhánh động, thay vì hai nhánh chạy hoàn toàn tách rời. Cả hai nhánh cuối cùng đều đổ về cùng một bước đối chiếu với danh sách lỗ hổng đã biết và gán nhãn kết quả.



Hình 2.10. Luồng kiểm thử tĩnh và động trên ứng dụng đích

Một chi tiết trong quy trình cần được giải thích là việc mỗi lượt chạy được lặp lại ba lần và lấy giá trị trung vị. Đối với các tiêu chí về phát hiện, ba lần lặp gần như luôn cho kết quả giống nhau vì các công cụ hoạt động tất định trên cùng một đầu vào. Nhưng đối với tiêu chí tốc độ phân tích, chênh lệch giữa các lần chạy có thể lên tới vài chục phần trăm do ảnh hưởng của bộ nhớ đệm và tải hệ thống tại thời điểm đo. Việc lấy trung vị của ba lần thay vì một lần duy nhất giúp con số thời gian phản ánh đúng hơn chi phí thực tế của công cụ.

2.5.6. Các mối đe doạ tới tính hợp lệ của phép đo

Một nghiên cứu benchmark có giá trị chỉ khi nó thẳng thắn về những yếu tố có thể làm sai lệch kết luận của chính nó. Mục này liệt kê bốn nhóm mối đe doạ tới tính hợp lệ, theo cách phân loại thường dùng trong nghiên cứu thực nghiệm phần mềm, kèm biện pháp giảm thiểu đã áp dụng trong thiết kế.

Bảng 2.18. Các mối đe doạ tới tính hợp lệ và biện pháp giảm thiểu

Nhóm	Mối đe doạ	Biện pháp giảm thiểu
Hợp lệ nội tại	Kết quả bị ảnh hưởng bởi trạng thái tồn dư giữa các lượt chạy	Quy trình năm bước đưa môi trường về trạng thái sạch trước mỗi lượt (mục 2.3.5)
Hợp lệ nội tại	Chấm điểm hai tiêu chí Likert mang tính chủ quan	Neo mỗi mức điểm vào tiêu chí quan sát được (Bảng 2.17); một người chấm nhất quán cho toàn bộ
Hợp lệ cấu trúc	Cách định nghĩa phát hiện đúng có thể thiên vị một nhóm công cụ	Quy ước gán nhãn và ba điều kiện bổ sung được cố định trước khi chạy (mục 2.5.2)
Hợp lệ cấu trúc	Mẫu số của tỷ lệ phát hiện có thể trừng phạt công cụ không phải công cụ quét	Tính chỉ số trên mẫu số kỳ vọng theo ma trận, báo cáo song song tỷ lệ trên tổng

Nhóm	Mối đe doạ	Biện pháp giảm thiểu
Hợp lệ ngoại tại	Ứng dụng đích có lỗ hổng có chủ đích, dễ hơn ứng dụng thật	Nêu rõ kết quả là cận trên của năng lực công cụ; không suy rộng ra môi trường sản xuất
Hợp lệ ngoại tại	Kết quả gắn với một ứng dụng đích duy nhất	Chọn ứng dụng có tập lỗ hổng trải rộng chín hạng mục OWASP; nêu hạn chế và đề xuất mở rộng tập ứng dụng
Độ tin cậy	Phiên bản công cụ thay đổi làm kết quả không lặp lại được	Ghi cố định phiên bản từng công cụ; mô tả môi trường đủ chi tiết để tái lập
Độ tin cậy	Đo thời gian bị nhiễu do chạy chung một máy	Lặp ba lần lấy trung vị; không chạy hai công cụ đồng thời trong một lượt đo

Trong bốn nhóm trên, nhóm hợp lệ ngoại tại là nhóm mà thiết kế hiện tại còn yếu nhất và cũng khó khắc phục nhất trong khuôn khổ một đồ án. Kết quả thu được gắn với một ứng dụng đích duy nhất, nên chỉ có thể kết luận về năng lực của công cụ trên chính ứng dụng đó, chứ chưa thể suy rộng thành năng lực chung. Việc mở rộng tập ứng dụng đích sang một vài ứng dụng có lỗ hổng có chủ đích khác là hướng cải thiện trực tiếp nhất cho tính hợp lệ ngoại tại, và được ghi lại trong phần kết luận như một hướng phát triển của đề tài.

2.6. Kết luận chương 2

Chương 2 đã hoàn tất toàn bộ khâu chuẩn bị cho phần thực nghiệm của đồ án, đi qua bốn nhóm nội dung nối tiếp nhau. Về công cụ, chương đã phân tích chi tiết cơ chế hoạt động, cách vận hành, định dạng đầu ra và đặc biệt là ranh giới năng lực của từng công cụ trong bảy công cụ được lựa chọn, rồi tổng hợp thành một sơ đồ quyết định lựa chọn công cụ theo đặc trưng của lỗ hổng. Kết quả phân tích cho thấy mỗi công cụ có một vùng năng lực xác định bởi chính phương pháp mà nó dựa vào, và làm lộ ra một quy luật quan trọng: mức độ tự động hoá và chiều sâu phát hiện có xu hướng nghịch nhau.

Về ứng dụng đích, chương đã lập luận cho việc lựa chọn InsecureBankv2, mô tả luồng nghiệp vụ và các điểm cuối API, xây dựng một mô hình mối đe doạ gồm tài sản, tác nhân và bề mặt tấn công, đồng thời xác lập tập mười hai lỗ hổng đã biết làm đáp án chuẩn. Phân bố của tập lỗ hổng này cho một dự đoán định lượng có thể kiểm chứng và đồng thời đóng vai trò một phép kiểm tra tính nhất quán cho chính quy trình đo. Về môi trường, chương đã mô tả cấu hình thực nghiệm, trình tự triển khai mười bước kèm cách xác nhận, và một bảng các sự cố thường gặp — trong đó nhấn mạnh những sự cố làm sai lệch số liệu một cách âm thầm.

Về thiết kế đánh giá, chương đã trình bày mười hai kịch bản kiểm thử chi tiết tương ứng mười hai lỗ hổng, ma trận kiểm thử gồm ba mươi tư trường hợp, quy ước gán nhãn kết quả cùng ba điều kiện bổ sung, bảy tiêu chí đánh giá kèm thang neo điểm, cách tổng hợp kết quả và pipeline tự động hoá, và cuối cùng là một phân tích thẳng thắn về các mối đe doạ tới tính hợp lệ của phép đo. Điểm cần nhấn mạnh là toàn bộ khung đo được cố định trước khi chạy công cụ đầu tiên, và quyết định lấy mẫu số của tỷ lệ phát hiện theo kỳ vọng trong ma trận thay vì theo tổng số lỗ hổng là một khác biệt về phương pháp so với các nghiên cứu đã khảo sát ở Chương 1.

Khi bắt đầu thực nghiệm, một quan sát đã làm thay đổi trọng tâm của đồ án: cùng một bộ luật phân tích tĩnh cho mọi cảnh báo đúng trên InsecureBankv2 nhưng chỉ 1/19 cảnh báo đúng trên ứng dụng mã nguồn mở NewPipe. Vì vậy, với sự đồng ý của giáo viên hướng dẫn, Chương 3 không thực hiện toàn bộ ma trận ba mươi tư trường hợp mà tập trung xây dựng và đánh giá công cụ phân tích tĩnh apiscan. Các kết quả của chương này được sử dụng tiếp như sau: bộ đáp án mười hai lỗ hổng, sau khi được hiệu chỉnh ở mục 3.3.1, dùng để đo recall của công cụ; quy ước gán nhãn ở mục 2.5.2 được mở rộng để gán nhãn cảnh báo trên ứng dụng thật; phân tích ranh giới năng lực ở mục 2.1 là cơ sở lựa chọn kỹ thuật cho công cụ. Trong bảy công cụ, phần thực nghiệm chỉ chạy nhóm công cụ tĩnh — JADX, apktool và phân tích tĩnh của MobSF — để đối chứng với apiscan (mục 3.5). Các trường hợp kiểm thử động với Burp Suite, OWASP ZAP và Frida chưa được thực hiện và được nêu là hướng phát triển ở phần Kết luận.

CHƯƠNG 3. XÂY DỰNG VÀ ĐÁNH GIÁ CÔNG CỤ PHÁT HIỆN LỖ HỔNG TĨNH APISCAN

Chương 2 đã phân tích các công cụ hiện có, xây dựng môi trường thực nghiệm và bộ đáp án lỗ hổng trên ứng dụng InsecureBankv2. Chương 3 trình bày phần đóng góp chính của đồ án: thiết kế, cài đặt và đánh giá công cụ `apiscan` — một công cụ phân tích tĩnh tệp APK nhằm phát hiện dấu hiệu lỗ hổng theo OWASP Mobile Top 10 (2024) [1] và OWASP API Security Top 10 (2023) [2], có khả năng giảm báo nhầm theo ngữ cảnh và chấm điểm tin cậy cho từng cảnh báo. Chương gồm bảy phần: bối cảnh và câu hỏi nghiên cứu (3.1), thiết kế công cụ (3.2), phương pháp đánh giá (3.3), kết quả (3.4), đối chứng với công cụ sẵn có (3.5), recall trên lỗ hổng đã công bố (3.6), quét động có dẫn đường (3.7), thảo luận và hạn chế (3.8), kết luận chương (3.9).

3.1. Bối cảnh và câu hỏi nghiên cứu

Khi chạy thử một bộ luật phân tích tĩnh đơn giản ở giai đoạn đầu, đồ án ghi nhận một khoảng cách lớn giữa hai môi trường. Trên ứng dụng có lỗ hổng có chủ đích InsecureBankv2, mọi cảnh báo đều chỉ đúng một điểm yếu có thật (precision 24/24). Nhưng trên ứng dụng mã nguồn mở thực tế NewPipe, chỉ 1 trong 19 cảnh báo là lỗ hổng thật (precision 5,3%) — phần còn lại là mã nâng cấp http→https, URI namespace XML, thành phần buộc phải exported, hay một dòng log có chữ “secret” trong lời chào. Một báo cáo mà 18/19 cảnh báo là nhầm sẽ không được người kiểm thử sử dụng.

Từ quan sát đó, đồ án đặt ra ba mục tiêu đo được cho công cụ: (1) giữ recall 100% trên các lỗ hổng tĩnh của InsecureBankv2; (2) đạt precision từ 50% trở lên trên ứng dụng thật chưa từng dùng để chỉnh luật; (3) cho đầu ra dùng được — một lệnh từ tệp APK tới báo cáo HTML/JSON gắn nhãn OWASP, bản đồ endpoint và điểm tin cậy. Ba câu hỏi nghiên cứu tương ứng là:

- **RQ1.** Các kỹ thuật giảm báo nhầm dựa trên ngữ cảnh có giữ được recall trên bộ đáp án không?
- **RQ2.** Precision đo trên tập ứng dụng dùng để phát triển luật phản ánh precision trên ứng dụng mới đến mức nào?

- **RQ3.** Điểm tin cậy theo từng cảnh báo có giúp lọc báo nhầm mà không làm mất lỗ hổng thật không?

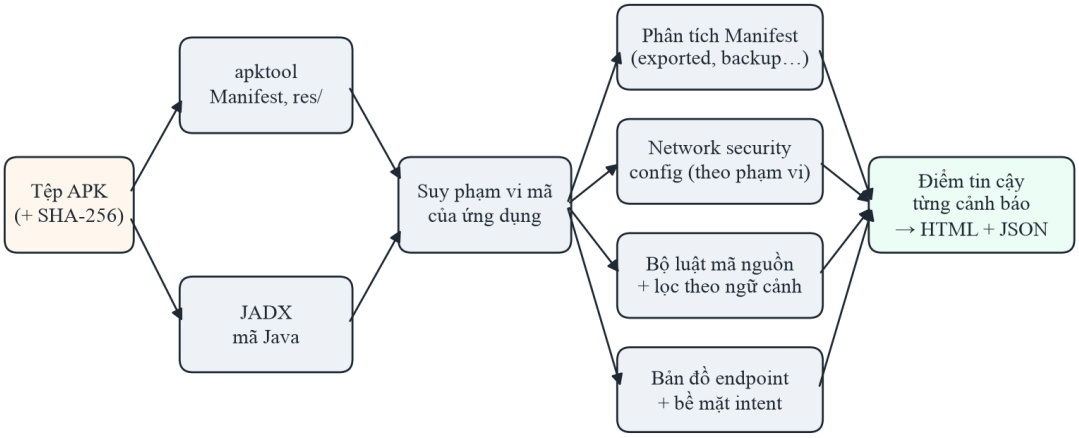
3.2. Thiết kế công cụ apiscan

3.2.1. Kiến trúc tổng thể

Hình 3.1 mô tả luồng xử lý của công cụ. Người dùng chỉ chạy một lệnh:

```
python3 apiscan.py scan app.apk --min-confidence medium
```

Công cụ tính SHA-256 của tệp APK, giải nén bằng apktool [16] (lấy AndroidManifest.xml và thư mục tài nguyên) và dịch ngược bằng JADX [15] (lấy mã Java) vào một thư mục làm việc đặt theo SHA-256, để lần quét sau dùng lại. Sau đó công cụ suy ra phạm vi mã của ứng dụng (mục 3.2.4), chạy bốn nhóm phân tích — Manifest, network security config, bộ luật trên mã nguồn, bản đồ endpoint và bề mặt tiếp nhận intent — rồi chấm điểm tin cậy cho từng cảnh báo và xuất báo cáo HTML cùng tệp JSON.



Hình 3.1. Kiến trúc của công cụ apiscan

Nguyên tắc thiết kế xuyên suốt là **chỉ đọc**: công cụ không cài đặt, không chạy ứng dụng và không gửi bất kỳ yêu cầu nào tới máy chủ của ứng dụng. Điều này cho phép áp dụng hợp pháp lên ứng dụng của bên thứ ba ở dạng phân tích tĩnh, phù hợp với ràng buộc đạo đức đã nêu ở Chương 1. Mã nguồn gồm `apiscan.py` (giao diện dòng lệnh) và `static_scan.py` (bộ luật và phân tích), kèm bộ kiểm thử hồi quy 37 ca trong `04_code/tests/`.

3.2.2. Bộ luật và ánh xạ OWASP

Công cụ có 20 luật, mỗi luật gắn một mục OWASP Mobile Top 10 – 2024 (Bảng 3.1). Luật cấu hình đọc Manifest và tệp network security config; luật mã nguồn quét từng dòng mã Java trong phạm vi ứng dụng.

Bảng 3.1. Bộ luật của apiscan theo nhóm OWASP Mobile Top 10 – 2024

Nhóm OWASP	Luật	Phát hiện
M1 – Improper Credential Usage	CRY-KEYLIT, CRY-KEYHARD, KEY-SERVICE	Bí mật viết cứng; khoá mã hoá truy về hằng; chuỗi có dạng khoá API dịch vụ
M4 – Insufficient Input/Output Validation	WEB-JS, WEB-IFACE	WebView bật JavaScript; cầu nối JavaScript ↔ mã native
M5 – Insecure Communication	NET-CLEAR, NET-NSC, NET-TRUST, NET-SDK, NET-HTTP	Cho phép cleartext; tin CA do người dùng cài; targetSdk < 28; URL http khi kết nối
M8 – Security Misconfiguration	CFG-DEBUG, CFG-BACKUP, CFG-EXPORT, API-HIDDEN	debuggable; allowBackup; thành phần exported không bảo vệ; endpoint dev/test
M9 – Insecure Data Storage	STO-EXT, STO-PREF-WORLD, LOG-SENS	Tạo tệp ở bộ nhớ ngoài; SharedPreferences world-readable; ghi log giá trị nhạy cảm
M10 – Insufficient Cryptography	CRY-WEAK, CRY-IV0, CRY-HASH	ECB/DES/RC4; IV tĩnh; MD5/SHA-1

Ngoài danh sách cảnh báo, công cụ xuất hai thông tin hỗ trợ kiểm thử: **bản đồ endpoint** (các URL và đường dẫn API xuất hiện trong mã có gọi mạng) và **bề mặt tiếp nhận intent** — các thành phần exported có lý do chính đáng (LAUNCHER, intent hệ thống, giao thức với ứng dụng khác), được liệt kê để rà soát cách xử lý dữ liệu đầu vào nhưng không tính là lỗ hổng.

3.2.3. Các kỹ thuật giảm báo nhầm theo ngữ cảnh

Phần lớn công sức của đồ án nằm ở việc biến các luật so khớp mẫu thô thành luật xét ngữ cảnh. Mỗi kỹ thuật dưới đây xuất phát từ một lớp báo nhầm cụ thể quan sát được trên ứng dụng thật, và đều được kiểm tra lại với ràng buộc không làm mất lỗ hổng nào của InsecureBankv2.

(1) NET-HTTP theo ngữ cảnh của chuỗi. Chuỗi `http://` chỉ là lỗ hổng khi được dùng làm địa chỉ kết nối. Công cụ phân tích hàm bao quanh chuỗi (bằng bộ khớp

ngoặc trên dòng mã) và bỏ qua các trường hợp chuỗi là định danh: đối số của hàm XML/namespace (`findChild`, `setFeature`, `setAttribute`...), đối số của phép so sánh (`equals`, `startsWith`), URI có dạng đặc tả (có `#`, chứa `/protocol/`, `/ns/`, `.dtd`), hằng namespace, liên kết mở bằng trình duyệt ngoài, và mã chuẩn hoá scheme (`"http://"` cùng `"https://"` trên một dòng). Phép hạ cấp `replace("https://", "http://")` vẫn bị cảnh báo. Trên Conversations, kỹ thuật này loại 42/42 báo nhầm do namespace giao thức XMPP mà vẫn giữ các cảnh báo `protocol = "http://"` của InsecureBankv2.

(2) CFG-EXPORT tách cảnh báo và bề mặt tiếp nhận. Manifest được đọc bằng bộ phân tích XML; thành phần exported ngầm định (có intent-filter khi targetSdk < 31, provider khi targetSdk < 17) cũng được tính. Một thành phần exported không có permission chỉ bị cảnh báo khi không có intent-filter, hoặc khi nhận một action tuỳ biến *do chính ứng dụng tự gửi* (kênh nội bộ bị mở ra ngoài). Ví dụ, `MyBroadCastReceiver` của InsecureBankv2 vẫn bị cảnh báo vì ứng dụng tự gửi action `theBroadcast` ở `ChangePassword.java:173`, trong khi các thành phần LAUNCHER hay nhận intent `VIEW`/`MEDIA_BUTTON` của NewPipe được chuyển sang mục bề mặt tiếp nhận.

(3) LOG-SENS chỉ khi ghi giá trị. Một lời gọi log chỉ bị cảnh báo khi in ra biến hay biểu thức mang tên nhạy cảm, hoặc khi chuỗi có từ khoá được nối với một giá trị (`"token=" + x`); thông điệp cố định như “Authentication exception for new token” bị bỏ qua. Trên K-9 Mail, kỹ thuật này loại 9/9 báo nhầm.

(4) STO-EXT chỉ khi tạo đường dẫn tệp. Chỉ cảnh báo khi thư mục bộ nhớ ngoài được ghép với một tên tệp (ứng dụng tự tạo tệp ở vùng dùng chung), bỏ qua việc chỉ lấy thư mục làm gốc cho bộ chọn tệp hay so sánh đường dẫn.

(5) CRY-KEYLIT phân biệt tên khoá và bí mật. Các hằng như `KEY_TITLE = "title"` (tên cột cơ sở dữ liệu) hay `PREF_PROXY_PASSWORD = "prefProxyPassword"` (khoá SharedPreferences) chiếm 141 báo nhầm ở AntennaPod và Nextcloud. Công cụ coi một hằng là tên khoá khi giá trị có dạng định danh và: là câu SQL, là tên đầy đủ theo package, tên biến theo quy ước `KEY_*`/`PREF_*`/`EXTRA_*`, hoặc giá trị trùng từ với tên biến. Bí mật dạng câu như khoá `"This is the super`

`secret key 123"` của InsecureBankv2 vẫn bị bắt. Luật này cũng quét thêm `BuildConfig.java`, nhờ đó phát hiện khoá API nhúng cứng của AntennaPod vốn bị bỏ sót trước đó.

(6) CRY-KEYHARD bằng phân tích luồng dữ liệu trong tệp. Thay vì cảnh báo mọi lời gọi `SecretKeySpec`, công cụ chỉ cảnh báo khi byte khoá truy ngược được về một hằng: literal chuỗi, mảng byte literal, `.getBytes()` hay giải mã Base64 của hằng. Giá trị "nhiễm" được lan truyền qua phép gán (trong phạm vi từng hàm, để tránh trùng tên biến tự sinh của JADX) và qua tham số: nếu một lời gọi trong cùng tệp truyền hằng vào vị trí tham số thứ k thì tham số đó bị coi là nhiễm. Đây chính là đường đi của khoá trong InsecureBankv2 — field `key` → biến `keyBytes` → tham số của `aes256decrypt()`. Kỹ thuật này loại 14/14 báo nhầm (khoá HMAC của SCRAM, khoá tin nhắn OMEMO, khoá dẫn xuất PBKDF2).

(7) NET-NSC/NET-TRUST theo phạm vi cấu hình. Tệp network security config [6] được đọc theo cấu trúc: chỉ xét tệp được Manifest tham chiếu; cleartext ở `base-config` là cảnh báo mức cao; ở `domain-config` chỉ cảnh báo khi có domain công khai (bỏ qua `.onion`, `localhost`, IP nội bộ); tin CA người dùng trong `debug-overrides` bị bỏ qua vì chỉ có hiệu lực ở bản debug.

3.2.4. Tự xác định phạm vi mã của ứng dụng

Một tệp APK chứa cả mã của ứng dụng lẫn hàng trăm thư viện. Quét toàn bộ sẽ sinh nhiễu từ thư viện, nhưng chỉ quét theo package khai báo trong Manifest lại dễ bỏ sót: package đó là applicationId, có thể khác namespace mã (KeePassDX có applicationId `com.kunzisoft.keepass.libre` nhưng mã ở `com.kunzisoft.keepass`) hoặc chỉ là một trong nhiều namespace (K-9 Mail có mã ở `com.fsck.k9`, `app.k9mail` và `net.thunderbird`). Công cụ suy phạm vi từ Manifest: package Manifest (nếu có mã), namespace của lớp Application, và namespace của các thành phần không thuộc danh sách thư viện đã biết khi namespace đó có activity LAUNCHER hoặc từ hai thành phần trở lên.

Tác động thực tế: với KeePassDX, cách quét theo package Manifest chỉ cho 1 cảnh báo (bỏ sót toàn bộ mã), còn phạm vi tự suy cho 50 cảnh báo; với Nextcloud, phạm vi mở rộng thêm `com.owncloud` và số cảnh báo tăng từ 9 lên 46.

3.2.5. Điểm tin cậy theo từng cảnh báo

Công cụ tách hai khái niệm: **mức nghiêm trọng** (hậu quả nếu cảnh báo đúng) và **độ tin cậy** (khả năng cảnh báo đúng). Độ tin cậy được chấm theo tín hiệu của chính cảnh báo đó, theo các quy tắc đặt ra trước khi đo (Bảng 3.2). Tầng high dành cho bằng chứng trực tiếp; tầng medium cho dấu hiệu mạnh nhưng mã có thể có kiểm tra mà phân tích tĩnh không thấy; tầng low cho bề mặt rủi ro tự nó chưa phải lỗ hổng.

Bảng 3.2. Một số quy tắc chấm độ tin cậy theo tín hiệu của cảnh báo

Luật	high	medium	low
LOG-SENS	In biến/field/getter có danh từ chính nhạy cảm (`this.password`, `distinctCredentials`)	Chuỗi kết thúc bằng từ khoá rồi nối giá trị	Từ khoá chỉ ở tên lớp/TAG hoặc giữa câu
CRY-KEYLIT	Giá trị dạng khoá ngẫu nhiên (≥ 16 ký tự, entropy ≥ 3,5 bit, có chữ và số) hoặc tiền tố khoá đã biết	Dạng passphrase	Nhãn chữ, tên định danh, đường dẫn/URL, bảng mã, chuỗi < 8 ký tự
CRY-IV0	Mảng thực sự được truyền vào `IvParameterSpec`/`GCMParameterSpec`	—	Không dùng làm IV
CFG-EXPORT	—	Mặc định	Thành phần LAUNCHER hoặc mã có kiểm tra danh tính ứng dụng gọi
CFG-BACKUP	Không có quy tắc backup/backupAgent	Có quy tắc backup	—
Cấu hình mạng, CRY-KEYHARD	Mặc định	`domain-config` cho domain công khai	—

Một quy tắc ban đầu — "thành phần không đọc dữ liệu từ intent thì xếp low" — đã bị loại bỏ vì sai về nguyên lý: activity chuyển tiền `DoTransfer` của InsecureBankv2 không đọc intent, nhưng việc ứng dụng khác mở thẳng được màn hình này mà không qua đăng nhập chính là lỗ hổng. Lỗi được phát hiện vì recall trên

InsecureBankv2 ở ngưỡng ≥ medium giảm còn 5/6; đây là ví dụ cho vai trò của bộ đáp án như một ràng buộc khi thiết kế quy tắc.

3.2.6. Đầu ra của công cụ

Báo cáo HTML nhóm cảnh báo theo mục OWASP, mỗi dòng có mức nghiêm trọng, độ tin cậy, vị trí tệp:dòng, trích đoạn bằng chứng và ghi chú, kèm bảng bề mặt tiếp nhận intent và bản đồ endpoint (Hình 3.2). Tệp JSON chứa cùng nội dung ở dạng máy đọc được, kèm SHA-256 của APK và phạm vi mã đã quét. Tham số `--min-confidence` lọc báo cáo theo tầng tin cậy.

Ngoài lệnh `scan`, công cụ có ba lệnh phục vụ sử dụng thực tế:

- **`batch`** quét nhiều tệp APK (hoặc cả thư mục) và xuất báo cáo riêng cho từng ứng dụng, kèm trang tổng hợp `index.html` và tệp `summary.csv`/`summary.json`: số cảnh báo theo tầng tin cậy, số endpoint, thời gian, và tỉ lệ ứng dụng có cảnh báo theo từng luật. Một APK lỗi không làm dừng cả lượt quét mà được ghi vào bảng tổng hợp.
- **Tệp cấu hình YAML** (`--config`) cho phép tắt luật, đặt ngưỡng tin cậy và khai báo danh sách bỏ qua (suppression). Mỗi cảnh báo có một dấu vân tay 12 ký tự tính từ luật, vị trí (bỏ số dòng) và bằng chứng, nên không đổi khi mã xung quanh dịch chuyển. Một mục bỏ qua khớp theo dấu vân tay, mã luật hoặc mẫu glob trên vị trí và bằng chứng, và bắt buộc có lý do. Cảnh báo bị bỏ qua vẫn được đếm và liệt kê kèm lý do trong báo cáo; mục bỏ qua không khớp cảnh báo nào được báo ra để phát hiện mục lỗi thời. Lệnh **`baseline`** sinh tệp cấu hình bỏ qua mọi cảnh báo hiện có, để các lần quét sau chỉ hiện cảnh báo mới.
- **`dynamic`** dùng bản đồ endpoint tĩnh làm điểm khởi đầu cho OWASP ZAP (truy cập, spider rồi active scan), gắn nhãn OWASP API Security Top 10 – 2023 cho cảnh báo theo CWE, và gộp với báo cáo tĩnh (cùng kết quả MobSF nếu có) thành một báo cáo hợp nhất. Lệnh có cổng an toàn: mục tiêu phải là địa chỉ cục bộ hoặc mạng riêng, nếu không phải có tệp văn bản cho phép (SHA-256 của tệp được ghi vào báo cáo); endpoint thuộc host khác trong bản đồ bị loại khỏi danh sách gửi cho ZAP. Ngoài đường dẫn, bước phân tích tĩnh còn trích phương thức và tên tham số của từng yêu cầu HTTP từ mã dịch ngược: lời tạo `HttpPost`/`HttpGet` kèm các `BasicNameValuePair` ngay sau đó (Apache HttpClient), và các annotation `@POST`, `@Field`, `@Query`, `@Path`, `@Body` của Retrofit. Lệnh `dynamic` dùng danh sách này để tự sinh yêu cầu mẫu với giá trị

giả cho ZAP; người dùng vẫn có thể cung cấp yêu cầu viết tay (ví dụ kèm thông tin đăng nhập hợp lệ) để thay thế yêu cầu tự sinh cùng phương thức và đường dẫn.



Báo cáo phân tích tĩnh Android — com.android.insecurebankv2

Phân tích CHỈ ĐỌC tệp APK · 24 cảnh báo · 6 endpoint · 2026-10-04 11:34

Mức / tin cậy	Cảnh báo	Vị trí	Bằng chứng / ghi chú
M1 — Improper Credential Usage (3)			
high tin cậy: high	SecretKeySpec dùng khoá bắt nguồn từ hằng viết cứng	com/android/insecurebankv2/CryptoClass.java:27	SecretKeySpec newKey = new SecretKeySpec(keyBytes, "AES"); *Byte khoá truy ngược (trong cùng tệp, qu số) về literal chuỗi hoặc mảng byte.*
high tin cậy: high	SecretKeySpec dùng khoá bắt nguồn từ hằng viết cứng	com/android/insecurebankv2/CryptoClass.java:35	SecretKeySpec newKey = new SecretKeySpec(keyBytes, "AES"); *Byte khoá truy ngược (trong cùng tệp, qu số) về literal chuỗi hoặc mảng byte.*
high tin cậy: medium	Khoá/bí mật viết cứng dạng chuỗi	com/android/insecurebankv2/CryptoClass.java:22	String key = "This is the super secret key *Chuỗi bí mật nhúng thẳng trong mã (đã lọ regex/PATTERN, SQL và hằng TÊN khoá r KEY_*/PREF_*).*
M10 — Insufficient Cryptography (1)			
high tin cậy: high	IV tĩnh (mảng byte khởi tạo bằng số 0)	com/android/insecurebankv2/CryptoClass.java:23	byte[] ivBytes = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0}; *IV cố định làm mất tính ngẫu nhiên của CI*
M4 — Insufficient Input/Output Validation (1)			
low tin cậy: low	WebView bật JavaScript	com/android/insecurebankv2/ViewStatement.java:30	mWebView.getSettings().setJavaScriptEn *Rủi ro khi kết hợp tải nội dung không tin c*
M5 — Insecure Communication (4)			
medium tin cậy: high	targetSdk=22 (<28): cleartext HTTP được phép mặc định	apktool.yml	targetSdkVersion=22 *Từ Android 9 (SDK 28) cleartext mới bị ch*
medium tin cậy: medium	Chuỗi http:// trong mã (cleartext)	com/android/insecurebankv2/DoLogin.java:51	String protocol = "http://"; *Có thể là endpoint gọi không mã hoá (đã namespace và mã kiểm tra/nâng cấp http*
medium tin cậy: medium	Chuỗi http:// trong mã (cleartext)	com/android/insecurebankv2/DoTransfer.java:65	String protocol = "http://"; *Có thể là endpoint gọi không mã hoá (đã namespace và mã kiểm tra/nâng cấp http*
medium tin cậy: medium	Chuỗi http:// trong mã (cleartext)	com/android/insecurebankv2/ChangePassword.java:57	String protocol = "http://"; *Có thể là endpoint gọi không mã hoá (đã namespace và mã kiểm tra/nâng cấp http*
M8 — Security Misconfiguration (9)			
high tin cậy: high	Ứng dụng bật cờ debuggable	AndroidManifest.xml:16	android:debuggable="true" *Bản phát hành không được để true.*

Hình 3.2. Báo cáo HTML của apiscan trên InsecureBankv2

3.3. Phương pháp đánh giá

3.3.1. Bộ đáp án InsecureBankv2 và hiệu chỉnh đáp án

Recall được đo trên sáu lỗ hổng thuộc phạm vi phân tích tĩnh của đáp án V01–V12 (Bảng 2.10): V01 (khoá viết cứng), V02 (HTTP không mã hoá), V05 (mật mã yếu), V06 (thành phần exported), V07 (cửa hậu `/devlogin`) và V11 (allowBackup). Điều kiện chấm được đặt chặt: V06 chỉ đạt khi phát hiện đủ cả sáu thành phần exported; V02 đạt khi phát hiện cả cấu hình targetSdk lẫn chuỗi `http://` ở ba tệp gọi mạng. Tệp APK có SHA-256 `b18af2a0…94a4`.

Trước khi dùng đáp án để chấm công cụ, đáp án được đối chiếu lại với hai nguồn sự thật: mã dịch ngược của ứng dụng và mã nguồn công khai của máy chủ đi kèm (`AndroLabServer/app.py`). Việc đối chiếu cho thấy mô tả của bốn mục và một phần Bảng 2.7 không khớp với mã thật (Bảng 3.3); các mục này đã được hiệu chỉnh trong Chương 2. Nhóm OWASP của cả bốn mục giữ nguyên. Trong sáu mục tĩnh dùng để đo recall, chỉ V05 thay đổi mô tả và vẫn thuộc M10, vẫn phát hiện được bằng phân tích tĩnh; V04, V08, V09 thuộc nhánh động, không tham gia phép đo. Vì vậy kết quả recall không bị ảnh hưởng. Sai sót trong chính tài liệu đáp án cho thấy đáp án cũng cần được kiểm chứng bằng mã nguồn trước khi dùng để chấm công cụ.

Bảng 3.3. Hiệu chỉnh đáp án InsecureBankv2 sau khi đối chiếu mã nguồn

Mục	Mô tả ban đầu	Mô tả sau đối chiếu	Bằng chứng
V04 (M9)	Tên đăng nhập và mật khẩu lưu dạng rõ trong SharedPreferences	Tên đăng nhập mã hoá Base64, mật khẩu mã hoá AES bằng khoá viết cứng — giải mã ngược được	`DoLogin.java:144-148`
V05 (M10)	Dùng DES khoá 56 bit	AES/CBC với khoá viết cứng và IV tĩnh toàn số 0	`CryptoClass.java:22-23`
V08 (API1)	Đổi tham số định danh để xem giao dịch của người khác	`/dotransfer` nhận tài khoản nguồn tuỳ ý → chuyển tiền từ tài khoản của người khác	`app.py`, hàm `dotransfer`
V09 (M3, API2)	Mã thông báo không được kiểm tra, phiên không hết hạn	Không có cơ chế phiên; `/changepassword` không xác thực; phản hồi `/login` cho phép dò tên người dùng	`app.py`, hàm `login`, `changepassword`
Bảng 2.7	Có điểm cuối `/getusers`; `/getaccounts` lộ dữ liệu thừa	Máy chủ chỉ có 5 điểm cuối; `/getaccounts` chỉ trả số tài khoản — rủi ro thật là gửi lại thông tin đăng nhập ở mọi yêu cầu	`app.py`

3.3.2. Tập ứng dụng thật

Precision được đo trên 19 ứng dụng mã nguồn mở tải từ kho F-Droid [26], chọn trải rộng nhiều miền chức năng để tránh kết luận chỉ đúng cho một loại ứng dụng (Bảng 3.4). Mọi tệp APK đều được ghi SHA-256 trong

`04_code/target-app/APK_SOURCES.md` để tái lập. Việc chỉ dùng ứng dụng mã nguồn mở và chỉ phân tích tĩnh bảo đảm thực nghiệm hợp pháp và không tác động tới máy chủ của bên thứ ba.

Bảng 3.4. Tập ứng dụng thật và vai trò trong đánh giá

Vai trò	Ứng dụng (phiên bản)	Miền chức năng
Phát triển ban đầu	NewPipe 0.29.1	Xem video
Holdout đợt 1	Tusky 32.2; K-9 Mail 23.1	Mạng xã hội; email
Holdout đợt 2	AntennaPod 3.12.2; Nextcloud 35.0.0	Podcast; đồng bộ tệp
Holdout đợt 3	Conversations 2.20.4; Jellyfin 2.7.3	Nhắn tin XMPP; máy chủ media
Holdout đợt 4	DAVx⁵ 4.5.20; Feeder 2.23.2	Đồng bộ lịch/danh bạ; RSS
Holdout đợt 5	Aegis 3.4.3; RedReader 1.26	Xác thực 2 lớp; Reddit
Holdout đợt 6	KeePassDX 4.5.5; Breezy Weather 6.2.2	Quản lý mật khẩu; thời tiết
Holdout đợt 7	Gadgetbridge 0.94.0; Wikipedia	Thiết bị đeo; tri thức
Holdout đợt 8	Briar 1.5.21; Money Manager Ex 5.5.13	Nhắn tin qua Tor; tài chính cá nhân
Holdout đợt 9	Termux 0.118.3; Catima 2.45.1	Terminal; ví thẻ
Holdout đợt 10	Organic Maps 2026.09.29; Etar 1.0.57; FlorisBoard 0.5.2; OpenTracks 4.29.0	Bản đồ; lịch; bàn phím; theo dõi thể thao

3.3.3. Quy trình gán nhãn

Mỗi cảnh báo được đối chiếu với mã dịch ngược, Manifest hoặc tệp cấu hình liên quan và gán một nhãn kèm lý do. **TP** là cảnh báo chỉ đúng một điểm yếu bảo mật có thật. **FP** được chia hai loại: *pattern* (luật khớp mẫu sai, ví dụ URI namespace bị coi là kết nối) và *intended* (hành vi cố ý hoặc bắt buộc của ứng dụng, ví dụ WebView đăng nhập OAuth cần JavaScript). Nhãn *borderline* đánh dấu các nhận định có thể tranh luận; với các mục này, chương báo cáo thêm số liệu "chặt" khi coi chúng là FP. Một số quy ước được giữ nhất quán xuyên ứng dụng: mật mã do giao thức quy định (CRAM-MD5, APOP, giao thức thiết bị đeo) là FP *intended*; cho phép cleartext hoặc tin CA người dùng ở cấp toàn ứng dụng là TP (điểm yếu có thật dù có chủ đích). Tổng cộng có 608 dòng nhãn trên 23 ứng dụng (589 dòng trên 19 ứng dụng của Bảng 3.7 và

19 dòng của holdout đợt 10), lưu ở `03_benchmark/ground-truth/`. Các lỗ hổng mà công cụ bỏ sót nhưng phát hiện được khi rà soát được ghi riêng ở trường `known_false_negatives`.

3.3.4. Giao thức đánh giá holdout

Khi một luật được chỉnh để loại các báo nhầm quan sát được trên một ứng dụng, precision trên chính ứng dụng đó tăng lên — nhưng không cho biết công cụ hoạt động thế nào trên ứng dụng mới. Để đo trung thực, đồ án dùng giao thức **holdout nhiều đợt**: trước mỗi đợt, toàn bộ luật được đóng băng tại một phiên bản mã (ghi lại mã commit); công cụ chạy trên hai ứng dụng mới; các cảnh báo được gán nhãn và đo precision; chỉ sau đó mới được sửa luật dựa trên ứng dụng này, và ứng dụng chuyển thành tập phát triển. Đợt kế tiếp lại dùng hai ứng dụng mới. Từ đợt 8, holdout được đo bằng chính lệnh `apiscan scan` với phạm vi mã tự suy, tức là đúng hành vi mặc định mà người dùng nhận được. Đợt 10 dùng bốn ứng dụng thay vì hai, vì mục đích chính của đợt này là đo hai thay đổi chỉ tác động tới luật CFG-EXPORT (mục 3.6), nên cần nhiều thành phần exported hơn; tiêu chí chọn ứng dụng và tiêu chí kết luận được commit trước khi tải APK.

3.3.5. Chỉ số đánh giá

- Recall tĩnh trên InsecureBankv2 = số mục tĩnh phát hiện được / 6.
- Precision = TP / (TP + FP) theo nhãn gán tay; báo cáo dạng gộp (micro, trên tổng số cảnh báo) và trung bình theo ứng dụng (macro).
- Precision theo tầng tin cậy: riêng từng tầng và cộng dồn (từ tầng đó trở lên).
- Tỉ lệ giảm số cảnh báo và số TP bị mất khi lọc theo ngưỡng tin cậy.
- Thời gian phân tích trên máy MacBook Apple Silicon.

3.4. Kết quả

3.4.1. Recall trên InsecureBankv2 (RQ1)

Công cụ phát hiện đủ **6/6 mục tĩnh** ở mọi phiên bản luật trong quá trình phát triển, với 24 cảnh báo đều là TP (precision 24/24) và bản đồ 6 endpoint (`/login`, `/devlogin`, `/getaccounts`, `/dotransfer`, `/changepassword`, `/Statements_`). Công cụ còn phát hiện thêm V04 (SharedPreferences ở chế độ world-readable) và V10

(ghi log tên đăng nhập và mật khẩu) dù đáp án xếp hai mục này vào nhánh động, vì bản thân mã nguồn đã để lộ dấu hiệu. Khi lọc ở ngưỡng tin cậy ≥ medium, recall vẫn là 6/6; chỉ giữ tầng high thì còn 2/6, nên ngưỡng khuyến nghị cho báo cáo là ≥ medium. Lỗ hổng V03 (thiếu certificate pinning) không phát hiện được vì là lỗi "vắng mặt một cơ chế", nằm ngoài khả năng của luật dựa trên dấu hiệu.

Như vậy, với RQ1: các kỹ thuật giảm báo nhầm ở mục 3.2.3 giữ được recall trên bộ đáp án. Mọi thay đổi luật đều được kiểm tra tự động bằng bộ kiểm thử hồi quy, gồm cả ca tích hợp kiểm tra recall và đối chiếu mọi cảnh báo của InsecureBankv2 với nhãn TP.

3.4.2. Precision trên tập phát triển

Bảng 3.5 cho thấy precision trên NewPipe tăng dần qua từng kỹ thuật, trên cùng một tệp APK.

Bảng 3.5. Precision trên NewPipe qua các cải tiến luật

Phiên bản luật	Số cảnh báo	TP	Precision
Ban đầu	19	1	5,3%
+ NET-HTTP theo ngữ cảnh	13	1	7,7%
+ CFG-EXPORT tách bề mặt tiếp nhận	7	1	14,3%
+ LOG-SENS theo giá trị	6	1	16,7%
+ STO-EXT theo đường dẫn tệp	3	1	33,3%

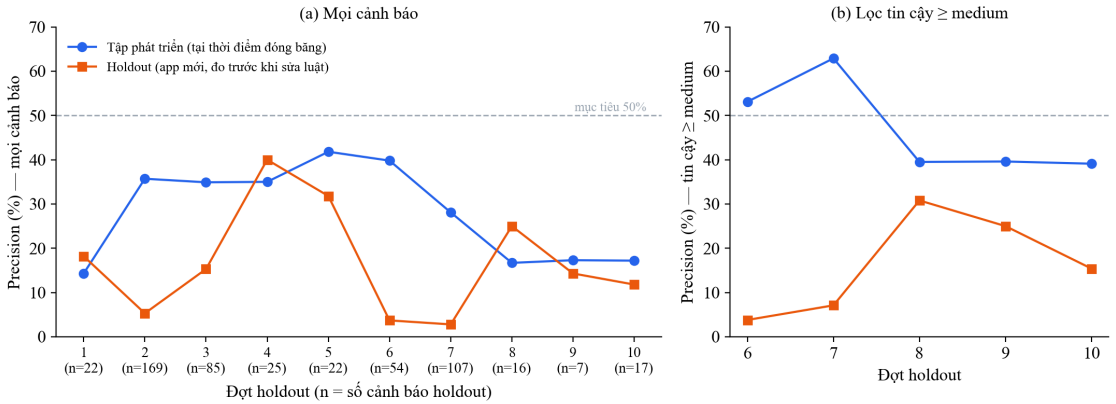
Trên toàn tập phát triển tại thời điểm đóng băng mỗi đợt, precision gộp dao động 14–42% với mọi cảnh báo và 40–63% ở ngưỡng ≥ medium (Hình 3.3). Nếu chỉ nhìn các con số này, có thể kết luận rằng công cụ gần đạt mục tiêu 50%.

3.4.3. Precision trên holdout (RQ2)

Bảng 3.6 trình bày precision của mười đợt holdout, mỗi đợt đo trên các ứng dụng mới trước khi luật được sửa dựa trên chúng. Đợt 10 được đo bằng bộ luật của chương này (v1).

Bảng 3.6. Precision trên holdout theo đợt

Đợt	Ứng dụng	Số cảnh báo	TP	Precision	Precision chặt
1	Tusky, K-9 Mail	22	4	18,2%	18,2%
2	AntennaPod, Nextcloud	169	9	5,3%	5,3%
3	Conversations, Jellyfin	85	13	15,3%	9,4%
4	DAVx⁵, Feeder	25	10	40,0%	32,0%
5	Aegis, RedReader	22	7	31,8%	27,3%
6	KeePassDX, Breezy Weather	54	2	3,7%	1,9%
7	Gadgetbridge, Wikipedia	107	3	2,8%	2,8%
8	Briar, Money Manager Ex	16	4	25,0%	18,8%
9	Termux, Catima	7	1	14,3%	14,3%
10	Organic Maps, Etar, FlorisBoard, OpenTracks	17	2	11,8%	5,9%



Hình 3.3. Precision trên tập phát triển và trên holdout qua các đợt

Kết quả trả lời RQ2 một cách rõ ràng: **precision trên tập phát triển không phản ánh precision trên ứng dụng mới.** Precision holdout dao động từ 2,8% đến 40% và không hội tụ theo thời gian, dù luật được cải tiến sau mỗi đợt. Ở ngưỡng ≥ medium, khoảng cách giữa tập phát triển và holdout ở các đợt 6–10 là từ 9 đến 56

điểm phần trăm (Hình 3.3b). **Mục tiêu precision ≥ 50% trên ứng dụng mới không đạt** ở bất kỳ đợt nào.

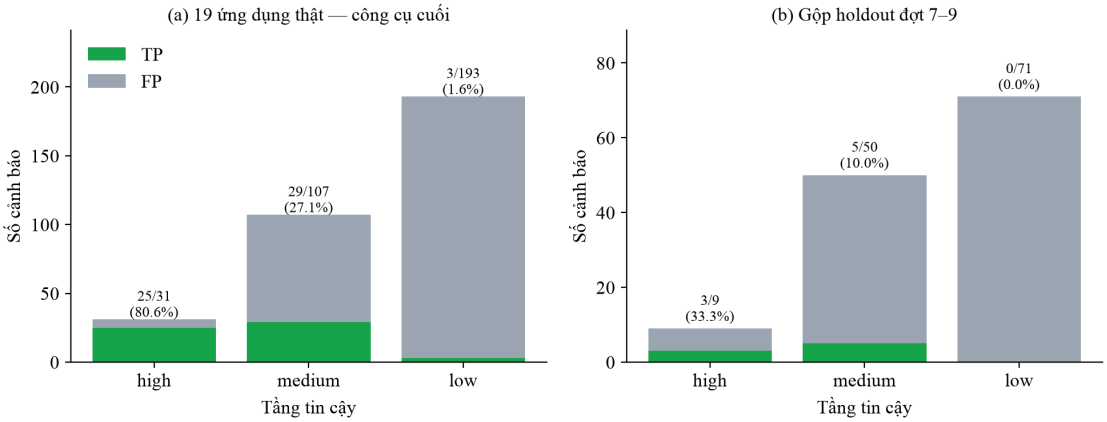
Nguyên nhân là mỗi miền ứng dụng sinh ra một lớp báo nhầm mới mà luật chưa gặp (mục 3.4.5). Ví dụ, đợt 2 làm lộ việc luật CRY-KEYLIT nhận nhầm hàng trăm tên khoá cơ sở dữ liệu là bí mật; đợt 3 làm lộ namespace giao thức XMPP; đợt 6 cho thấy ứng dụng quản lý mật khẩu có từ vựng trùng đúng các từ khoá mà luật dùng để tìm (password, token, secret).

3.4.4. Hiệu quả của điểm tin cậy (RQ3)

Bảng 3.7 và Hình 3.4 trình bày precision theo tầng tin cậy của phiên bản công cụ cuối cùng trên 19 ứng dụng thật.

Bảng 3.7. Precision theo tầng tin cậy, phiên bản công cụ cuối trên 19 ứng dụng thật (331 cảnh báo)

Tầng	Riêng tầng	Cộng dồn (≥ tầng)	Cộng dồn, chặt	Cộng dồn, nếu cấu hình mạng/backup là FP
high	80,6% (25/31)	80,6%	80,6%	9,7%
medium	27,1% (29/107)	39,1% (54/138)	31,2%	21,0%
low	1,6% (3/193)	17,2% (57/331)	13,6%	9,7%



Hình 3.4. TP và FP theo tầng tin cậy

Ba nhận xét rút ra:

- **Tầng low là bộ lọc nhiễu đáng tin cậy.** Chỉ 3/193 cảnh báo tầng low là TP. Lọc ở ngưỡng ≥ medium giảm số cảnh báo từ 331 xuống 138 (−58,3%) và chỉ

mất 3/57 TP. Trên holdout đợt 7–9 (đo khi điểm tin cậy đã đóng băng), tầng low không chứa TP nào (0/71); holdout đợt 10 cho kết quả cùng chiều (0/4). Đây là đóng góp thực tế nhất của điểm tin cậy: người kiểm thử chỉ cần đọc khoảng 40% số cảnh báo mà gần như không bỏ sót lỗ hổng thật.

- **Tầng high có precision cao (80,6%) nhưng phụ thuộc mạnh vào quy ước gán nhãn.** 22/25 TP của tầng high là cấu hình mạng hoặc sao lưu ở cấp toàn ứng dụng. Nếu hội đồng coi nhóm này là hành vi có chủ đích (FP), precision tầng high chỉ còn 9,7%. Các FP còn lại của tầng high đều là trường hợp ứng dụng buộc phải dùng giá trị cố định: IV và khoá theo giao thức thiết bị (Gadgetbridge), IV theo định dạng sao lưu của ứng dụng khác (Aegis) và khoá công khai của Google Play (Briar).
- **Tầng medium không vượt được giới hạn của phân tích tĩnh.** Precision 27,1% trên 19 ứng dụng và 10% trên holdout đợt 7–9: phần lớn cảnh báo medium là thành phần exported chỉ mở giao diện, cầu nối JavaScript tới trang tin cậy, hay chuỗi `http://` mà phân tích tĩnh không phân biệt được với kết nối thật.

Như vậy, với RQ3: điểm tin cậy theo từng cảnh báo giúp lọc khoảng 60% báo nhầm gần như không mất lỗ hổng thật, nhưng không nâng precision ở ngưỡng ≥ medium lên 50% trên ứng dụng mới.

3.4.5. Phân tích báo nhầm theo luật và theo miền ứng dụng

Bảng 3.8 cho thấy precision rất khác nhau giữa các luật. Luật cấu hình mạng và sao lưu có precision cao; luật dựa trên từ khoá trong mã (LOG-SENS, CRY-KEYLIT, STO-EXT) và luật mật mã có precision thấp.

Bảng 3.8. Precision theo luật, phiên bản công cụ cuối trên 19 ứng dụng thật

Luật	TP / tổng	Precision	Lớp báo nhầm chính
NET-NSC	7/7	100%	—
NET-TRUST	7/7	100%	—
NET-CLEAR	5/5	100%	—
CFG-BACKUP	6/11	54,5%	Có backupAgent/quy tắc chỉ sao lưu dữ liệu không nhạy cảm

Luật	TP / tổng	Precision	Lớp báo nhầm chính
CFG-EXPORT	12/43	27,9%	Activity chỉ mở giao diện; provider được bảo vệ bằng permission `dangerous`; kiểm tra quyền trong mã
NET-HTTP	9/39	23,1%	Hằng `"http://"` dùng để so sánh; URL mẫu; link mở trình duyệt; namespace mới
WEB-IFACE	5/22	22,7%	Cầu nối tới trang cục bộ hoặc trang tin cậy; hàm tự viết trùng tên
CRY-KEYLIT	2/36	5,6%	Nhãn trường, tên tham số, đường dẫn API, khoá công khai
LOG-SENS	2/40	5,0%	Tên lớp/TAG chứa từ khoá; thông điệp nối với giá trị không nhạy cảm
STO-EXT	2/75	2,7%	Tệp xuất/cache do người dùng chủ động; đường dẫn chỉ để so sánh
WEB-JS, CRY-HASH, CRY-WEAK, CRY-IV0, CRY-KEYHARD	0/46	0%	JS cần cho trang; hash không dùng cho bảo mật; mật mã do giao thức quy định

Theo miền ứng dụng, báo nhầm tập trung theo kiểu: ứng dụng dùng giao thức dựa trên XML (podcast, XMPP, GPX) sinh báo nhầm namespace; ứng dụng quản lý mật khẩu sinh báo nhầm từ khoá; ứng dụng thiết bị đeo sinh báo nhầm mật mã giao thức; ứng dụng lớn có nhiều thành phần cấu hình exported. Precision theo ứng dụng vì vậy dao động từ 0% (Aegis, Wikipedia, Briar, Termux) tới 70% (Jellyfin), với trung bình theo ứng dụng 26,4% và gộp 17,2%.

3.4.6. Các phát hiện bảo mật trên ứng dụng thật

Dù precision chưa đạt mục tiêu, công cụ đã dẫn tới một số phát hiện có giá trị khi người kiểm thử rà soát cảnh báo (Bảng 3.9). Mọi phát hiện đều từ phân tích tĩnh và cần kiểm chứng động — chỉ được thực hiện khi có sự cho phép của chủ ứng dụng.

Bảng 3.9. Một số phát hiện bảo mật đáng chú ý

Lớp phát hiện	Ứng dụng	Ghi chú
Hai lỗ hổng mức nghiêm trọng cao ở thành phần exported không được bảo vệ (một liên quan kho chứng chỉ tin cậy, một liên quan dữ liệu tài chính)	Hai ứng dụng trong tập đánh giá	Ẩn danh cho tới khi công bố có trách nhiệm cho nhà phát triển
Activity cắt/sửa ảnh exported trả dữ liệu ảnh đã giải mã cho ứng dụng gọi	Tusky, Conversations (thư viện CanHub cropper), Gadgetbridge	Rủi ro chuỗi cung ứng (M2)
`PreviewActivity` của công cụ debug Compose lọt vào bản phát hành	Feeder, RedReader, Breezy Weather	Cấu hình build sai
Provider exported bỏ kiểm tra người gọi cho bảng metadata tệp	Nextcloud	Độ tin cậy trung bình
Sao lưu token/cookie/cơ sở dữ liệu không yêu cầu mã hoá	Jellyfin, RedReader	Trích xuất được qua adb backup trên Android ≤ 11
Thông tin xác thực/khoá API nhúng cứng	Feeder (Basic Auth đồng bộ), AntennaPod (PodcastIndex)	
Ghi log mật khẩu ICE	Conversations	Mức nhẹ

3.4.7. Tốc độ

Phân tích (không gồm giải nén) mất trung vị 0,82 giây mỗi ứng dụng, chậm nhất 6,92 giây với Gadgetbridge (4.918 tệp Java trong phạm vi). Toàn bộ quy trình một lệnh trên InsecureBankv2, gồm cả giải nén bằng apktool và dịch ngược bằng JADX, mất 14 giây; với ứng dụng lớn, phần lớn thời gian nằm ở bước dịch ngược JADX.

Lệnh `batch` chạy lại trên toàn bộ 20 tệp APK (19 ứng dụng thật và InsecureBankv2) cho đúng số cảnh báo của từng ứng dụng như phiên bản công cụ cuối (tổng 331 cảnh báo trên 19 ứng dụng thật), kể cả với bốn ứng dụng được giải nén và dịch ngược lại từ đầu. Kết quả nằm ở `03_benchmark/ket-qua-test/batch_20apk/`.

3.5. Đối chứng với công cụ sẵn có

Trên InsecureBankv2, JADX 1.5.0 làm lộ bằng chứng cho 6/12 lỗ hổng (thêm 2 ở mức gợi ý) trong 22,9 giây mỗi lượt, apktool 2.9.3 cho 2/12 trong 7,8 giây, cả hai không có báo nhầm. Tuy nhiên hai công cụ này chỉ dịch ngược; người kiểm thử phải tự đọc mã để tìm lỗ hổng. `apiscan` tự động hoá chính bước “tìm” đó trên đầu ra của hai công cụ, và đạt 6/6 mục tĩnh mà không cần đọc mã thủ công.

Để so sánh với công cụ phân tích tĩnh phổ biến nhất hiện nay, đồ án chạy MobSF v4.5.4 [10] (Docker, cấu hình mặc định, chỉ phân tích tĩnh) trên đúng 20 tệp APK đã dùng và chấm bằng cùng định nghĩa nhãn. Với MobSF, một cảnh báo là một cặp (luật, tệp) ở phần mã, hoặc một mục manifest, network security config hay chứng chỉ ở mức high/warning. Mức info và danh sách "secrets" không được tính. Vì MobSF đưa ra quá nhiều cảnh báo để gán nhãn hết, precision của MobSF được ước lượng từ 100 cảnh báo lấy ngẫu nhiên (seed cố định), kèm khoảng tin cậy 95% theo phương pháp Wilson [27]. Khi một cảnh báo trùng thành phần hoặc tệp đã gán nhãn cho `apiscan`, nhãn cũ được dùng lại để giữ nhất quán. Kết quả ở Bảng 3.10.

Bảng 3.10. Đối chứng apiscan với MobSF trên cùng tập ứng dụng và cùng định nghĩa nhãn

Chỉ số	apiscan	MobSF 4.5.4
Recall tĩnh InsecureBankv2 (cấu hình mặc định)	6/6	2/6 (V06, V11)
Recall tĩnh InsecureBankv2 (MobSF chỉnh `SKIP_CLASS_PATH`)	—	3/6 + V05 một phần
Số cảnh báo trên 19 ứng dụng thật	331	1.182 (+2.177 mức info)
Precision, mọi cảnh báo (KTC 95%)	17,2% (13,5–21,7%)	5,0% (2,2–11,2%), mẫu 100
Precision phần phân tích mã (KTC 95%)	7,8% (20/258; 5,1–11,7%)	1,3% (1/75; 0,2–7,2%)
Precision theo quy ước nhạy (cấu hình mạng → FP)	9,7%	3,0%
TP của apiscan mà MobSF cũng báo ở mức high/warning	—	41/57
Thời gian mỗi ứng dụng (trung vị)	0,82 s phân tích	84 s, gồm giải nén

Trên InsecureBankv2, MobSF ở cấu hình mặc định không báo được lỗ hổng nào trong mã: danh sách bỏ qua `SKIP_CLASS_PATH` có tiền tố `com/android/`, trùng với package `com.android.insecurebankv2` của ứng dụng. Ngay cả khi bỏ tiền tố này, MobSF vẫn không phát hiện V02 (HTTP cleartext) và V07 (cửa hậu `/devlogin`). Với V05, MobSF chỉ báo chế độ CBC có nguy cơ padding oracle, không báo IV tĩnh và khoá cứng.

Trên ứng dụng thật, MobSF báo được phần lớn các TP cấu hình mà `apiscan` tìm thấy (thành phần exported, sao lưu, cleartext, tin CA người dùng). MobSF bỏ sót 16/57 TP:

- 9 URL `http://` ghép chuỗi trong mã, vì MobSF không có luật tương ứng.
- 5 cầu nối JavaScript, vì luật của MobSF đòi `setJavaScriptEnabled` và `addJavascriptInterface` nằm cùng một tệp.
- 2 lệnh log dữ liệu nhạy cảm, mà MobSF chỉ xếp chung vào mức info cùng mọi lệnh log.

Ngược lại, mẫu của MobSF chứa một lỗ hổng mà `apiscan` bỏ sót. Một ứng dụng trong tập tạo cơ sở dữ liệu người dùng mặc định ở bộ nhớ ngoài, nhưng đoạn mã này bị R8 dời sang package `defpackage`, nằm ngoài phạm vi `apiscan` tự suy.

Precision của MobSF thấp hơn ở mọi quy ước gán nhãn. Ở quy ước chính, hai khoảng tin cậy không chồng nhau; ở quy ước nhạy, chúng chồng nhau một phần (1,0–8,5% so với 6,9–13,3%). Phần phân tích mã của MobSF gần như toàn báo nhầm. Trong mẫu có 36 cảnh báo `android_hardcoded`: 35 là tên khoá preference, hằng số `*_KEY` hoặc chuỗi `toString()`, cái còn lại là mật khẩu cố định của định dạng backup của một ứng dụng khác mà ứng dụng đang xét chỉ dùng để nhập dữ liệu. Đây đúng là lớp báo nhầm mà kỹ thuật nhận diện hằng số tên khoá ở mục 3.2.3 loại bỏ. Các cảnh báo về Random không an toàn, lộ địa chỉ IP và truy vấn SQL thô (27 cảnh báo, không có luật tương ứng trong `apiscan`) cũng đều sai.

Về số lỗ hổng thật, MobSF ước lượng có khoảng 59 TP (KTC 26–132), tương đương 57 TP của `apiscan`. Khác biệt chính vì vậy nằm ở chi phí kiểm tra: để có cùng số lỗ hổng thật, người kiểm thử phải đọc số cảnh báo nhiều gấp khoảng 3,6 lần khi dùng MobSF. Phép đối chứng có ba hạn chế:

- Precision của MobSF là ước lượng từ mẫu.
- Người gán nhãn đã biết nhãn của `apiscan`.
- `apiscan` đã được chỉnh luật qua các đợt holdout trên chính các ứng dụng này; trên ứng dụng mới, precision của `apiscan` chỉ 2,8–40% (mục 3.4.3), nên lợi thế có thể nhỏ hơn.

Chi tiết số liệu và nhãn có lý do nằm ở `03_benchmark/ket-qua-test/doi_chung_mobsf.md` và `03_benchmark/ground-truth/mobsf_sample_labels.json`.

3.6. Recall trên lỗ hổng đã công bố

Nhãn ở mục 3.4 chỉ gán cho cảnh báo mà công cụ đưa ra, nên chưa trả lời được câu hỏi bổ sung: *trên ứng dụng thật, `apiscan` bỏ sót bao nhiêu lỗ hổng?* Để có đáp án độc lập với quá trình chỉnh luật, đồ án lấy các lỗ hổng do GitHub Security Lab công bố [29] trên ứng dụng Android mã nguồn mở có trên F-Droid. Hai phát hiện của chính `apiscan` không được dùng, để tránh thiên lệch. Tập gồm 10 lỗ hổng trên 5 ứng dụng: Nextcloud (CVE-2021-43863, CVE-2021-41166), ownCloud (GHSL-2022-059, GHSL-2022-060), Nextcloud News (CVE-2021-41256), Wikipedia (CVE-2026-65993, CVE-2026-65994) và OsmAnd (CVE-2026-65995, CVE-2026-65996, CVE-2026-65997).

Với mỗi lỗ hổng, đồ án tải bản APK bị ảnh hưởng từ kho lưu của F-Droid (đối chiếu SHA-256 với chỉ mục F-Droid; với hai lỗ hổng năm 2026, chọn bản phát hành trước ngày bản sửa được xác nhận). Luật được giữ nguyên như phiên bản cuối của mục 3.4, và tiêu chí chấm được ghi lại trước khi quét. Mỗi lỗ hổng được xếp vào một trong ba mức: - **phát hiện đúng cơ chế**: có cảnh báo gọi tên đúng thành phần, và luật của cảnh báo mô tả đúng nguyên nhân; - **chỉ đúng thành phần**: thành phần xuất hiện trong cảnh báo hoặc trong mục bề mặt tiếp nhận intent, nhưng luật không mô tả nguyên nhân; - **bỏ sót**.

Có một sai lệch so với tiêu chí ghi trước. Manifest của Nextcloud 3.18.0 cho thấy CVE-2021-41166 đã được sửa ở bản này, nên lỗ hổng được chấm trên bản 3.17.0 mà advisory nêu. Kết quả ở Bảng 3.11.

Bảng 3.11. Recall của apiscan trên 10 lỗ hổng đã công bố

Mức	Số lỗ hổng	Lỗ hổng
Phát hiện đúng cơ chế	3/10 (2/10 ở ngưỡng ≥ medium)	CVE-2021-43863 (low), CVE-2021-41166 (medium), GHSL-2022-059 (medium)

Mức	Số lỗ hổng	Lỗ hổng
Chỉ đúng thành phần	7/10	GHSL-2022-060, CVE-2021-41256, CVE-2026-65993, CVE-2026-65994, CVE-2026-65995, CVE-2026-65996, CVE-2026-65997
Bỏ sót	0/10	—

Ba lỗ hổng phát hiện đúng cơ chế đều là content provider exported thiếu permission, tức là lỗi cấu hình đọc được trực tiếp từ Manifest. Bảy lỗ hổng còn lại là lỗi logic khi xử lý dữ liệu đầu vào của các thành phần có lý do chính đáng để exported: - kiểm tra host của deep link bằng `endsWith`; - path traversal qua tham số tên tệp; - trả nguyên intent nhận được qua `setResult`, khiến cờ cấp quyền URI có hiệu lực; - dịch vụ AIDL không xác thực bên gọi.

Mọi thành phần này đều có trong mục bề mặt tiếp nhận intent, nên báo cáo dẫn người rà soát tới đúng chỗ. Nhưng luật dựa trên dấu hiệu không kết luận được đó là lỗ hổng. Kết quả này khớp với thiết kế của công cụ: tách cảnh báo khỏi bề mặt tiếp nhận giúp giảm báo nhầm (mục 3.2.3), và cái giá là các lỗi logic ở bề mặt đó chỉ được chỉ ra, không được cảnh báo.

Phân tích sau khi xem kết quả (không dùng để sửa các số liệu trên) chỉ ra hai điểm cần cải tiến: - **CVE-2021-43863 bị xếp tầng low** vì mã provider có gọi API kiểm tra ứng dụng gọi. Có lời gọi kiểm tra không có nghĩa là kiểm tra đúng, nên quy tắc hạ tin cậy này làm mất một lỗ hổng thật ở ngưỡng khuyến nghị. - **CVE-2026-65995 là dịch vụ exported không có permission.** Công cụ xếp nó vào bề mặt tiếp nhận chỉ vì dịch vụ nhận một action tuỳ biến. Nếu nâng riêng nhóm “service nhận action tuỳ biến, không permission” thành cảnh báo, trên 19 ứng dụng thật chỉ phát sinh thêm 6 cảnh báo. Tác động của thay đổi này lên precision cần được đo bằng một đợt holdout mới.

Hai thay đổi trên được cài thành bộ luật thử nghiệm v2 (tuỳ chọn `--export-rules v2`, bộ luật mặc định v1 giữ nguyên) và đo bằng holdout đợt 10 trên bốn ứng dụng mới (Bảng 3.4). Luật được đóng băng, tiêu chí chọn ứng dụng và tiêu chí kết luận được commit trước khi tải APK: giữ một thay đổi nếu nhóm cảnh báo bị nó tác động có precision không thấp hơn precision ≥ medium của v1 trong cùng đợt, bỏ nếu thấp hơn, và chưa kết luận nếu nhóm đó có ít hơn 3 cảnh báo. - Precision của v1 là 11,8% (2/17), ở ngưỡng ≥ medium là 15,4% (2/13). Precision của v2 là 10,5% (2/19),

ở ngưỡng ≥ medium là 13,3% (2/15). - Thay đổi thứ nhất không tác động tới cảnh báo nào trên bốn ứng dụng này. - Thay đổi thứ hai thêm 2 cảnh báo, cả hai là FP. Đó là các dịch vụ do thư viện khai báo (Android Auto, Health Connect) và có xác thực bên gọi trong mã.

Theo tiêu chí ghi trước, cả hai thay đổi **chưa kết luận được**, vì mỗi nhóm có ít hơn 3 cảnh báo. Chạy lại trên tập phát triển và tập lỗ hổng đã công bố (chỉ để tham khảo, vì đây là dữ liệu đã dùng để rút ra hai thay đổi) cho thấy hai thay đổi đi theo hai chiều khác nhau: - Thay đổi thứ nhất nâng 2 cảnh báo từ low lên medium trên 19 ứng dụng (1 TP, 1 FP), và đưa CVE-2021-43863 lên ngưỡng khuyến nghị. - Thay đổi thứ hai biến CVE-2026-65995 thành cảnh báo, nên recall đúng cơ chế tăng từ 3/10 lên 4/10. Nhưng cả 6 cảnh báo mới trên 19 ứng dụng đều là dịch vụ của thư viện (Media3, UnifiedPush, Health Connect) nhận action giao thức của thư viện, cùng dạng với 2 FP của holdout: luật coi action của thư viện là action tuỳ biến của ứng dụng.

Vì vậy bộ luật mặc định vẫn là v1, và mọi số liệu khác của chương dùng v1. Chi tiết ở `03_benchmark/ket-qua-test/m3_holdout10/`.

Trên các bản đã sửa, công cụ phân biệt được bản sửa dạng cấu hình: khi Nextcloud thêm `android:permission`, cảnh báo biến mất. Các bản sửa logic (Nextcloud News, OsmAnd, Wikipedia) thì công cụ không phân biệt được. Tập lỗ hổng còn nhỏ và thiên về nhóm thành phần exported, vì nguồn công bố chính tập trung vào nhóm này; recall của các luật mật mã, lưu trữ và ghi log vì vậy chưa được đo. Dữ liệu và báo cáo nằm ở `03_benchmark/recall_cve/`.

3.7. Quét động có dẫn đường trên InsecureBankv2

Phần này kiểm tra lệnh `dynamic` (mục 3.2.6) trên backend của InsecureBankv2. Backend chạy trong container chỉ lắng nghe ở 127.0.0.1, và OWASP ZAP 2.17.0 dùng chung network namespace với nó. Để đo phân tích tĩnh giúp quét động được bao nhiêu, ZAP được chạy với bốn cấu hình. Mỗi cấu hình dùng một cặp container mới để cơ sở dữ liệu luôn ở trạng thái sạch. Kết quả ở Bảng 3.12.

Bảng 3.12. Kết quả ZAP trên backend InsecureBankv2 theo đầu vào dẫn đường

Cấu hình	Đầu vào cho ZAP	Cảnh báo	Mức high	Endpoint API được quét
A	Chỉ URL gốc	3	0	0/6
B	A + 6 đường dẫn từ bản đồ endpoint tĩnh	9	0	6/6, chỉ bằng GET
C	B + 5 POST viết tay, tài khoản hợp lệ	27	2	6/6, POST có tham số
D	B + 5 POST tự trích, giá trị giả	26	2	6/6, POST có tham số

Backend không có trang liên kết nào, nên spider của ZAP không tự tìm được endpoint; không có bản đồ thì ZAP không quét được API. Riêng đường dẫn cũng chưa đủ: các endpoint chỉ nhận POST với tham số form, nên chỉ cấu hình C và D cho kết quả đáng kể. Hai cảnh báo high của cấu hình C là **XSS phản xạ** ở `/login` và `/devlogin` (tham số `username`); kiểm tra lại bằng tay cho thấy phản hồi `text/html` chứa nguyên đoạn mã chèn vào. Đây là lỗ hổng không có trong đáp án V01–V12. Các cảnh báo còn lại là thiếu header bảo mật.

Cấu hình D kiểm tra việc tự động hoá bước dựng yêu cầu. Công cụ tự trích được đủ 5 yêu cầu POST, khớp từng tên tham số với tệp viết tay của cấu hình C. Vì giá trị tham số không có trong mã tĩnh, yêu cầu được điền giá trị giả, và ZAP thay bằng payload khi quét chủ động. Kết quả D có đúng hai cảnh báo XSS phản xạ như C; toàn bộ lệnh chạy trong 123 giây. Khác biệt chỉ ở mức low và medium: D có thêm hai cảnh báo lộ thông tin lỗi do `/dotransfer` trả HTTP 500 với tên người dùng giả, và ít hơn ba cảnh báo về header. Trên 16 ứng dụng thật đã giải nén, việc trích yêu cầu không làm thay đổi số cảnh báo tĩnh; công cụ trích được yêu cầu ở Wikipedia (33, Retrofit) và Conversations (2), còn Tusky dù dùng Retrofit nhưng bị R8 đổi tên cả thư viện lẫn annotation nên không trích được. Cách dựng yêu cầu bằng builder của OkHttp chưa được hỗ trợ.

ZAP không phát hiện được lỗ hổng nghiệp vụ nào của đáp án. Dựa trên bản đồ endpoint và tham số, đồ án kiểm chứng thủ công từng mục bằng các yêu cầu có chủ đích, và xác nhận được cả bốn: - **V07:** `/devlogin` chấp nhận `devadmin` với mật khẩu bất kỳ. - **V08 (API1):** người dùng dinesh chuyển được tiền *từ* tài khoản của jack. -

V09 (API2): `/changepassword` đổi mật khẩu của người khác mà không cần xác thực; thông báo lỗi của `/login` cho phép dò tên người dùng. - **V12:** mật khẩu dài một ký tự vẫn được chấp nhận.

Như vậy, quét tự động theo chữ ký chỉ phát hiện lỗi chèn mã và lỗi cấu hình. Lỗi phân quyền và lỗi luồng xác thực, vốn là trọng tâm của OWASP API Top 10, cần yêu cầu có chủ đích mà phân tích tĩnh giúp xác định. Thực nghiệm này chỉ dùng một ứng dụng có chủ đích với ZAP ở cấu hình mặc định. Burp Suite và Frida trong ma trận Chương 2 vẫn chưa được chạy. Dữ liệu nằm ở `03_benchmark/ket-qua-test/dynamic_ib/`.

3.8. Thảo luận và hạn chế

Bài học về phương pháp đánh giá. Kết quả quan trọng nhất của chương không phải một con số precision cụ thể, mà là khoảng cách giữa tập phát triển và holdout. Nếu chỉ đo trên các ứng dụng đã dùng để chỉnh luật, đồ án có thể báo cáo precision 40–63% và kết luận gần đạt mục tiêu. Giao thức holdout nhiều đợt cho thấy con số thật trên ứng dụng mới thấp hơn nhiều và phụ thuộc mạnh vào miền ứng dụng. Đồ án đề xuất rằng việc đánh giá công cụ phân tích tĩnh nên kết hợp một bộ đáp án có chủ đích (để đo recall) với nhiều đợt holdout trên ứng dụng thật (để đo precision).

Giới hạn của luật dựa trên mẫu. Các kỹ thuật ngữ cảnh giảm mạnh từng lớp báo nhầm đã biết, nhưng mỗi miền ứng dụng mới lại sinh lớp báo nhầm mới. Nhiều báo nhầm còn lại là hành vi có chủ đích (WebView cần JavaScript, người dùng chủ động xuất tệp) mà phân tích tĩnh không phân biệt được với lỗ hổng nếu không hiểu ngữ nghĩa ứng dụng.

Ảnh hưởng của quy ước gán nhãn. Precision tầng high thay đổi từ 80,6% xuống 9,7% chỉ vì một quy ước: coi cấu hình cleartext và tin CA người dùng ở cấp toàn ứng dụng là lỗ hổng hay hành vi có chủ đích. Mọi so sánh precision giữa các công cụ chỉ có ý nghĩa khi dùng cùng một quy ước gán nhãn được công bố rõ.

Hạn chế của nghiên cứu:

- *Phạm vi mã*: namespace không có thành phần nào khai báo trong Manifest (mã lõi của Briar và Conversations) không được tự suy và cần chỉ định bằng `--app-`

`pkg`. Trước khi có bộ suy phạm vi, precision của Nextcloud bị thổi phồng (44,4% với một phần ba mã so với 13,0% khi quét đủ phạm vi).

- *Mã đã tối ưu*: R8 [28] đổi tên hàm (hàm `replace` thành `p.g0`) và gộp mã thư viện vào package ứng dụng làm một số heuristic kết luận sai.
- *Phân tích luồng dữ liệu* chỉ trong một tệp; khoá khai báo ở lớp này và dùng ở lớp khác không được theo dõi.
- *Phân tích tĩnh không thấy lỗi phía máy chủ* (V08, V09, V12) và lỗi "vắng mặt cơ chế" (V03). Phần động (mục 3.7) mới thực hiện trên một ứng dụng có chủ đích; ZAP không tự phát hiện lỗi nghiệp vụ, mà các lỗi này phải kiểm chứng thủ công.
- *Recall trên ứng dụng thật mới đo trên 10 lỗ hổng đã công bố* (mục 3.6), thiên về thành phần exported; recall trên 19 ứng dụng của tập đánh giá vẫn chưa biết vì nhãn chỉ gán cho cảnh báo công cụ đưa ra.
- *Một người gán nhãn*, chưa đo độ đồng thuận giữa người gán; mỗi đợt holdout có ít cảnh báo (7–169). Bộ phiếu gán nhãn ẩn nhãn gốc (80 cảnh báo lấy mẫu phân tầng) và script tính Cohen's kappa [30] đã được chuẩn bị ở `03_benchmark/ground-truth/dong_thuan/` cho người gán thứ hai.
- *Đối chứng MobSF dựa trên mẫu*: precision của MobSF ước lượng từ 100/1.182 cảnh báo, do cùng một người gán nhãn.

Đạo đức. Thực nghiệm chỉ dùng ứng dụng có lỗ hổng có chủ đích và ứng dụng mã nguồn mở, chỉ phân tích tĩnh, không gửi yêu cầu tới máy chủ của bên thứ ba. Hai phát hiện mức nghiêm trọng cao được ẩn danh trong báo cáo cho tới khi nhà phát triển được thông báo theo quy trình công bố có trách nhiệm.

3.9. Kết luận chương

Chương 3 đã xây dựng và đánh giá công cụ `apiscan`. Về mặt công cụ, `apiscan` cho đầu ra dùng được bằng một lệnh: tự giải nén, tự suy phạm vi mã, áp dụng 20 luật với bảy kỹ thuật giảm báo nhầm theo ngữ cảnh, chấm độ tin cậy cho từng cảnh báo và xuất báo cáo HTML/JSON gắn nhãn OWASP trong khoảng một giây phân tích cho mỗi ứng dụng. Về ba câu hỏi nghiên cứu:

- **RQ1:** các kỹ thuật giảm báo nhầm giữ được recall 6/6 trên các lỗ hổng tĩnh của InsecureBankv2, kể cả khi lọc ở ngưỡng tin cậy ≥ medium.

- **RQ2:** precision trên tập phát triển (14–42% với mọi cảnh báo, 40–63% ở ngưỡng ≥ medium) không phản ánh precision trên ứng dụng mới; qua mười đợt holdout, precision dao động 2,8–40% và không hội tụ. Mục tiêu precision ≥ 50% trên ứng dụng mới không đạt.
- **RQ3:** điểm tin cậy theo từng cảnh báo lọc được khoảng 60% cảnh báo mà gần như không mất lỗ hổng thật (tầng low không chứa TP nào trên holdout đợt 7–9), nhưng không nâng precision ở ngưỡng ≥ medium lên mức mục tiêu.
- **Recall trên lỗ hổng đã công bố:** trên 10 lỗ hổng do GitHub Security Lab công bố, công cụ phát hiện đúng cơ chế 3/10 và trỏ đúng thành phần cho 7/10 còn lại, không bỏ sót hoàn toàn lỗ hổng nào; phần thiếu là các lỗi logic ở thành phần có lý do để exported.
- **Quét động có dẫn đường:** không có bản đồ endpoint, ZAP không quét được endpoint API nào của InsecureBankv2; có bản đồ và tham số do công cụ tự trích từ mã, ZAP tìm ra XSS phản xạ ở hai endpoint, và các lỗi V07, V08, V09, V12 được kiểm chứng thủ công.

So với MobSF trên cùng tập ứng dụng và cùng định nghĩa nhãn:

- Trên InsecureBankv2, `apiscan` đạt 6/6 mục tĩnh, MobSF ở cấu hình mặc định đạt 2/6.
- Trên ứng dụng thật, precision của `apiscan` là 17,2%, của MobSF ước lượng 5,0%. Số lỗ hổng thật tìm được tương đương nhau, nhưng `apiscan` đưa ra ít cảnh báo hơn khoảng 3,6 lần.

Bên cạnh công cụ, đồ án đóng góp một bộ dữ liệu đánh giá gồm 23 ứng dụng thật với 608 nhãn có lý do (thêm 100 nhãn cho mẫu cảnh báo MobSF), một giao thức đánh giá holdout nhiều đợt, và một số phát hiện bảo mật trên ứng dụng thật. Phần Kết luận chung sẽ tổng kết các kết quả này và nêu hướng phát triển: phân tích luồng dữ liệu xuyên lớp, tự suy phạm vi cho namespace không có thành phần khai báo và cho mã bị R8 dời vào package mặc định, và kết hợp kiểm thử động trên ứng dụng được phép.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết quả đạt được

Đồ án bắt đầu từ việc khảo sát các công cụ tự động phát hiện lỗ hổng bảo mật API của ứng dụng Android. Từ đó, đồ án xây dựng được một công cụ phân tích tĩnh có

đầu ra dùng được, kèm một phương pháp đánh giá công cụ trung thực. Đối chiếu với các mục tiêu đặt ra trong đề cương, đồ án đạt được các kết quả sau.

Thứ nhất, về tổng quan và môi trường thực nghiệm. Chương 1 hệ thống hoá: - cơ chế giao tiếp API của ứng dụng Android; - hai bộ tiêu chuẩn OWASP Mobile Top 10 (2024) và OWASP API Security Top 10 (2023); - ba phương pháp kiểm tra bảo mật; - bảy công cụ tự động tiêu biểu.

Chương 2 phân tích chi tiết ranh giới năng lực của từng công cụ, xây dựng môi trường thực nghiệm trên máy Apple Silicon và bộ đáp án 12 lỗ hổng của InsecureBankv2. Khi đối chiếu đáp án với mã nguồn thật, đồ án phát hiện bốn mục mô tả sai, trong đó mục V05 vốn được mô tả là DES nhưng thực tế là AES/CBC với khoá cứng và IV tĩnh. Các mục này đã được hiệu chỉnh trước khi dùng đáp án để chấm công cụ.

Thứ hai, về công cụ. Công cụ `apiscan` đi từ một tệp APK tới báo cáo HTML/JSON chỉ bằng một lệnh. Công cụ có các đặc điểm sau: - Tự giải nén và dịch ngược, rồi tự suy ra phạm vi mã của ứng dụng để tách khỏi mã thư viện. - Áp dụng 20 luật gắn nhãn OWASP. - Mỗi cảnh báo có bằng chứng `tệp:dòng`, mức nghiêm trọng và điểm tin cậy. - Báo cáo kèm bản đồ điểm cuối API và danh sách bề mặt tiếp nhận intent. - Bảy kỹ thuật giảm báo nhầm theo ngữ cảnh (mục 3.2.3) thay các luật so khớp mẫu thô bằng điều kiện mà một lỗ hổng thật phải thoả mãn. - Công cụ chỉ đọc tệp APK, không cài đặt, không chạy ứng dụng và không gửi yêu cầu tới máy chủ của ứng dụng. - Thời gian phân tích trung vị là 0,82 giây mỗi ứng dụng. - Quét hàng loạt nhiều APK kèm trang tổng hợp; cấu hình YAML để tắt luật, đặt ngưỡng và bỏ qua các báo nhầm đã xác nhận (kèm lý do, dựa trên dấu vân tay ổn định của cảnh báo). - Lệnh quét động dẫn đường cho OWASP ZAP bằng bản đồ endpoint tĩnh cùng phương thức và tên tham số tự trích từ mã (Apache HttpClient, Retrofit), và gộp kết quả thành báo cáo hợp nhất, có cổng an toàn chỉ cho phép quét máy chủ cục bộ hoặc có văn bản cho phép.

Thứ ba, về kết quả đánh giá định lượng. - Công cụ phát hiện đủ 6/6 lỗ hổng tĩnh của InsecureBankv2 ở mọi phiên bản luật, kể cả khi lọc ở ngưỡng tin cậy ≥ medium. - Trên 19 ứng dụng mã nguồn mở thực tế, phiên bản cuối đạt precision

17,2% với mọi cảnh báo, 39,1% ở ngưỡng ≥ medium và 80,6% ở tầng high. - Quét động có dẫn đường trên backend InsecureBankv2: không có bản đồ endpoint, ZAP không quét được endpoint API nào; có bản đồ và tham số từ phân tích tĩnh, ZAP đưa ra 27 cảnh báo, trong đó có XSS phản xạ ở hai endpoint (ngoài đáp án). Khi yêu cầu mẫu do công cụ tự trích từ mã thay cho yêu cầu viết tay, ZAP vẫn tìm ra đúng hai lỗi XSS này (26 cảnh báo). Các lỗi nghiệp vụ V07, V08, V09, V12 mà ZAP không tự phát hiện được đều được kiểm chứng thủ công. - Trên 10 lỗ hổng do GitHub Security Lab công bố ở 5 ứng dụng mã nguồn mở, công cụ phát hiện đúng cơ chế 3/10 (đều là provider exported thiếu permission) và trỏ đúng thành phần cho 7/10 còn lại (lỗi logic ở deep link, intent, AIDL); không lỗ hổng nào bị bỏ sót hoàn toàn. - Tầng tin cậy low hoạt động như một bộ lọc nhiễu: bỏ tầng này giảm 58,3% số cảnh báo, trong khi chỉ mất 3/57 lỗ hổng thật. - Đối chứng với MobSF trên cùng tập ứng dụng và cùng định nghĩa nhãn: - Trên InsecureBankv2, MobSF ở cấu hình mặc định chỉ đạt 2/6. - Precision của MobSF ước lượng 5,0% (khoảng tin cậy 95%: 2,2–11,2%), so với 17,2% của `apiscan` (13,5–21,7%). - Số lỗ hổng thật hai công cụ tìm được tương đương, nhưng `apiscan` đưa ra ít cảnh báo hơn khoảng 3,6 lần.

Thứ tư, về phương pháp và dữ liệu. Đồ án đóng góp: - Giao thức đánh giá holdout mười đợt: luật được đóng băng trước khi gặp ứng dụng mới. - Bộ dữ liệu 608 nhãn đúng/sai có ghi lý do trên 23 ứng dụng, cùng 100 nhãn cho mẫu cảnh báo của MobSF. Bộ dữ liệu tái lập được để đánh giá các công cụ khác.

Kết quả quan trọng nhất về phương pháp là precision đo trên tập ứng dụng dùng để chỉnh luật không phản ánh precision trên ứng dụng mới. Trên tập phát triển, precision ở ngưỡng ≥ medium là 40–63%, trong khi trên mười đợt holdout precision chỉ dao động 2,8–40% và không hội tụ.

Thứ năm, về giá trị thực tế. Khi rà soát cảnh báo của công cụ trên ứng dụng thật, đồ án ghi nhận một số phát hiện bảo mật: - Hai lỗ hổng mức nghiêm trọng cao ở thành phần exported không được bảo vệ, đang được công bố có trách nhiệm cho nhà phát triển. - Activity của thư viện cắt ảnh bị exported. - Công cụ debug lọt vào bản phát hành. - Khoá API nhúng cứng.

Từ đó, đồ án đề xuất quy trình sử dụng công cụ trong kiểm thử bảo mật API gồm ba bước: 1. Sàng lọc tĩnh tự động bằng `apiscan` ở ngưỡng ≥ medium. 2. Chuyên gia rà soát theo thứ tự tầng tin cậy: high trước, medium sau. 3. Kiểm thử động để xác nhận, chỉ khi có văn bản cho phép của chủ ứng dụng.

Hạn chế

- **Mục tiêu precision ≥ 50% trên ứng dụng mới không đạt.** Precision trên holdout phụ thuộc mạnh vào miền ứng dụng, vì mỗi miền mới sinh ra một lớp báo nhầm mà luật dựa trên mẫu chưa gặp.
- **Kết quả precision phụ thuộc quy ước gán nhãn.** Nếu cấu hình cleartext và tin chứng chỉ do người dùng cài ở cấp toàn ứng dụng được coi là hành vi có chủ đích, precision tầng high giảm từ 80,6% xuống 9,7%.
- **Chỉ một người gán nhãn**, chưa đo độ đồng thuận giữa các người gán (bộ phiếu và script tính kappa đã chuẩn bị sẵn). Recall trên ứng dụng thật mới đo được trên 10 lỗ hổng đã công bố, thiên về thành phần exported; recall của các luật mật mã, lưu trữ, ghi log chưa đo.
- **Lỗi logic ở bề mặt tiếp nhận intent chỉ được chỉ ra, không được cảnh báo**: 7/10 lỗ hổng đã công bố thuộc nhóm này.
- **Phân tích còn hạn chế ở ba điểm.** Luồng dữ liệu chỉ được phân tích trong phạm vi một tệp. Phạm vi mã chưa nhận ra mã ứng dụng bị R8 dời vào package mặc định; đối chứng với MobSF đã chỉ ra một lỗ hổng thật bị bỏ sót vì lý do này. Công cụ cũng không phát hiện được lỗ hổng phía máy chủ (V08, V09, V12) hay lỗi "vắng mặt cơ chế" như thiếu certificate pinning (V03).
- **Phần benchmark động chưa thực hiện.** Khi đồ án chuyển trọng tâm sang xây dựng công cụ, phần benchmark bảy công cụ theo ma trận kiểm thử ở Chương 2 chỉ được thực hiện với nhóm công cụ tĩnh: JADX, apktool và phân tích tĩnh của MobSF. Các ca kiểm thử động với Burp Suite và Frida chưa được chạy. Phần ZAP đã chạy qua lệnh `dynamic` của `apiscan` trên backend InsecureBankv2 (mục 3.7), nhưng chỉ trên một ứng dụng có chủ đích với cấu hình mặc định.

Hướng phát triển

- Mở rộng phân tích luồng dữ liệu xuyên lớp để theo dõi khoá, mật khẩu và URL được khai báo ở lớp này nhưng dùng ở lớp khác.
- Kết luận hai quy tắc CFG-EXPORT mà phép đo recall chỉ ra. Holdout đợt 10 chưa đủ cảnh báo để kết luận. Cần một đợt holdout có nhiều thành phần

exported hơn, và không coi action thuộc namespace thư viện (`androidx.`, `org.unifiedpush.`) là action tuỳ biến của ứng dụng.

- Cải tiến bộ suy phạm vi mã cho hai trường hợp: namespace không có thành phần khai báo trong Manifest, và mã bị R8 dời vào package mặc định.
- Mở rộng bộ trích yêu cầu HTTP sang OkHttp và mã đã bị R8 làm rối, và trích cả giá trị tham số khi có thể. Thực hiện phần Burp Suite, Frida của ma trận Chương 2. Sau đó đưa kiểm thử động vào quy trình, chỉ áp dụng cho ứng dụng có sự cho phép của chủ sở hữu.
- Mời thêm người gán nhãn độc lập để đo độ đồng thuận, và mở rộng tập holdout sang các miền ứng dụng chưa có (ngân hàng, thương mại điện tử) với ứng dụng được phép phân tích.

TÀI LIỆU THAM KHẢO

[1]. OWASP Foundation. OWASP Mobile Top 10 – 2024 (Final Release). Online: https://owasp.org/www-project-mobile-top-10/2023-risks/ [Truy cập: 20/8/2026].

[2]. OWASP Foundation. OWASP API Security Top 10 – 2023. Online: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ [Truy cập: 20/8/2026].

[3]. OWASP Foundation. OWASP Mobile Application Security Testing Guide (MASTG). Online: https://mas.owasp.org/MASTG/ [Truy cập: 20/8/2026].

[4]. OWASP Foundation. OWASP Mobile Application Security Verification Standard (MASVS). Online: https://mas.owasp.org/MASVS/ [Truy cập: 20/8/2026].

[5]. Statista Research Department. Mobile operating systems' market share worldwide. Statista, 2024. Online: https://www.statista.com/statistics/272698/global-market-share-held-by-mobile-operating-systems/ [Truy cập: 20/8/2026].

[6]. Google. Network security configuration — Android Developers. Online: https://developer.android.com/privacy-and-security/security-config [Truy cập: 20/8/2026].

[7]. Google. App security best practices — Android Developers. Online: https://developer.android.com/privacy-and-security/security-best-practices [Truy cập: 20/8/2026].

[8]. Rescorla, E. RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. Internet Engineering Task Force, 8/2018. Online: https://www.rfc-editor.org/rfc/rfc8446.html [Truy cập: 20/8/2026].

[9]. OWASP Foundation. Pinning Cheat Sheet — OWASP Cheat Sheet Series. Online: https://cheatsheetseries.owasp.org/cheatsheets/Pinning_Cheat_Sheet.html [Truy cập: 20/8/2026].

[10]. MobSF Development Team. Mobile Security Framework (MobSF). GitHub, 2024. Online: https://github.com/MobSF/Mobile-Security-Framework-MobSF [Truy cập: 20/8/2026].

[11]. PortSwigger Ltd. Burp Suite Documentation — Community Edition. Online: https://portswigger.net/burp/documentation/community [Truy cập: 20/8/2026].

[12]. OWASP ZAP Team. OWASP Zed Attack Proxy (ZAP) — User Guide. Online: https://www.zaproxy.org/docs/ [Truy cập: 20/8/2026].

[13]. Ravnås, O. A. và cộng sự. Frida — Dynamic Instrumentation Toolkit for Developers, Reverse-Engineers, and Security Researchers. Online: https://frida.re/docs/ [Truy cập: 20/8/2026].

[14]. Sensepost. objection — Runtime Mobile Exploration. GitHub. Online: https://github.com/sensepost/objection [Truy cập: 20/8/2026].

[15]. Skylot. JADX — Dex to Java Decompiler. GitHub. Online: https://github.com/skylot/jadx [Truy cập: 20/8/2026].

[16]. iBotPeaches. Apktool — A Tool for Reverse Engineering Android APK Files. Online: https://apktool.org/ [Truy cập: 20/8/2026].

[17]. Shetty, D. Android-InsecureBankv2. GitHub, 2023. Online: https://github.com/dineshshetty/Android-InsecureBankv2 [Truy cập: 20/8/2026].

[18]. Mitra, J., Ranganath, V.-P. Ghera: A Repository of Android App Vulnerability Benchmarks. Proceedings of the 13th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE), 2017. Online: https://arxiv.org/abs/1708.02380 [Truy cập: 20/8/2026].

[19]. Ranganath, V.-P., Mitra, J. Are Free Android App Security Analysis Tools Effective in Detecting Known Vulnerabilities?. Empirical Software Engineering, Vol. 25, 2020, pp. 178–219. DOI: 10.1007/s10664-019-09749-y. Online: https://arxiv.org/abs/1806.09059 [Truy cập: 20/8/2026].

[20]. Armas Cabrera, J. và cộng sự. An Android Applications Vulnerability Analysis Using MobSF. IEEE, 2024. Online: https://ieeexplore.ieee.org/document/10581312/ [Truy cập: 20/8/2026].

[21]. Four Ways to Bypass Android SSL Verification and Certificate Pinning. Research publication, ResearchGate. Online: https://www.researchgate.net/publication/347893088 [Truy cập: 20/8/2026].

[22]. Google. Run apps on the Android Emulator — Android Developers. Online: https://developer.android.com/studio/run/emulator [Truy cập: 15/9/2026].

[23]. Docker Inc. Docker Desktop for Mac with Apple Silicon — Documentation. Online: https://docs.docker.com/desktop/setup/install/mac-install/ [Truy cập: 15/9/2026].

[24]. Pallets Projects. Flask — Web Development, One Drop at a Time. Online: https://flask.palletsprojects.com/ [Truy cập: 15/9/2026].

[25]. Wohlin, C. và cộng sự. Experimentation in Software Engineering. Springer, 2012 — chương về các mối đe doạ tới tính hợp lệ của nghiên cứu thực nghiệm.

[26]. F-Droid. F-Droid — Free and Open Source Android App Repository. Online: https://f-droid.org/ [Truy cập: 4/10/2026].

[27]. Wilson, E. B. Probable Inference, the Law of Succession, and Statistical Inference. Journal of the American Statistical Association, Vol. 22, No. 158, 1927, pp. 209–212.

[28]. Google. Shrink, obfuscate, and optimize your app — Android Developers. Online: https://developer.android.com/build/shrink-code [Truy cập: 4/10/2026].

[29]. GitHub Security Lab. Advisories. Online: https://securitylab.github.com/advisories/ [Truy cập: 8/11/2026].

[30]. Cohen, J. A Coefficient of Agreement for Nominal Scales. Educational and Psychological Measurement, 20(1), 37–46, 1960.

PHỤ LỤC

Phụ lục A. Kịch bản Frida vô hiệu hoá ghim chứng chỉ

Kịch bản dưới đây móc vào nhiều điểm kiểm tra chứng chỉ khác nhau của nền tảng Android và của thư viện OkHttp, nhằm bao phủ các cách ghim chứng chỉ phổ biến trong một lần chạy. Đây là bản rút gọn dùng cho ca kiểm thử TC-03.

```
Java.perform(function () {
    // 1. Vo hieu hoa TrustManager mac dinh cua nen tang
    var TMImpl = Java.use('com.android.org.conscrypt.TrustManagerImpl');
    TMImpl.verifyChain.implementation = function (chain, anchor, host,
            clientAuth, ocsp, tlsSct) {
        console.log('[+] verifyChain bi bo qua cho: ' + host);
        return chain;
    };

    // 2. Vo hieu hoa CertificatePinner cua OkHttp neu co
    try {
        var Pinner = Java.use('okhttp3.CertificatePinner');
        Pinner.check.overload('java.lang.String', 'java.util.List')
            .implementation = function (host, peerCerts) {
                console.log('[+] OkHttp pinning bi bo qua cho: ' + host);
                return;
            };
    } catch (e) {
        console.log('[-] Ung dung khong dung OkHttp CertificatePinner');
    }
});
```

Khối lệnh thứ nhất xử lý trường hợp ứng dụng dựa vào cơ chế kiểm tra chứng chỉ mặc định của hệ thống; khối lệnh thứ hai xử lý trường hợp ứng dụng dùng lớp ghim chứng chỉ riêng của thư viện OkHttp. Khối lệnh thứ hai được bọc trong cấu trúc bắt lỗi vì không phải ứng dụng nào cũng dùng OkHttp; nếu lớp không tồn tại, kịch bản in ra thông báo và tiếp tục thay vì dừng lại.

Phụ lục B. Các lệnh triển khai môi trường lab

Phụ lục này tập hợp các lệnh chính dùng để triển khai môi trường, theo đúng trình tự mười bước ở Bảng 2.13. Các giá trị cụ thể như tên ảnh hoặc khoá API được thay bằng chỗ trống trong ngoặc nhọn.

```
# Buoc 1 - Khoi chay may chu phia sau
cd InsecureBankv2/AndroLabServer && pip install -r requirements.txt
python app.py           # lang nghe tai cong 8888

# Buoc 2 - Khoi dong may ao va nang quyen
emulator -avd Pixel6_API33_arm64 -writable-system &
adb root && adb shell id  # xac nhan tra ve uid=0

# Buoc 3 - Cai ung dung dich
adb install InsecureBankv2.apk

# Buoc 4-5 - MobSF va OWASP ZAP qua Docker
docker run -p 8000:8000 opensecurity/mobile-security-framework-mobsf:latest
docker run -p 8090:8090 zaproxy/zap-stable zap.sh -daemon \
   -host 0.0.0.0 -port 8090 -config api.key=<khoa_api>

# Buoc 7 - Cai chung chi CA vao kho he thong
openssl x509 -inform DER -in burp.der -out burp.pem
HASH=$(openssl x509 -inform PEM -subject_hash_old -in burp.pem | head -1)
adb push burp.pem /system/etc/security/cacerts/$HASH.0
adb shell chmod 644 /system/etc/security/cacerts/$HASH.0

# Buoc 8-9 - frida-server va objection
adb push frida-server-arm64 /data/local/tmp/frida-server
adb shell 'chmod 755 /data/local/tmp/frida-server'
adb shell '/data/local/tmp/frida-server &'
objection --gadget com.android.insecurebankv2 explore
```

Phụ lục C. Cấu trúc thư mục lưu kết quả

Toàn bộ đầu ra của thực nghiệm được tổ chức theo cấu trúc thư mục cố định dưới đây, nhằm bảo đảm mỗi kết quả đều truy ngược được về công cụ, lượt chạy và minh chứng tương ứng.

```
results/
├── mobsf/
│   ├── run1.json     run2.json     run3.json
│   └── report.pdf
├── zap/
│   ├── run1.json     run2.json     run3.json
│   └── report.html
├── burp/
│   ├── findings.json         # ghi thu cong theo luoc do chung
│   └── screenshots/          # anh minh chung cho tung ca
├── frida/        objection/        jadx/        apktool/
├── benchmark_summary.csv       # bay chi so x bay cong cu
└── run_benchmark.py          # kich ban dieu phoi
```

Phụ lục D. Mẫu phiếu ghi kết quả một ca kiểm thử

Mỗi ca kiểm thử được ghi lại theo phiếu mẫu dưới đây. Phiếu được thiết kế để một người khác đọc lại có thể hiểu chính xác điều gì đã được kiểm tra và vì sao kết quả được gán nhãn như vậy, mà không cần hỏi lại người thực hiện.

Bảng D.1. Mẫu phiếu ghi kết quả một ca kiểm thử

Trường	Nội dung ghi
Mã ca kiểm thử	TC-08
Lỗ hổng	V08 – Phân quyền mức đối tượng (OWASP API1)
Công cụ	Burp Suite Community, chức năng Repeater
Phiên bản công cụ	Ghi rõ số hiệu phiên bản tại thời điểm chạy
Lượt chạy	1 / 2 / 3
Kết quả quan sát	Máy chủ trả về 200 OK cho giao dịch với tài khoản nguồn của người dùng khác
Nhãn	Phát hiện đúng (TP)
Minh chứng	results/burp/screenshots/tc08_run1.png
Ghi chú	Đường cơ sở ở bước 3 trả về thành công; phép thử ở bước 6 xác nhận lỗ hổng

Tập hợp các phiếu như trên, mỗi ca một phiếu cho mỗi công cụ và mỗi lượt chạy, là dữ liệu thô của toàn bộ nghiên cứu. Từ tập phiếu này, bảng tổng hợp bảy tiêu chí ở Chương 3 được lập một cách máy móc, không cần thêm phán đoán nào — đó chính là mục tiêu của việc cố định toàn bộ khung đo trong Chương 2.